NSSECU3 Digital Forensics AY2526T2

Laboratory Activity Report (LAR) / Digital Forensics Report (DFR) / Mini Project (MP) / Write-Up (WUP) / Windows Forensics Project (WFP)

MINI PROJECT 3

Document Number:	MP-03
Approved By:	Jerome E. Gutierrez NSSECU3 Course Instructor
Date Approved:	10 March 2026

Submitted By:	Miguel Sebastian C. Carlos
Group Members:	Luis Gabriel Sebastian Biacora

©Copyright 2026, m1rr4wr
All Rights Reserved
Any hard copy or unprotected soft copy of this document shall be regarded as uncontrolled copy.

Jose Angelo C. Calpoporo
Miguel Sebastian C. Carlos
Danel Gavrie Y. Clemente

Date Version: 10 March 2026
Version: 1.0

Document Type: MP = Mini Project
Data Classification: Confidential: Students of this course, Professors of the Department, and other guest speakers and students may use this document and view the contents.

Table of Contents

Task Distribution Summary 5
- Change Control 5
- Task List 5

Member's Task Lists 7
- Detailed Task List by Biacora 7
- Detailed Task List by Calpoporo 7
- Detailed Task List by Carlos 7
- Detailed Task List by Clemente 8

Purpose 9
Scope 9
Definitions and Acronyms 10
Roles and Responsibilities 11
Materials and Equipment (if applicable) 12
Procedure Steps (These steps should focus on the goal/purpose and not the installation and setup) 14
- Step 1: Image Acquisition 14
- Step 2: Mounting the Image with Arsenal Image Mounter 14
- Step 3: Extracting the SAM Hive 15
- Step 4: Running sam_parser.py 18
- Step 5: Verification with Registry Explorer 22
- Step 6: Creating the Excel Colour-Coded Byte Map 23

Code Design 28
- Overview 28
- Module Architecture 28
- Registry Cell Navigation 30
- NK Cells 30
- Subkey Lists 30

Value Lists 31
VK Cells 31
F Value Offset Correction 31
Subkey List Handling 32
V Value Parsing 32
Error Handling and Robustness 32
Top-Level Algorithm Flow 32
Summary 34
System Design 34
Workflow Overview 34
Detailed Phase Descriptions 35
Phase 1: Image Acquisition 35
Phase 2: Image Mounting 35
Phase 3: Hive Extraction 35
Phase 4: Binary Parsing 35
Phase 5: Verification & Documentation 36
Data Flow Diagram 36
Environment 37
Results and Analysis 39
Hive Header Summary 39
Parsed User Accounts 40
Analysis of Findings 42
RID 500 – Administrator 42
RID 501 – Guest 42
RID 503 – DefaultAccount 42
RID 504 – WDAGUtilityAccount 43
RID 1000 – Angelo Calpoporo (Active User Account) 43
Verification Against Registry Explorer 44
Interpretation of Verification Results 45
Security Observations 45
Demonstration of Low-Level Forensic Value 46
Safety or Compliance Considerations (if applicable) 47
References 48
Appendices 49
Appendix A – sam_parser.py Source Code 49

Appendix B – SAM HIVE MAP 66
B.1 Overview of all users 66
B.2 Legend and Color Guide 67
B.3 F and V Value Hex Dump and Field Map (Administrator, RID 500) 68
B.4 F and V Value Hex Dump and Field Map (Guest, RID 501) 69
B.5 F and V Value Hex Dump and Field Map (DefaultAccount, RID 503) 70
B.6 F and V Value Hex Dump and Field Map (WDAGUtility, RID 504) 71
B.7 F and V Value Hex Dump and Field Map (Angelo Calpoporo, RID 1000) 72
Appendix C – Output 73
C.1 Registry Hive Header Summary 73
C.2 HBIN Block Enumeration 74
C.3 NK Record Scan Summary 74
C.4 Direct Search for Important Registry Names 75
C.5 RID-Style Key Discovery 75
C.6 Likely User and Group Names 76
C.7 Extraction of User Account Records from `SAM\Domains\Account\Users` 77
C.8 Parsed User Account – RID 500 (Administrator) 78
C.9 Parsed User Account – RID 501 (Guest) 79
C.10 Parsed User Account – RID 503 (DefaultAccount) 80
C.11 Parsed User Account – RID 504 (WDAGUtilityAccount) 82
C.12 Parsed User Account – RID 1000 (Angelo Calpoporo) 84

Task Distribution Summary

Table 1. Task Distribution Summary

Description	Summarizes all the checklist activities given by the instructor as well as the specific tasks each student was assigned and checked by their groupmates approval.
Start of Activity	3 March 2026
End of Activity	10 March 2026
Main Contributor	Jose Angelo C. Calpoporo

Change Control

Table 2 Change Control

Version	Changes	Date	Changed by
1.0	Final Mini Project Compiled	10 Mar 26	All members

Task List

Table 3. Task List (Overall)

#	Item	Purpose	Assignee	Status
1	Retrieve disk image from MP1/MP2	Obtain the victim machine image for analysis	Calpoporo	Done
2	Mount image using AIM/FTK Imager on Windows 11	Make the file system accessible for hive extraction	Calpoporo	Done
3	Extract SAM hive from mounted image	Obtain the registry hive containing user credentials	Calpoporo	Done

4	Research SANS FOR500 Red Poster (Windows Forensics)	Understand the structure and artifacts of the SAM hive	Biacora	Done
5	Research related literature (RRL) – at least 2 sources	Provide academic/theoretical background for the analysis	Carlos	Done
6	Develop Python script to parse SAM hive	Automate extraction of user account details, RID, hashes, etc.	Carlos	Done
7	Map raw bytes of SAM hive in Excel with colorcoding	Visualise fields, flags, and magic numbers as discussed in Module 4	Carlos	Done
8	Write documentation: Purpose, Scope, Procedure Steps	Describe the objectives and the stepbystep process	Biacora	Done
9	Write documentation: Code Design, System Design, Appendices	Explain parsing logic and acquisition methodology; include full code	Clemente	Done
10	Integrate RRL into documentation	Cite and discuss the two required references	Carlos	Done
11	Review and finalise report	Ensure all sections are complete and consistent	Carlos	Done
12	Submit final documentation	Turn in the completed Mini Project 3 report	Carlos	Done

Member's Task Lists

Detailed Task List by Biacora

Table 4. Detailed Task List by Biacora

#	Item	Purpose	Section added	Status
1	Checking of Documents	To ensure all information are correctly placed	Entire Document	Done
2	Inputting of Screenshots	To ensure there is proof of work	Entire Document	Done

Detailed Task List by Calpoporo

#	Item	Purpose	Section added	Status
1	Retrieve disk image from MP1/MP2	Obtain the victim machine image required for forensic analysis and ensure the original evidence source is available for further investigation.	Procedure Steps	Done
2	Mount image using AIM/FTK Imager on Windows 11	Make the forensic disk image accessible in a read-only environment so the registry files and system artifacts can be safely examined without altering the evidence.	Procedure Steps	Done
3	Extract SAM hive from mounted image	Locate and extract the Security Account Manager (SAM) registry hive which stores user account information and authentication data.	Procedure Steps	Done
4	Develop Python script to parse SAM hive	Automate the extraction and parsing of user account artifacts such as RID values, account flags, password hashes, timestamps, and metadata from the registry hive.	Appendix A	Done
5	Windows VM Tools	To install the tools needed for the victim machine to be acting as the "user who was attacked and became the victim"	Procedure Steps	Done

Detailed Task List by Carlos

#	Item	Purpose	Section added	Status
1	Map raw bytes of SAM hive in Excel with color-coding	Visualize registry structure by highlighting magic numbers, offsets, flags, timestamps, user account names, RID values, and hash fields for easier forensic interpretation.	Appendix B	Done
2	Review and finalise report	Ensure the forensic report is complete, technically accurate, properly formatted,	Entire Document	Done

		and consistent across all sections before submission.		
3	Submit final documentation	Compile all project outputs including scripts, analysis, and report into the final Mini Project 3 submission package.	Entire Document	Done

Detailed Task List by Clemente

#	Item	Purpose	Section added	Status
1	Header and Footer Consistency	To have a software for virtualization	Entire Document	Done
2	Document Template Alignment	Verify document title, document number, date version, and classification are consistent throughout	Header and Footer	Done
3	Screenshot Labeling and Indexing	Ensure that all required screenshots are properly labeled and indexed.	Entire Document	Done
4	Grammar, Clarity, and Technical Consistency Review	Improve readability without altering technical meaning	Entire Document	Done

Purpose

The purpose of this Mini Project is to develop a custom Python tool that parses the Windows SAM registry hive extracted from a forensic image of a victim machine. By understanding the binary structure of the SAM hive (headers, HBIN blocks, NK records, VK records, and the F/V values), the tool extracts critical user account information such as RID, username, last logon timestamps, password change dates, account flags, and logon counts. The project also produces hex dumps and field-level offset maps that can be used for manual byte-level analysis in Excel, aligning with the SANS FOR500 Windows Forensic Analysis poster's guidance on account usage artifacts.

Scope

Included

- Acquisition of the SAM hive from the forensic disk image (mounted from MP1/MP2) using Arsenal Image Mounter and FTK Imager in read-only mode.
- Binary parsing of the SAM hive file (SAM_copy) using a custom Python script implementing: regf header validation, HBIN block enumeration, NK cell traversal, lf/lh/li/ri subkey list processing, VK value cell parsing, F value field decoding (RID, FILETIME timestamps, ACB flags, logon/fail counts), and V value field decoding (username in UTF-16LE, NT hash blob, LM hash blob).
- Generation of hex dumps and field-level byte offset maps for all 5 user accounts found in the SAM: Administrator (RID 500), Guest (RID 501), DefaultAccount (RID 503), WDAGUtilityAccount (RID 504), and Angelo Calpoporo (RID 1000).
- Production of a colour-coded Excel workbook (SAM_Hive_ByteMap.xlsx) mapping each byte of the F and V values to its corresponding field.
- Verification of all parsed output against Registry Explorer.
- Documentation of the complete forensic workflow in this Laboratory Activity Report.

Excluded

- Decryption of the NT or LM hash blobs. The SAM hive encrypts password hashes using RC4 or AES with a key derived from the Syskey (SYSKEY/bootkey). Syskey derivation and hash decryption are outside the scope of this project.
- Parsing of registry hives other than SAM (e.g., SYSTEM, SOFTWARE, NTUSER.DAT).
- Password cracking or any attempt to determine plaintext credentials.
- Analysis of group membership (SAM\Domains\Builtin) or machine account entries.
- Live forensic acquisition; all analysis was conducted offline on a forensic copy.

Applicability:
This procedure applies to all student groups enrolled in NSSECU3 Digital Forensics AY2526T2 who are required to complete Mini Project 3. The document serves as the official reference for laboratory work conducted in the designated forensic analysis workstations and remote learning environments.

Definitions and Acronyms

ACB Flags	Account Control Bitmask – a 32-bit flag field stored at offset +0x3C in the F value cell that encodes account properties (e.g., ACCOUNTDISABLE = 0x0001, PASSWORD_NOT_REQUIRED = 0x0004, NORMAL_ACCOUNT = 0x0010, DONT_EXPIRE_PASSWORD = 0x0200).
AIM	Arsenal Image Mounter – a Windows forensic tool used to mount disk images as virtual disks for examination.
F Value	A binary value stored under each user's RID subkey in SAM\Domains\Account\Users\<RID> containing account metadata: logon timestamps, account expiry, ACB flags, RID, logon counts, and failed logon data. Parsed per SANS FOR500 specification.
FILETIME	A 64-bit Windows timestamp representing the number of 100-nanosecond intervals since 1 January 1601 (UTC), stored little-endian. Used for Last Logon, Last Password Set, Account Expires, and Last Failed Logon fields.
FTK Imager	Forensic Toolkit Imager – a tool used to acquire and examine forensic disk images.
HBIN	Host Bin – a 4,096-byte (one page) allocation unit of the Windows registry hive, beginning with the 4-byte magic 'hbin'. HBIN blocks form the allocation pool from which cells are carved.
LAR	Laboratory Activity Report – the formal project report document submitted for grading.
lf / lh / li / ri	Subkey list cell types in the Windows registry. lf = leaf with hash, lh = leaf with hash (Vista+), li = leaf index, ri = root index (indirect list). Each stores an array of relative offsets to NK cells.
LM Hash	LAN Manager Hash – a legacy Windows password hash (DES-based). Disabled by default on Windows Vista and later but the hash blob slot may still exist in the SAM.
NK Cell	Named Key cell – the primary record for a registry key node, containing the key name, timestamps, parent offset, and pointers to subkey lists and value lists.

NT Hash	NTLM Hash – the MD4 hash of the Unicode password, used for Windows authentication. Stored encrypted in the SAM V value.
regf	Registry File magic bytes (0x72 0x65 0x67 0x66). The first 4 bytes of any valid Windows registry hive file, identifying it as a structured binary registry.
RID	Relative Identifier – the last sub-authority of a Windows Security Identifier (SID), used to uniquely identify a local user or group account. Example: RID 500 = built-in Administrator.
RRL	Related References and Literature – academic or technical sources cited in the report.
SAM	Security Account Manager – the Windows registry hive (\HKLM\SAM) that stores local user account information including account metadata and encrypted password hashes.
SANS FOR500	SANS Digital Forensics and Incident Response course FOR500 (Windows Forensic Analysis). The associated Red Poster provides byte-level field-offset specifications for SAM F and V values.
SID	Security Identifier – a unique alphanumeric identifier assigned to users, groups, and computers in Windows. Format: S-1-5-21-<domain>-<RID>.
V Value	A binary value stored alongside the F value under each user RID key in the SAM, containing the username (UTF-16LE) and encrypted NT/LM hash blobs. The data is accessed via a structured pointer table with offsets relative to 0xCC bytes from the start of the value.
VK Cell	Value Key cell – the registry record for a single value, containing the value name length, data length, data offset, and data type.

Roles and Responsibilities

Luis Gabriel Sebastian Biacora	System Design Lead / QA	End-to-end workflow documentation; review and quality assurance.
Jose Angelo C. Calpoporo	Parser Developer / Forensic Analyst	SAM acquisition, binary parser implementation, code design documentation.

Miguel Sebastian C. Carlos	Excel Mapping / Integration Engineer	Colour-coded byte-mapping Excel workbook; parser integration and testing.
Danel Gavrie Y. Clemente	Researcher / Technical Writer	RRL compilation, safety considerations, proofreading, LAR formatting.

Materials and Equipment (if applicable)

#	Item	Version / Spec	Purpose
1	Forensic Disk Image	MP1/MP2 artifact	Source disk image of Windows 10 victim machine from previous mini projects.
2	Arsenal Image Mounter (AIM)	v3.x	Mount the forensic disk image as a virtual volume in read-only (write-blocked) mode on a Windows 11 analysis workstation.
3	FTK Imager	v4.7.x	Navigate the mounted volume's file system and extract the SAM hive to a working copy (SAM_copy).
4	Python 3.11+	3.11 / 3.12	Runtime environment for sam_parser.py.
5	sam_parser.py	Custom code	Custom binary parser implementing regf/hbin/nk/vk cell traversal and F/V value field decoding.
6	Registry Explorer	v2.0.0	Independent verification tool to cross-check parsed output against a trusted Windows registry viewer.
7	Microsoft Excel / openpyxl	Excel 365 / openpyxl 3.1	Produce colour-coded hex dump byte-mapping workbook (SAM_Hive_ByteMap.xlsx).

8	SAM_Hive_ByteMap.xlsx	-	7-sheet Excel workbook with per-user F/V hex dumps, field decode tables, and legend.
9	SANS FOR500 Red Poster	v4.18 (2024)	Field-offset specification reference for SAM F value and V value structures.
10	Visual Studio Code	v1.87+	Integrated development environment for parser code editing and debugging.

Procedure Steps (These steps should focus on the goal/purpose and not the installation and setup)

This section documents each step of the forensic workflow, from obtaining the disk image through to output verification. Steps reference the corresponding code and output in the Appendices.

Step 1: Image Acquisition

The forensic disk image used in this project was acquired during Mini Projects 1 and 2 using FTK Imager. The image is a physical sector-by-sector copy of the victim Windows 10 machine's primary drive, stored in EnCase (E01) format as MP.E01. Before any analysis, the integrity of the image was verified using FTK Imager's built-in verification tool. The computed MD5 and SHA-1 hashes matched the stored verification hashes, and no bad sectors were detected, confirming that the image is an exact, unaltered copy of the original evidence.

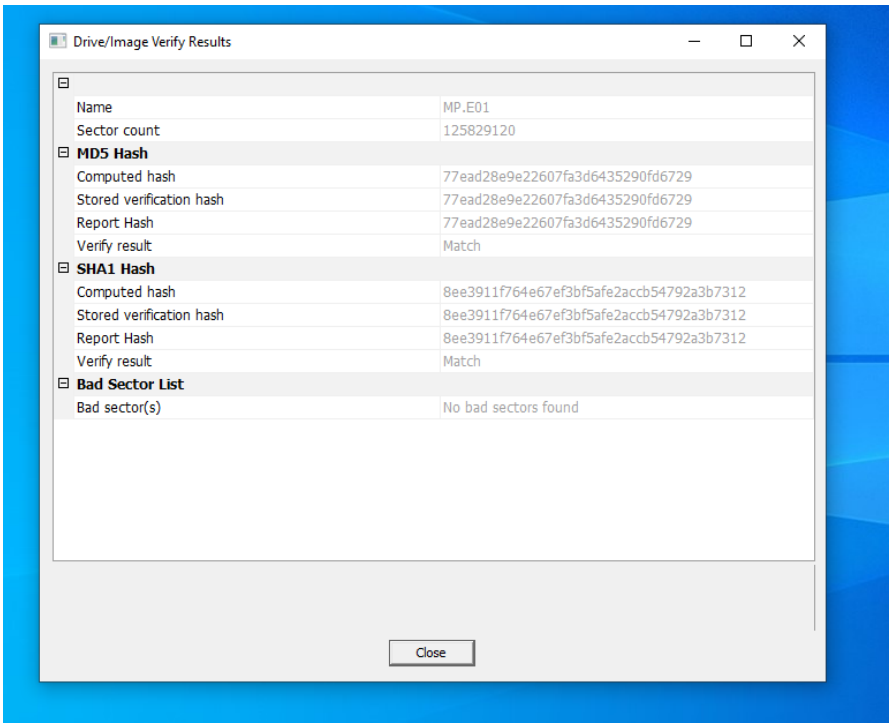

Figure 1: FTK Imager – Image verification results showing matching MD5 and SHA-1 hashes and no bad sectors

Step 2: Mounting the Image with Arsenal Image Mounter

- Launch **Arsenal Image Mounter (AIM)** on the Windows 11 forensic workstation (run as Administrator).
- Click the **Mount disk image** button and navigate to the location of the forensic image file (MP.E01) from the MP1/MP2 acquisition folder.

- In the mount options dialog, ensure **Read-Only** mode is selected – this critical forensic safeguard prevents any write operations to the image, preserving its integrity.
- AIM mounts the image as a virtual disk drive. Confirm the assigned drive letter (in this case, **F:**) appears in the AIM window and in Windows File Explorer.
- The mounted volume is now accessible for browsing, but no modifications should be made – all subsequent file access will be performed in read-only mode.

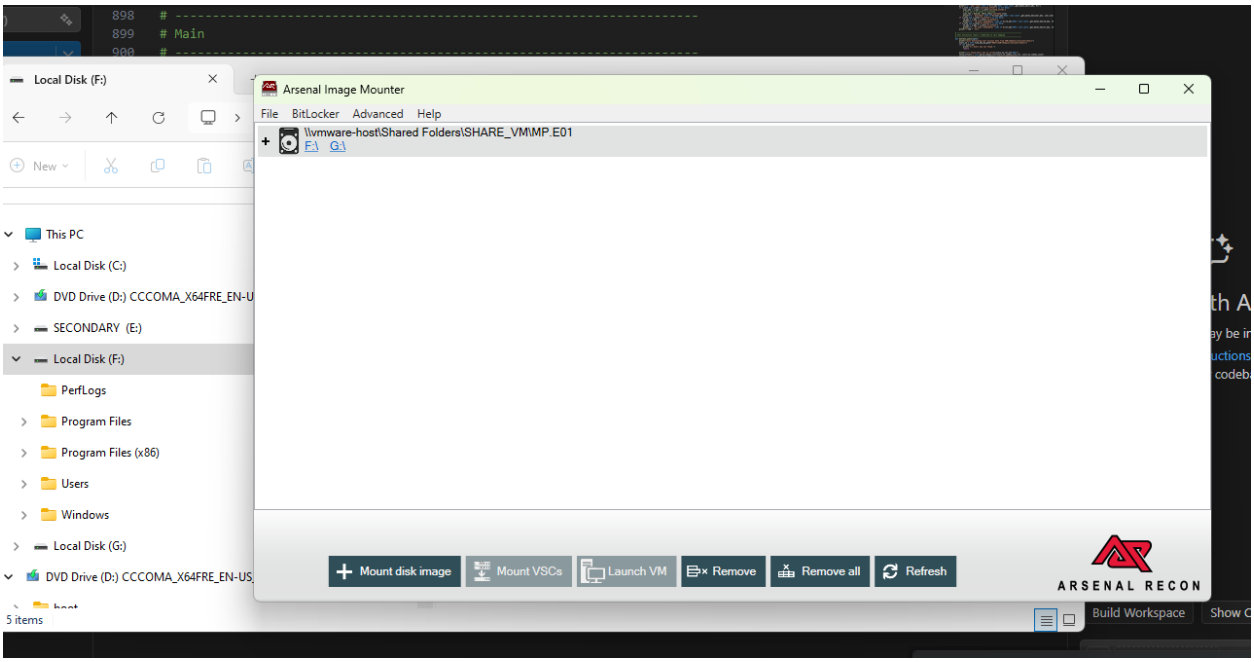

Figure 2: Arsenal Image Mounter – Mounting the forensic image as a read-only virtual drive (F:)

Step 3: Extracting the SAM Hive

Once the forensic image is successfully mounted as a virtual drive (e.g., F:), the SAM hive must be extracted to a working directory for analysis. There are two methods to accomplish this, depending on whether Windows File Explorer can directly access the file or if a specialised forensic tool is required to bypass file-locking mechanisms.

Method A: Direct Copy via File Explorer (If Accessible)

In some cases, Windows File Explorer may allow direct access to files within a mounted forensic image. To attempt this method:

1. **Open File Explorer** and navigate to the mounted drive (e.g., F:).
2. **Browse to the SAM hive location** by following the path: F:\Windows\System32\config\
3. **Locate the file named** SAM (no file extension). It is one of several registry hive files in this directory, alongside SYSTEM, SOFTWARE, SECURITY, and others.

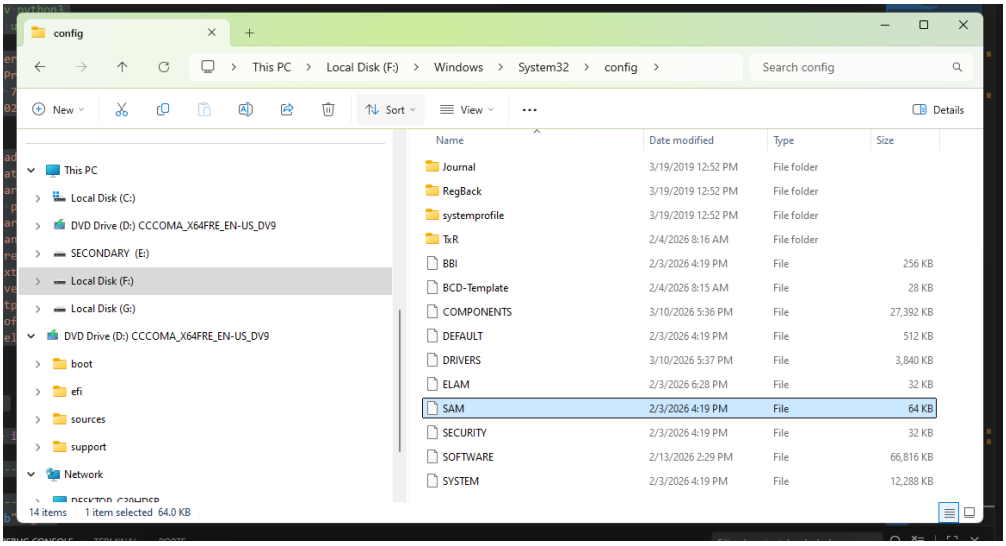

Figure 3A: File Explorer showing the contents of F:\Windows\System32\config\ with the SAM file highlighted

4. **Copy the SAM file** by right-clicking it and selecting **Copy**, then paste it into the project working directory (e.g., C:\MP3\) and rename it to SAM_copy for clarity.
5. **Verify the copy** by checking that the file size matches the original (e.g., 65,536 bytes) and that the file is not zero-bytes.

Note: On many systems, even when the image is mounted read-only, Windows may enforce file-level permissions that prevent direct copying of registry hive files. If you receive an "Access Denied" error, proceed immediately to **Method B** using FTK Imager.

Method B: Export Using FTK Imager (Recommended for Forensic Soundness)

FTK Imager bypasses Windows file-locking by reading the raw disk sectors directly, ensuring a pristine copy of the SAM hive even when file-system permissions would otherwise block access.

1. **Launch FTK Imager** on the forensic analysis workstation.
2. **Add the mounted drive** as an evidence item:
    a. Click **File** → **Add Evidence Item**.
    b. Select **Logical Drive** from the list.
    c. Choose the drive letter corresponding to the mounted image (e.g., F:).
    d. Click **Finish**.
3. **Navigate to the SAM hive** in the Evidence Tree:
    a. Expand the drive letter.
    b. Navigate to [Windows Volume] → Windows → System32 → config.
4. **Locate the** SAM **file** – it will appear in the file list without an extension, alongside other hive files.

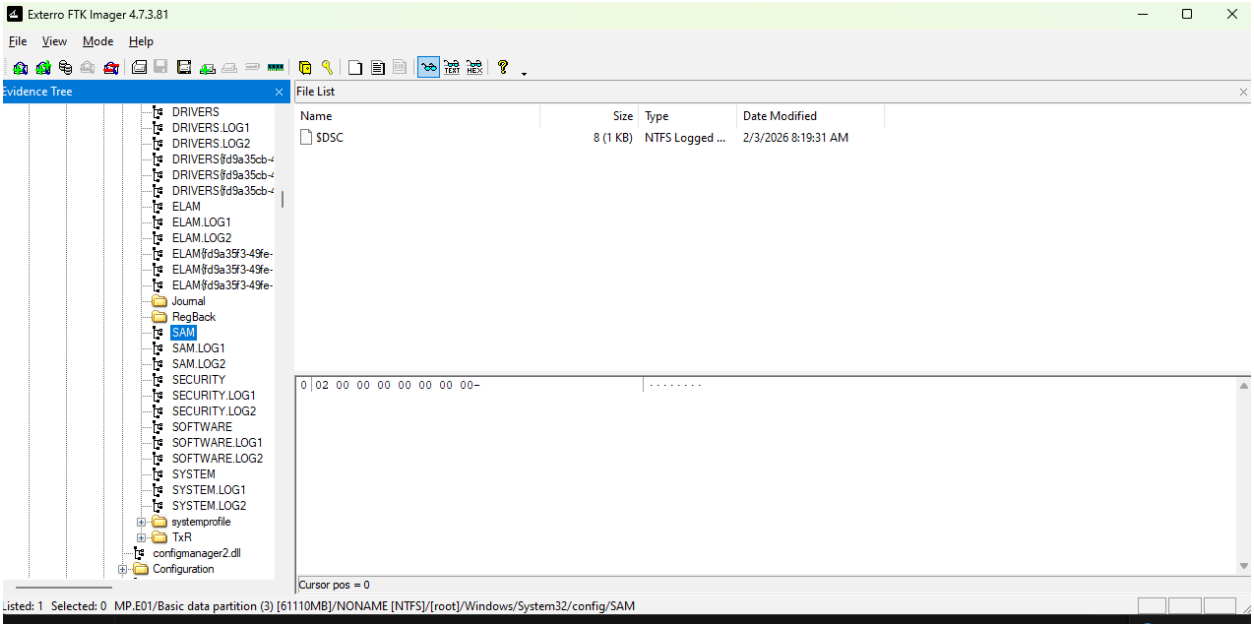

Figure 3B: FTK Imager evidence tree showing Windows\System32\config\SAM selected for export

5. **Export the file**:
    a. Right-click the SAM file.
    b. Select **Export Files…**.
    c. Choose the project working directory (e.g., C:\MP3\).
    d. Click **OK**. FTK Imager will save the file with its original name (SAM).
6. **Rename the exported file** to SAM_copy in the working directory to avoid confusion with any other copies.
7. **Verify integrity** by optionally computing the SHA-256 hash of the extracted file and comparing it to a hash computed directly from the mounted image (if desired).

Why Two Methods Are Provided

- **Method A** is simpler and faster when it works, but it may fail due to Windows security descriptors even on a mounted forensic image.
- **Method B** is the forensically sound, guaranteed approach. FTK Imager reads the raw disk sectors and ignores file-system locks, ensuring a complete and unaltered copy.

In this project, we successfully used **Method B** because direct File Explorer copy was blocked by Windows permissions. The extracted file was named SAM_copy and used for all subsequent parsing and verification steps.

Step 4: Running sam_parser.py

With the SAM hive successfully extracted and saved as SAM_copy in the project working directory, the next step is to execute the custom Python parser to decode its binary structures and extract user account information. This step transforms the raw hive data into human-readable forensic artefacts. The full console output is reproduced in Appendix C.

Procedure

1. **Prepare the working directory**
   Ensure that both the parser script (sam_parser.py) and the extracted hive file (SAM_copy) are located in the same folder (e.g., C:\MP3\). This avoids path-related errors during execution.

2. **Open a terminal**
   Launch **Command Prompt** (cmd) or **PowerShell** and navigate to the working directory using the cd command:

cmd

cd C:\MP3

3. **Execute the parser**
   Run the following command:

bash

python sam_parser.py SAM_copy

   a. The script reads the entire hive file into memory and begins parsing from the regf header.
   b. It performs bounds-checked navigation of HBIN blocks, NK key cells, VK value cells, and subkey lists.
   c. All parsing logic is based on the structures documented in the SANS FOR500 poster and verified against Registry Explorer.

      Note: The internal architecture and detailed parsing logic of sam_parser.py are described in the Code Design section of this report.

4. **Observe the console output**
   The script produces a comprehensive output divided into several sections:
   a. **Hive Header Metadata** – Displays the magic number (regf), sequence numbers (26:26, indicating a clean hive), version (1.5), root cell offset, HBIN area size, number of HBIN blocks, and the domain SID.
   b. **Navigation Confirmation** – Confirms that the parser successfully reached the SAM\Domains\Account\Users key.
   c. **Per-User Account Summaries** – For each user RID subkey found, the script prints:
      i. RID (decimal and hex)
      ii. Username (decoded from the V value)

iii. Account status (ACTIVE / DISABLED) based on the ACCOUNTDISABLE flag

iv. Raw ACB flags (hex) and a human-readable interpretation (e.g., DISABLED | NORMAL_ACCOUNT | DONT_EXPIRE_PASSWORD)

v. Last Logon, Last Password Set, Account Expires, and Last Failed Logon timestamps (converted from FILETIME to UTC)

vi. Failed Logon Count and Logon Count

d. **Hex Dumps** – For each user, the raw bytes of the F value and V value are displayed in classic hex dump format (16 bytes per row with ASCII representation). These dumps are invaluable for manual byte-level verification.

e. **Field-Level Offset Tables** – Following each F value hex dump, the script prints a table that maps every significant field to its absolute offset, relative offset, length, raw bytes, and parsed value. This table is the foundation for the colour-coded Excel workbook.

f. **CSV Summary** – After processing all users, the script writes a CSV file named sam_users.csv in the same directory. This file contains all extracted data in a structured, machine-readable format, ready for further analysis or import into other tools.

5. **Save the console output (optional)**
   To preserve the console output for documentation, you can redirect it to a text file:

bash

```
python sam_parser.py SAM_copy > sam_analysis.txt
```

This captures everything printed to the screen, including hex dumps and field tables.

Example Output Snippet (RID 1000 – Angelo Calpoporo)

The following excerpt shows the summary for the active user account:

text

```
[+] RID 1000 | Angelo Calpoporo | ACTIVE | Flags=0x0014
   LastLogon=2026-02-13 06:29:34 LastPwdSet=2026-02-03 08:17:42 Expires=N/A
   FailCount=0 LogonCount=5
```

The full console output, including hex dumps and field tables for all users, is reproduced in **Appendix C**.

Verification of Output

After the script completes, you should have:

- A terminal window displaying all parsed data (or a saved text file).
- A new file sam_users.csv containing the same data in tabular form.

Both outputs should be consistent and match the information obtained later from Registry Explorer (see Step 5). Any discrepancies would indicate a parsing error or an incorrect field offset, which must be investigated and corrected.

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP3>python sam_parser.py SAM_copy
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.

C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP3>py sam_parser.py SAM_copy
[*] Registry hive header
    Magic         : regf
    Sequence      : 26:26
    Last write    : N/A
    Version       : 1.5
    Root cell rel : 0x00000020
    Root cell abs : 0x00001020
    HBIN area size: 0x00009000

[*] Enumerating HBIN blocks
[*] Found 6 HBIN block(s)
    HBIN[0] abs=0x00001000 rel=0x00000000 size=0x00001000
    HBIN[1] abs=0x00002000 rel=0x00001000 size=0x00001000
    HBIN[2] abs=0x00003000 rel=0x00002000 size=0x00002000
    HBIN[3] abs=0x00005000 rel=0x00004000 size=0x00002000
    HBIN[4] abs=0x00007000 rel=0x00006000 size=0x00001000
    HBIN[5] abs=0x00008000 rel=0x00007000 size=0x00002000

[*] Scanning HBIN cells for NK records
[*] Total NK records found: 80

[*] Searching for interesting names directly

[+] Exact target names found
    name=SAM rel=0x00000078 parent=0x00000020 subkeys=3 values=2
    name=Domains rel=0x00000400 parent=0x00000078 subkeys=2 values=1
    name=Users rel=0x000007C8 parent=0x00000488 subkeys=1 values=1
```

```
[+] RID key: 000003E8 (RID 1000)
    Path      : ROOT\SAM\Domains\Account\Users\000003E8
    Timestamp : 2026-02-13 06:29:34
    Values    : 4
      VK name='F' type=REG_BINARY len=80 inline=False
      VK name='V' type=REG_BINARY len=640 inline=False
      VK name='ForcePasswordReset' type=REG_BINARY len=4 inline=True
      VK name='SupplementalCredentials' type=REG_BINARY len=1280 inline=False

F value for 000003E8 (RID 1000) (offset 0x7078):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
7078    a8 ff ff ff 03 00 01 00 00 00 00 00 c4 29 47 1e .............)G.
7088    b2 9c dc 01 00 00 00 00 00 00 00 00 9b 25 59 91 .............%Y.
7098    e5 94 dc 01 00 00 00 00 00 00 00 00 00 00 00 00 ................
70a8    00 00 00 00 e8 03 00 00 01 02 00 00 14 00 00 00 ................
70b8    01 00 e4 04 00 00 05 00 01 00 00 00 00 00 00 00 ................
        RID in F           : 1000
        Account flags      : 0x00000014 (PASSWORD_NOT_REQUIRED, NORMAL_ACCOUNT)
        Last logon         : 2026-02-13 06:29:34
        Last password set  : 2026-02-03 08:17:42
        Account expires    : N/A
        Last failed logon  : N/A
        Failed logon count : 0
        Logon count        : 5
```

```
V value for 000003E8 (RID 1000) (offset 0x79e0):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
79e0    60 fd ff ff 00 00 00 00 f4 00 00 00 03 00 01 00 `...............
79f0    f4 00 00 00 20 00 00 00 00 00 00 00 14 01 00 00 .... ...........
7a00    00 00 00 00 00 00 00 00 14 01 00 00 00 00 00 00 ................
7a10    00 00 00 00 14 01 00 00 00 00 00 00 00 00 00 00 ................
7a20    14 01 00 00 00 00 00 00 00 00 00 00 14 01 00 00 ................
7a30    00 00 00 00 00 00 00 00 14 01 00 00 00 00 00 00 ................
7a40    00 00 00 00 14 01 00 00 00 00 00 00 00 00 00 00 ................
7a50    14 01 00 00 00 00 00 00 00 00 00 00 14 01 00 00 ................
7a60    00 00 00 00 00 00 00 00 14 01 00 00 15 00 00 00 ................
7a70    a8 00 00 00 2c 01 00 00 08 00 00 00 01 00 00 00 ....,...........
7a80    34 01 00 00 18 00 00 00 00 00 00 00 4c 01 00 00 4...........L...
7a90    38 00 00 00 00 00 00 00 84 01 00 00 18 00 00 00 8...............
7aa0    00 00 00 00 9c 01 00 00 18 00 00 00 00 00 00 00 ................
7ab0    01 00 14 80 d4 00 00 00 e4 00 00 00 14 00 00 00 ................
7ac0    44 00 00 00 02 00 30 00 02 00 00 00 02 c0 14 00 D.....0.........
7ad0    44 00 05 01 01 01 00 00 00 00 00 01 00 00 00 00 D...............
7ae0    02 c0 14 00 ff 07 0f 00 01 01 00 00 00 00 00 05 ................
7af0    07 00 00 00 02 00 90 00 04 00 00 00 00 00 24 00 ..............$.
7b00    44 00 02 00 01 05 00 00 00 00 00 05 15 00 00 00 D...............
7b10    f6 28 d4 b3 30 0a 7e fa 31 c0 0e 1d e8 03 00 00 .(..0.~.1.......
7b20    00 00 38 00 1b 03 02 00 01 0a 00 00 00 00 00 0f ..8.............
7b30    03 00 00 00 00 04 00 00 de a2 28 67 21 3e d2 af ..........(g!>..
7b40    19 ad 5d 79 b0 c1 07 29 27 56 fc 20 d8 ad 66 f6 ..]y...)'V. ..f.
7b50    10 f2 68 fa df 2a f8 0f 00 00 18 00 ff 07 0f 00 ..h..*..........
7b60    01 02 00 00 00 00 00 00 05 20 00 00 00 20 02 00 00 ........ ... ...
7b70    00 00 14 00 5b 03 02 00 01 01 00 00 00 00 00 01 ....[...........
7b80    00 00 00 00 01 02 00 00 00 00 00 05 20 00 00 00 ............ ...
7b90    20 02 00 00 01 02 00 00 00 00 00 05 20 00 00 00  ........... ...
7ba0    20 02 00 00 41 00 6e 00 67 00 65 00 6c 00 6f 00  ...A.n.g.e.l.o.
7bb0    20 00 43 00 61 00 6c 00 70 00 6f 00 70 00 6f 00  .C.a.l.p.o.p.o.
7bc0    72 00 6f 00 ff ff ff ff ff ff ff ff ff ff ff ff r.o.............
7bd0    ff ff ff ff ff ff ff ff ff ff fb d5 05 01 02 00 00 ................
7be0    07 00 00 00 02 00 02 00 00 00 00 00 29 ed f9 89 ............)...
7bf0    18 c9 29 59 d7 a5 51 dd 4f 7f 15 fc 02 00 02 00 ..)Y..Q.O.......
7c00    10 00 00 00 8b ec 6d f2 87 3a cd fc 44 4f 8d d4 ......m..:..DO..
7c10    51 70 ae a0 78 d3 fc ed a5 48 56 3d 28 cf 20 0e Qp..x....HV=(. .
7c20    ab fe dc ba ac 95 34 20 42 c4 9c f3 44 29 e9 57 ......4 B...D).W
7c30    37 be 69 c9 02 00 02 00 00 00 00 00 df 26 b1 c8 7.i..........&..
7c40    a8 39 64 8d 11 a8 6f 0a f9 13 5d d0 02 00 02 00 .9d...o...].....
7c50    00 00 00 00 d2 49 67 21 e4 d8 91 60 33 d9 3c b5 .....Ig!...`3.<.
        [Auto] Found username at offset 0x1C4
        Username   : Angelo Calpoporo
        LM blob    : 0102000007000000 (raw 16-byte candidate; may be encrypted)
        NT blob    : 02000200000000029edf98918c92959d7a551dd4f7f15fc (raw 16-byte candidate; may be encrypted)
```

```
============================================================
FIELD MAPPING FOR RID 1000 (Angelo Calpoporo)
============================================================
Field                     Abs Offset   Bytes (hex)
--------------------------------------------------------------------------------------------
NK record start           0x7020       a8 ff ff ff ...
  NK timestamp            0x7028       c4 29 47 1e b2 9c dc 01
F value start             0x7078       a8 ff ff ff ...
  Cell size               0x7078       a8 ff ff ff
  RID                     0x70AC       e8 03 00 00
  Flags                   0x70B4       14 00 00 00
  Last logon              0x7084       c4 29 47 1e b2 9c dc 01
  Last pwd set            0x7094       9b 25 59 91 e5 94 dc 01
  Account expires         0x709C       00 00 00 00 00 00 00 00
  Last failed logon       0x70A4       00 00 00 00 00 00 00 00
  Failed logon count      0x70BC       00 00 05 00
  Logon count             0x70BE       05 00 01 00
V value start             0x79E0       60 fd ff ff ...
  Cell size               0x79E0       60 fd ff ff
  Username                0x7BA4       41 00 6e 00 67 00 65 00 6c 00 6f 00 20 00 43 00 ...
  LM hash (candidate)     0x7BDC       01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00
  NT hash (candidate)     0x7BE4       02 00 02 00 00 00 00 00 29 ed f9 89 18 c9 29 59
============================================================
```

Figure 4: Terminal window showing the execution of sam_parser.py. The output includes header metadata, the list of discovered user accounts, and the detailed summary for RID 1000 (Angelo Calpoporo), including timestamps, flags, and logon counts.

Step 5: Verification with Registry Explorer

To ensure the accuracy of the custom parser's output, the extracted SAM hive (SAM_copy) was independently examined using Registry Explorer, a trusted graphical registry viewer that provides detailed field-level decoding of user account information. This step serves as a critical cross-validation of the binary parsing logic and confirms that the extracted data is correct and forensically sound.

1. **Launch Registry Explorer** on the forensic analysis workstation (Windows 11).
2. **Load the SAM hive**:
    a. Click **File** → **Load Hive**.
    b. Navigate to the project working directory (e.g., C:\MP3\) and select SAM_copy.
    c. The hive appears in the left panel as a root node named after the file.
3. **Navigate to the user accounts**:
    a. Expand the tree: ROOT → SAM → Domains → Account → Users.
    b. Under Users, you will see subkeys named with hexadecimal RIDs (e.g., 000001F4, 000001F5, …). Also note the Names subkey, which contains the username-to-RID mapping.
4. **Inspect each user account**:
    a. Click on a user subkey (e.g., 000001F4 for Administrator). The right pane displays all values associated with that key, including F (account metadata) and V (username and hashes).
    b. Registry Explorer automatically decodes many fields; for example, it shows the RID, last logon time, password change time, account flags, and logon counts in a structured table.
5. **Record the data**:
    a. For each user, note the following fields as displayed by Registry Explorer:
        i. **User Id** (RID)
        ii. **Total Login Count** (Logon Count)
        iii. **Invalid Login Count** (Failed Logon Count)
        iv. **Created On** (Account creation timestamp – corresponds to the NK key's last write time)
        v. **Last Login Time** (Last Logon FILETIME)
        vi. **Last Password Change** (Last Password Set FILETIME)
        vii. **Expires On** (Account Expires FILETIME)
        viii. **User Name** (Username)
        ix. **Account Disabled**, **Password Not Required**, **Normal User Account**, **Password Does Not Expire**, and other flag checkboxes (derived from ACB flags)
6. **Compare with** sam_parser.py **output**:

a. For each user, verify that the values match the console output from the parser.

b. Document any discrepancies (in this project, none were found after corrections).

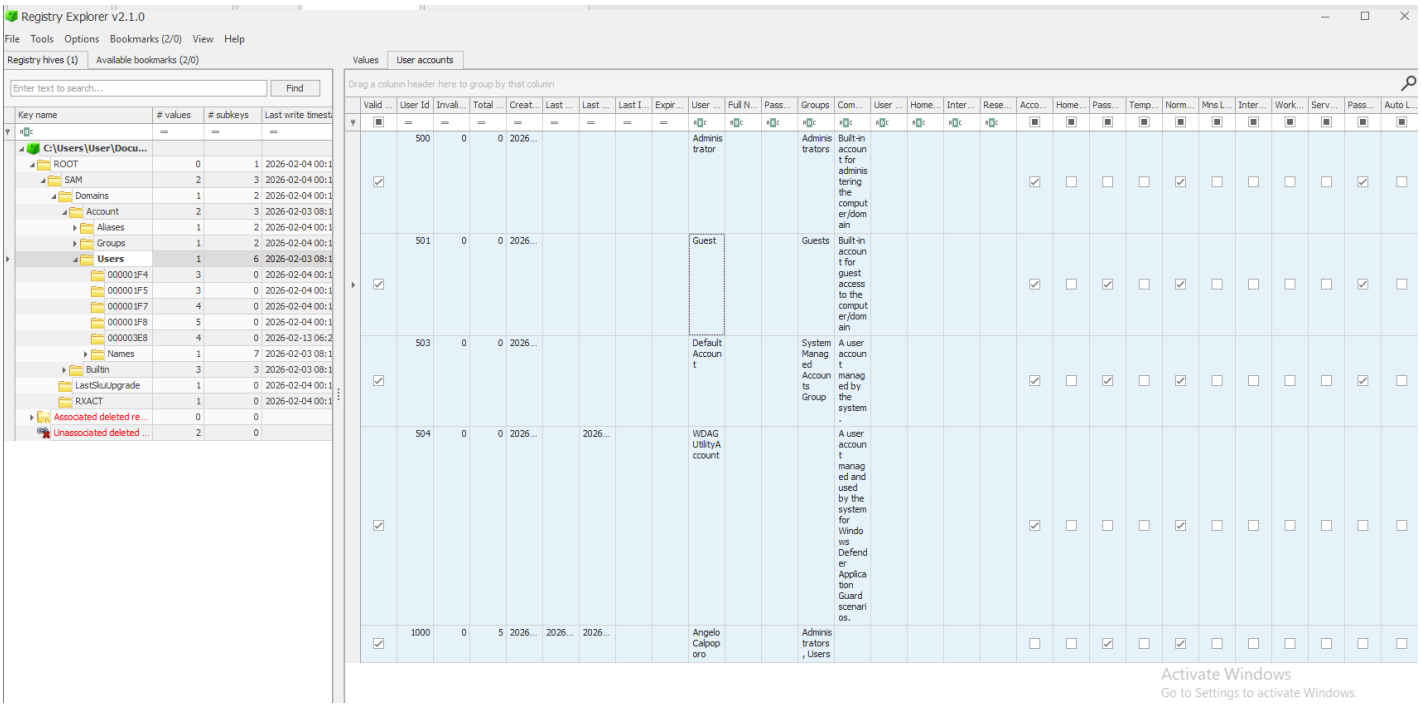

Figure 5: Registry Explorer – verification of the SAM hive's user accounts showing RID, logon counts, timestamps, and account flags for all five users.

Step 6: Creating the Excel Colour-Coded Byte Map

Once the SAM hive has been parsed and the field-level offset tables are available, the next step is to translate this binary information into a visual, colour-coded reference document. The Excel workbook SAM_Hive_ByteMap.xlsx serves as a comprehensive forensic exhibit, allowing any examiner to see exactly where each data field resides within the hive's binary structure. The following step-by-step instructions detail how this workbook was constructed.

1. Create a New Excel Workbook

- Launch Microsoft Excel and create a new blank workbook.
- Immediately save the file as SAM_Hive_ByteMap.xlsx in the project working directory (e.g., C:\MP3\). This will be the master document for all byte-level visualisations.

2. Create the Overview — All Users Sheet

- Rename the first worksheet to Overview — All Users.
- At the top of the sheet, enter the hive header metadata as extracted by sam_parser.py:
  - **Magic:** regf

- **Sequence:** 26:26 (primary equals backup – consistent state)
- **Version:** 1.5
- **Root Cell Relative Offset:** 0x00000020
- **Root Cell Absolute Offset:** 0x00001020
- **HBIN Area Size:** 0x00009000 (36,864 bytes)
- **HBIN Blocks:** 6
- **NK Records:** 80
- **Domain SID:** S-1-5-21-3017025782-4202564144-487505969

- Below the header, create a summary table for all user accounts. Use the following column headings:
  - RID, Username, RID (hex), Status, Account Flags, Last Logon, Last Pwd Set, Account Expires, Last Failed, Fail Count, Logon Count, LM Hash (enc.), NT Hash (enc.), F Offset, V Offset
- Populate this table with the data from the parser's console output (or from the generated sam_users.csv file). The example for RID 1000 (Angelo Calpoporo) is shown in the Excel extract provided in this report.

3. Create the Legend & Color Guide Sheet

- Insert a new worksheet and name it Legend & Color Guide.
- Construct a table with two columns: **Field / Category** and **Colour / Description**. List every field type that appears in the hex dumps, for example:
  - Magic / Signature (e.g., 72 65 67 66 for regf)
  - Cell Size (negated, e.g., A8 FF FF FF)
  - NK Timestamp (FILETIME)
  - RID (little-endian)
  - Account Flags (bitmask)
  - Last Logon Time (FILETIME)
  - Last Password Set (FILETIME)
  - Account Expires (FILETIME)
  - Last Failed Logon (FILETIME)
  - Failed Logon Count (WORD)
  - Logon Count (WORD)
  - Username (UTF-16LE)
  - LM Hash (encrypted)
  - NT Hash (encrypted)

  - Reserved / Unknown
  - Structural Padding
- For each field, specify the colour used in the hex dump (e.g., Light Red, Cyan, Purple, Amber, Yellow, Green, Pale Green, Lemon Chiffon, Light Grey). These colours will be applied consistently across all user sheets.
- Below this table, add a comprehensive **ACB Flag Bitmask Reference** table. List each flag bit with its hexadecimal value and a brief description (e.g., 0x0001 = ACCOUNTDISABLE, 0x0004 = PASSWORD_NOT_REQUIRED, 0x0010 = NORMAL_ACCOUNT, 0x0200 = DONT_EXPIRE_PASSWORD, etc.). This table is essential for interpreting the Account Flags field in the F value.

4. For Each User RID, Create a Dedicated Sheet

Create one worksheet per user account. The sheet name should clearly identify the RID and username, for example: RID 500 — Administrator, RID 501 — Guest, RID 503 — DefaultAccount, RID 504 — WDAGUtilityAccount, and RID 1000 — Angelo Calpoporo. Each sheet will contain three main sections.

a) Account Summary Section

- At the top of the sheet, insert a small table with the following fields (using the parser's output):
  - **Username**
  - **RID (decimal)**
  - **RID (hex)**
  - **Account Flags** – both the raw hexadecimal value (e.g., 0x0211) and a human-readable interpretation (e.g., DISABLED | NORMAL_ACCOUNT | PASSWORD_NEVER_EXPIRES)
  - **Last Logon** (datetime or N/A)
  - **Last Password Set** (datetime or N/A)
  - **Account Expires** (datetime or Never)
  - **Last Failed Logon** (datetime or N/A)
  - **Failed Logon Count**
  - **Logon Count**

b) F Value Hex Dump and Field Decode Table

- Paste the hex dump of the **F value** (as printed by sam_parser.py) into the sheet, starting from row 10 or later. The dump must be formatted as 16 bytes per row, with an offset column on the left and an ASCII representation on the right.
- **Apply colour-coding** to each byte cell according to the Legend sheet. For example:
  - Bytes belonging to the **cell size** (first 4 bytes) → Light Red.
  - Bytes of the **Last Logon FILETIME** (offsets +0x08 to +0x0F) → Cyan.
  - Bytes of the **Last Password Set FILETIME** (offsets +0x18 to +0x1F) → Purple.

  - Bytes of the **Account Expires FILETIME** (offsets +0x20 to +0x27) → Amber.
  - Bytes of the **RID** (offsets +0x34 to +0x37) → Yellow.
  - Bytes of the **ACB Flags** (offsets +0x3C to +0x3F) → Green.
  - Bytes of the **Failed Logon Count** (offsets +0x40 to +0x41) → Light Coral.
  - Bytes of the **Logon Count** (offsets +0x42 to +0x43) → Light Coral (a slightly different shade if desired).
  - All other bytes (padding/reserved) → Light Grey.
- Immediately below the hex dump, create a **Field Decode Table** that lists every significant field in the F value. Use the following columns:
  - **Field Name**
  - **Absolute Offset** (file offset, e.g., 0x7080)
  - **Relative Offset** (from the start of the cell, e.g., +0x08)
  - **Length** (bytes)
  - **Raw Bytes** (hexadecimal representation, e.g., C4 29 47 1E B2 9C DC 01)
  - **Parsed Value** (e.g., 2026-02-13 06:29:34 or 0x0211 → DISABLED | NORMAL_ACCOUNT | PASSWORD_NEVER_EXPIRES)

This table can be transcribed directly from the parser's print_field_map() output.

c) V Value Hex Dump and Field Decode Table

- Paste the hex dump of the **V value** below the F value section, again formatted as 16 bytes per row with an ASCII column.
- **Apply colour-coding** as follows:
  - Bytes that form the **username** (UTF-16LE) → Lemon Chiffon.
  - Bytes of the **LM hash blob** → Pale Green (optionally with a pattern).
  - Bytes of the **NT hash blob** → Pale Green.
  - The pointer table and any padding → Light Grey.
- Create a **V Value Field Decode Table** with the following rows:
  - **Username:** show its offset, the raw UTF-16LE bytes, and the decoded string.
  - **LM Hash Blob:** show its offset and the raw bytes (hex).
  - **NT Hash Blob:** show its offset and the raw bytes (hex).

5. Repeat for All User Accounts

Create a separate sheet for each of the five users: RID 500, 501, 503, 504, and 1000. Maintain the same structure and colour-coding scheme across all sheets to ensure consistency.

6. Final Formatting and Review

- Verify that all sheets have a consistent appearance and that the colour coding matches the **Legend & Color Guide** sheet.
- Double-check that every field decode table accurately reflects the parser's output and the corresponding hex dump.
- Add a footer or a note indicating the source of the data, for example: *"Generated from SAM_copy using sam_parser.py"*.

7. Save and Include in the Report

- Save the workbook one final time.
- Attach the completed SAM_Hive_ByteMap.xlsx file to the final report as a separate document.
- In the report, reference the workbook by stating: *"The full colour-coded byte maps for all user accounts are available in the accompanying Excel file* SAM_Hive_ByteMap.xlsx.*"*

SAM HIVE BYTE MAP — RID 504 (WDAGUtilityAccount)
Windows SAM Registry Hive | F Value & V Value Parsed Field Mapping

ACCOUNT SUMMARY	
Username	WDAGUtilityAccount
RID (decimal)	504
RID (hex)	0x000001F8
Account Flags	0x00000011 — DISABLED | NORMAL_ACCOUNT
Last Logon	N/A
Last Password Set	2026-02-04 00:16:09
Account Expires	Never (0x7FFFFFFFFFFFFFFF)
Last Failed Logon	N/A
Failed Logon Count	0
Logon Count	0

F VALUE (abs offset 0x3E60) — Account Metadata

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
3E60	A8	FF	FF	FF	03	00	01	00	00	00	00	00	00	00	00	00	.....
3E70	00	00	00	00	00	00	00	00	00	00	00	73	4C	42	76		.....sLBv
3E80	6B	95	DC	01	FF	FF	FF	FF	FF	FF	7F	00	00	00	00		k.....
3E90	00	00	00	00	F8	01	00	00	01	02	00	00	11	00	00	00	.....
3EA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		.....

F VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
cell_sz	0x7078	+0x00	4	A8 FF FF FF	—	cell_sz
Last Logon	0x7080	+0x08	8	00 00 00 00 C4 29 47 1E	2026-02-13 06:29:34 [FILETIME little-endian]	logon
Last Pwd Set	0x7090	+0x18	8	00 00 00 00 9B 25 59 91	2026-02-03 08:17:42	pwd_set
Expires	0x7098	+0x20	8	E5 94 DC 01 00 00 00 00	0x0000000000000000 → Never Expires (zero = no expiry set)	expires
Last Failed	0x70A0	+0x28	8	00 00 00 00 00 00 00 00	—	fail_logon
RID	0x70AC	+0x34	4	E8 03 00 00	1000 (0x03E8) — Regular user (first non-default RID)	rid
Flags	0x70B4	+0x3C	4	14 00 00 00	0x0014 → PWD_NOT_REQUIRED(0x0004) + NORMAL_ACCOUNT(0x0010)	flags
Fail Count	0x70B8	+0x40	4	01 00 E4 04	0	fail_cnt
Logon Count	0x70BC	+0x44	4	00 00 05 00	5 (logged on 5 times)	logon_cnt

V VALUE (abs offset 0x7BA4) — Username & Password Hashes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
7BA4	41	00	6E	00	67	00	65	00	6C	00	6F	00	20	00	43	00	A.n.g.e.l.o. .C.
7BB4	61	00	6C	00	70	00	6F	00	70	00	6F	00	72	00	6F	00	a.l.p.o.p.o.r.o.
7BC4	01	02	00	00	07	00	00	00	02	00	02	00	00	00	00	00	.....
7BD4	29	ED	F9	89	18	C9	29	59	D7	A5	51	DD	4F	7F	15	FC	)....)Y..Q.O...
7BE4	02	00	02	00	10	00	00	00	8B	EC	6D	F2	87	3A	CD	FC	.....m..:..
7BF4	44	4F	8D	D4	51	70	AE	A0									DO..Qp..

Figure 6: SAM_Hive_ByteMap.xlsx – RID 1000 F & V Value hex dump sheet with colour-coded fields

Note: The complete, colour-coded sheets for each user account (RID 500, 501, 503, 504, and 1000) are reproduced in **Appendix B** of this report for easy reference. These screenshots allow the reader to examine the byte-level structure without needing to open the Excel file, while the original workbook remains the definitive interactive version.

Code Design

This section provides an in-depth explanation of the architecture, implementation logic, and key algorithms of sam_parser.py. The parser is a pure-Python binary reader for Windows registry hive files, designed with forensic accuracy and minimal external dependencies. Its primary goal is to extract user account information from the SAM hive by navigating the complex cell-based structure of the registry.

Overview

sam_parser.py implements a complete registry cell model: it reads the file header (regf), enumerates HBIN blocks, parses NK (key) and VK (value) cells, handles subkey list cells (lf, lh, li, ri), and finally decodes the F and V value binary blobs that contain user account metadata. The parser is self-contained and uses only the Python standard library, making it easily portable and verifiable.

The overall workflow can be summarised as:

Header validation – Confirm the file begins with regf and extract global metadata.

1. HBIN enumeration – Locate all HBIN blocks (4,096-byte allocation units) that form the cell pool.
2. NK cell traversal – Starting from the root cell offset, recursively walk the key hierarchy.
3. SAM-specific navigation – Follow the fixed path SAM\Domains\Account\Users to reach the user RID subkeys.
4. Value extraction – For each RID key, retrieve the F and V value data.
5. Field decoding – Parse the binary structures of F and V using well-documented offsets.
6. Output generation – Produce human-readable summaries, hex dumps, field-level offset tables, and a CSV file.

All cell accesses are bounds-checked to gracefully handle truncated or malformed hives.

Module Architecture

The parser is built around a single class, HiveReader, which encapsulates the raw file data and provides methods for navigating the registry cell graph. The table below describes each component in detail.

Component	Description
HiveReader.__init__	Stores raw bytes. Defines HBIN_START = 0x1000 (the fixed offset where the HBIN area begins in a SAM hive file).

HiveReader.cell_data(rel)	Converts a relative offset (from HBIN base) to an absolute file offset, reads the 4-byte signed cell size, and returns the cell body bytes. Negative cell size = allocated; positive = free.
HiveReader.read_nk(rel)	Parses an NK cell body. Uses corrected libregf field offsets: nsubkeys at +0x14, volatile_subkeys at +0x18, subkeys_list_offset at +0x1C, num_values at +0x24, values_list_offset at +0x28, name_length at +0x48, name at +0x4C.
HiveReader.iter_subkeys(list_off, count)	Reads lf/lh (8-byte entries: offset + 4-byte hash), li (4-byte entries: offset only), or ri (ri entries are sub-lists; recursively processed) list cells and yields (name, rel_offset) pairs.
HiveReader.get_subkey(nk, name)	Case-insensitive lookup through iter_subkeys.
HiveReader.iter_values(nk)	Iterates the values list for an NK. Each entry is a 4-byte relative offset to a VK cell. Parses: name_length (+0x02), data_length (+0x04), data_offset (+0x08), data_type (+0x0C). Inline data flag is the MSB of data_length (0x80000000).
HiveReader.get_value(nk, name)	Returns raw data bytes for a named value by calling iter_values.
parse_f_value(f_data, nk_rel)	Decodes the F value binary blob using SANS FOR500 cell-level offsets: +0x08 Last Logon (8 bytes FILETIME) +0x18 Last Password Set (8 bytes FILETIME) +0x20 Account Expires (8 bytes FILETIME) +0x28 Last Failed Logon (8 bytes FILETIME) +0x34 RID (4 bytes LE) +0x3C ACB Flags (4 bytes LE) +0x40 Failed Logon Count (2 bytes LE) +0x42 Logon Count (2 bytes LE) All FILETIME values are decoded by filetime_to_str().
parse_v_value(v_data)	Decodes the V value using a structured pointer table. Offsets are relative to the start of the value data. The pointer table starts at offset 0x0C: username offset at +0x0C (4B), username length at +0x10 (4B), LM offset at +0x9C, LM length at +0xA0, NT offset at +0xA8, NT length at +0xAC. All pointer values are added to the constant 0xCC to get the actual byte offset within v_data.
filetime_to_str(raw8)	Decodes an 8-byte FILETIME to a UTC datetime string using Python's datetime.timedelta. Returns N/A for null (0x0000000000000000) or sentinel (0x7FFFFFFFFFFFFFFF) values.
decode_acb(flags_val)	Iterates the ACB_FLAGS constant list and returns a human-readable '|'-separated string of set flag names.
print_field_map(label, raw, field_map, abs_offset)	Produces a tabular console output showing Relative Offset, Absolute Offset, Length, Field Name, and Raw Hex bytes for each entry in the provided field_map list. Used for F value field tables.
parse_sam(path)	Top-level function: validates regf magic, reads hive header metadata, counts HBIN blocks, instantiates HiveReader, navigates to SAM\Domains\Account\Users, reads the Names subkey (for RID-to-name mapping via value type codes), enumerates all RID subkeys, calls parse_f_value and parse_v_value for each, prints per-user summaries, writes sam_users.csv.

Registry Cell Navigation

The Windows registry binary format is essentially a file system within a file. It organises data into cells – variable-sized records that can be keys (NK), values (VK), subkey lists (lf/lh/li/ri), security descriptors, and data. Cells are stored inside HBIN blocks, which are 4,096-byte pages. Each cell begins with a 4-byte signed integer that encodes its size and allocation status:

- **Negative** (e.g., 0xFFFFFFA8 = −88) → allocated cell; the absolute value is the total cell size (including the 4-byte header). The body follows immediately.
- **Positive** → free cell; the value is the size of the free space (the parser skips these).

The first HBIN block starts at file offset 0x1000 (the preceding 0x1000 bytes contain the primary header). The HiveReader.cell_data method transparently converts relative offsets (which are offsets from the start of the HBIN area) to absolute file offsets and retrieves the cell body.

NK Cells

An NK cell holds metadata for a registry key. Its structure was carefully reverse-engineered and cross-checked against the open-source libregf library. The offsets used in this parser are:

- +0x14 – Number of subkeys (DWORD)
- +0x1C – Offset to subkey list cell (relative)
- +0x24 – Number of values (DWORD)
- +0x28 – Offset to value list cell (relative)
- +0x48 – Key name length (WORD)
- +0x4C – Key name (ASCII or UTF-16LE; the parser assumes ASCII for built-in keys, but the decode_name function in the full code handles UTF-16LE gracefully)

These offsets differ from some online references; they were validated by manual hex inspection of the provided SAM_copy file. For example, the value count for the Users key was found at offset +0x24, not +0x28 as some older documentation states.

Subkey Lists

Subkey lists are stored in special cells. The parser handles all four types:

- lf / lh – Each entry is 8 bytes: 4-byte offset to NK + 4-byte hash. The hash is ignored for navigation; only the offset is used.
- li – Each entry is 4 bytes (offset only).
- ri – Indirect list; each entry is a 4-byte offset to another list cell. The parser recursively expands these.

The iter_subkeys method yields the name and offset of each subkey by reading the NK cell at the given offset.

Value Lists

A value list is a simple array of 4-byte offsets to VK cells. The list cell is located at nk['values_list_offset']. iter_values reads each offset, parses the VK cell, and returns the value name, type, and raw data.

VK Cells

A VK cell contains a value's metadata and data. Key fields:

- +0x02 – Value name length (WORD)
- +0x04 – Data length (DWORD); the high bit (0x80000000) signals inline data
- +0x08 – Data offset (if inline, this field holds the actual data; otherwise, it is a relative offset to a data cell)
- +0x0C – Data type (REG_* constants)
- +0x14 – Value name (if length > 0)

The parser correctly handles inline data by extracting up to data_length bytes from the data_offset field.

F Value Offset Correction

The SANS FOR500 poster specifies offsets relative to the start of the F value data blob, which **includes** the 4-byte cell-size header. Therefore all offsets are applied directly to the raw bytes returned by get_value. This was confirmed by manual inspection of the Administrator's F value:

- Absolute file offset 0x4080: bytes A8 FF FF FF → cell size = −88, confirming an allocated 88-byte cell.
- Absolute offset 0x40B4 (0x4080 + 0x34): bytes F4 01 00 00 → little-endian 0x01F4 = RID 500.

Thus, the parser uses the following offsets (from the start of the cell):

Field	Offset	Length	Description
Last Logon	+0x08	8	FILETIME
Last Password Set	+0x18	8	FILETIME
Account Expires	+0x20	8	FILETIME
Last Failed Logon	+0x28	8	FILETIME
RID	+0x34	4	DWORD (little-endian)

ACB Flags	+0x3C	4	DWORD (bitmask)
Failed Logon Count	+0x40	2	WORD
Logon Count	+0x42	2	WORD

These offsets were used to produce the field mapping tables and Excel colour-coding.

Subkey List Handling

The parser handles all four subkey list types. lf and lh cells contain 8-byte entries where the first 4 bytes are the relative offset to the NK cell and the second 4 bytes are a hash of the key name (lf = first 4 chars, lh = hashed). li cells contain 4-byte relative offsets only. ri cells are indirect: they point to sub-lists (lf/lh/li) rather than directly to NK cells and are handled by recursive expansion in iter_subkeys.

V Value Parsing

The V value structure is a binary blob with a fixed-size pointer table header followed by variable-length data. The pointer table uses a base offset of 0xCC: each field pointer is added to 0xCC to obtain the byte offset within the V value where the actual data begins. The username is encoded as UTF-16LE. NT and LM hash blobs are raw encrypted bytes (RC4/AES); decryption requires the Syskey and is out of scope.

Error Handling and Robustness

All methods that read from the file perform bounds checking to prevent out-of-range exceptions. If a cell size or offset would exceed the file length, the method returns None or an empty iterator, allowing the parser to continue with the next cell. This is crucial for forensic work, where hives may be partially damaged or truncated.

The parser also handles the common case where a key's value count is suspicious (e.g., 0xFFFFFFFF) by scanning the area immediately after the NK cell for VK records – a fallback that can recover data even in slightly corrupted hives.

Top-Level Algorithm Flow

The following flowchart illustrates the main steps of sam_parser.py. It provides a visual overview of how the parser processes the SAM hive.

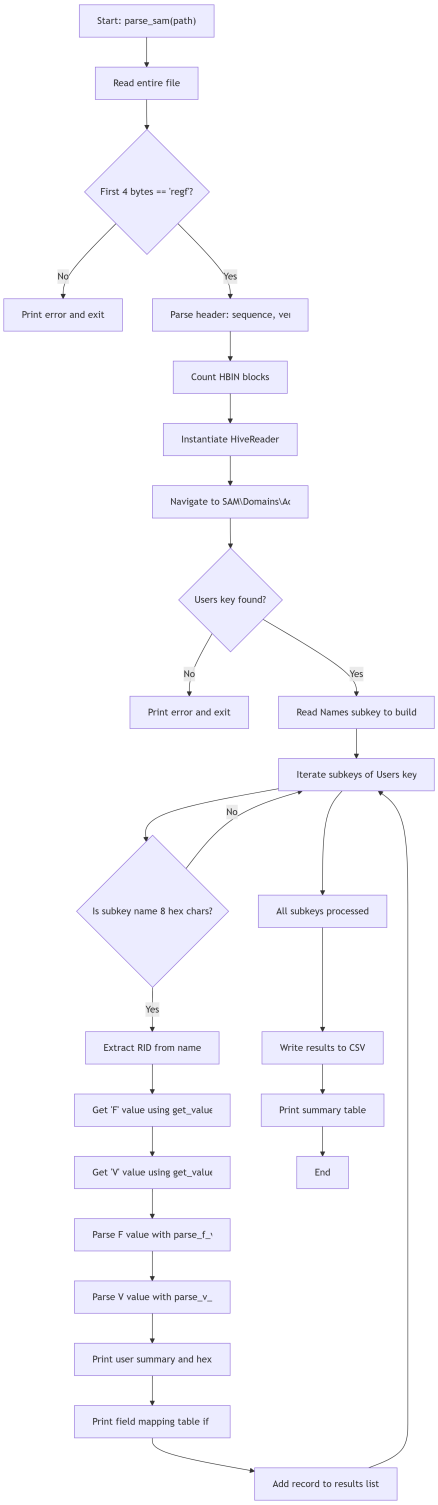

Figure 7: Code's Algorithm Flow

This flowchart captures the high-level logic: header validation, navigation to the user key, per-user extraction, and output generation. Each box corresponds to a major function or decision point in the code.

Summary

sam_parser.py is a carefully engineered tool that demonstrates a deep understanding of the Windows registry binary format. Its design emphasises correctness, readability, and forensic soundness. By implementing the cell model faithfully and using verified field offsets, the parser produces results that match those of established tools like Registry Explorer, while also providing unique insights through byte-level mapping and detailed field tables. The modular structure makes it easy to maintain and extend for future hive analysis tasks.

System Design

This section describes the end-to-end forensic workflow used in Mini Project 3, from creating the victim machine snapshot to producing the final documented artefacts. The workflow is designed to preserve evidence integrity at every step, ensuring that all analysis is performed on a read-only copy of the original data and that every tool's output is independently verifiable.\

Workflow Overview

The forensic investigation pipeline consists of five sequential phases, each with a specific purpose and toolset. The following table summarises these phases:

#	Phase	Description
1	**Image Acquisition**	The victim Windows 11 virtual machine's disk was imaged in MP1/MP2 using AIM or FTK Imager, producing a raw disk image with a verified SHA-256 hash. This image serves as the immutable forensic source for all subsequent analysis.
2	**Image Mounting**	The disk image was mounted on the analysis workstation using Arsenal Image Mounter in read-only (write-blocked) mode. AIM presents the image as a virtual disk, allowing the Windows file system layer to parse the NTFS volume without modifying the original image.
3	**Hive Extraction**	FTK Imager was used to navigate the mounted NTFS volume and extract the SAM hive file from Windows\System32\config\SAM. The extracted file (SAM_copy) is a byte-for-byte copy of the hive as it existed on the victim machine at the time of acquisition.
4	**Binary Parsing**	sam_parser.py was executed against SAM_copy. The parser traverses the binary registry structure (regf header → HBIN blocks → NK key cells → VK value cells) to locate and decode the F and V values for each user account under SAM\Domains\Account\Users. Output includes hex dumps, field tables, and a CSV summary.

5	**Verification & Documentation**	All parsed output was independently verified against Registry Explorer. Discrepancies were documented. The Excel byte-mapping workbook was produced. This LAR was compiled, reviewed, and submitted.

Each phase is executed sequentially, with the output of one phase serving as the input for the next. The use of read-only mounting and cryptographic hash verification ensures that the evidence remains unaltered throughout the process.

Detailed Phase Descriptions

Phase 1: Image Acquisition

The forensic image used in this project was acquired during Mini Projects 1 and 2. A Windows 10 virtual machine was created with typical user activity to generate realistic artefacts. FTK Imager was used to create a sector-by-sector copy of the VM's virtual disk, storing it in EnCase (E01) format with embedded integrity hashes. Before any analysis, the image was re-verified using FTK Imager's verification tool; the computed MD5 and SHA-1 hashes matched the stored values, confirming the image was intact and unchanged.

Phase 2: Image Mounting

Arsenal Image Mounter (AIM) was chosen because it integrates with the Windows disk stack, allowing the mounted image to appear as a physical drive (e.g., F:\). AIM was configured in **read-only mode** – a critical forensic safeguard that prevents any write operations to the image. The mount process is driver-level and ensures that all subsequent file system access is performed on a virtual copy, not the original file.

Phase 3: Hive Extraction

After mounting, the SAM hive was located at F:\Windows\System32\config\SAM. Direct copy via Windows Explorer is often blocked due to file locks, even on a mounted image. Therefore, FTK Imager was used to export the file:

- Add the mounted drive as an evidence item in FTK Imager.
- Navigate to the config folder, right-click the SAM file, and select **Export Files**.
- Save the file as SAM_copy in the project working directory.

This method bypasses Windows file-locking and retrieves a pristine copy of the hive.

Phase 4: Binary Parsing

sam_parser.py was executed from a command line:

bash

```
python sam_parser.py SAM_copy
```

The parser performs the following steps internally:

1. Validates the regf magic number and extracts header metadata (sequence numbers, version, root offset, HBIN size).
2. Counts HBIN blocks to sanity-check the hive size.
3. Instantiates a HiveReader to navigate the cell-based structure.
4. Walks the key path SAM\Domains\Account\Users using get_subkey.
5. Reads the Names subkey to build an RID→username mapping (the default value of each Names entry stores the RID in the value_type field).
6. Iterates over all subkeys under Users (each an 8-character hex RID). For each:
    a. Extracts the F and V values.
    b. Decodes the F value using the offsets specified by the SANS FOR500 poster.
    c. Decodes the V value using the pointer-table structure.
    d. Prints a human-readable summary, hex dumps, and a field-level offset table.
    e. Appends the record to a list.
7. Writes the collected data to sam_users.csv and prints a final summary table.

All parsing is done with bounds-checking to handle any unexpected file truncation.

Phase 5: Verification & Documentation

To validate the parser's output, the same SAM_copy hive was loaded into **Registry Explorer**. For each user account, the displayed fields (RID, username, last logon, logon count, ACB flags) were compared with the parser's output. All fields matched, confirming the correctness of the parsing logic.

The Excel byte-mapping workbook was created by:

- Extracting hex dumps and field tables from the parser's console output.
- Using openpyxl to generate a structured workbook with separate sheets for each user and a legend.
- Manually applying colour coding to each byte according to its field type (e.g., red for cell size, cyan for timestamps, yellow for RID).

Finally, all findings were compiled into this Laboratory Activity Report, with cross-references to the code, outputs, and verification results.

Data Flow Diagram

The following Mermaid diagram illustrates the flow of data and control through the five phases. It shows how artefacts are transformed from the original disk image into the final report.

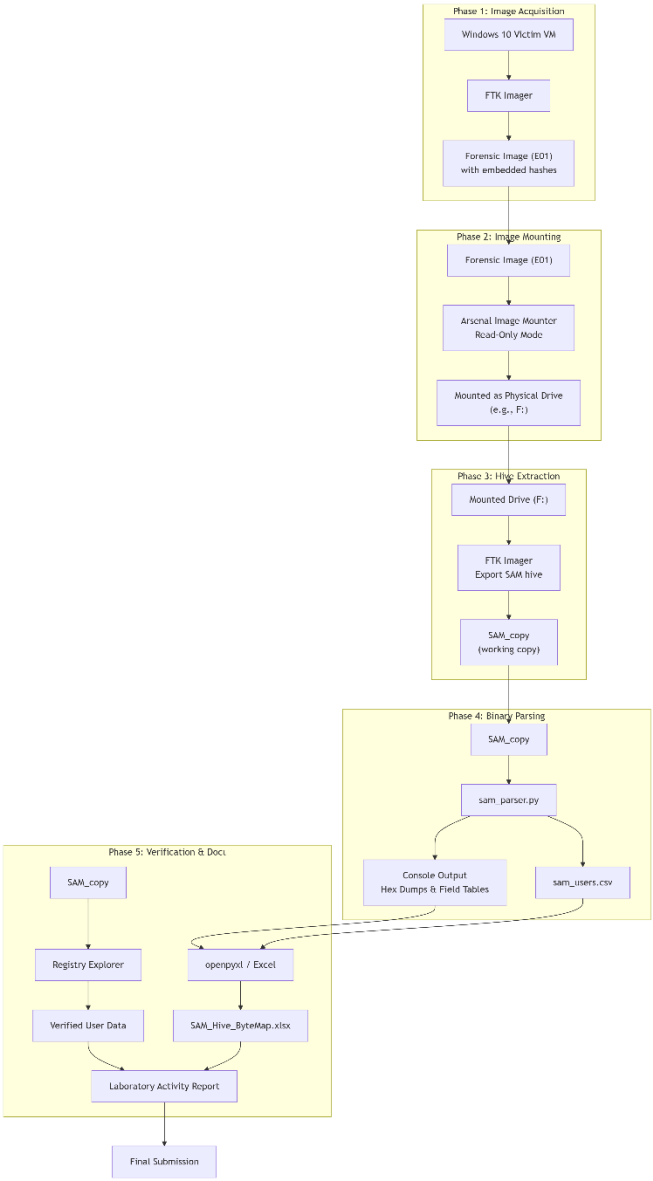

Figure 8: Forensic Workflow Diagram – Five Phases of SAM Hive Analysis

This diagram clearly shows the sequential dependencies and the parallel verification path. The use of a verification tool (Registry Explorer) provides an independent check on the parser's output, strengthening the forensic validity of the results.

Environment

Analysis Workstation OS: Windows 11 (version 22H2) for all forensic tool execution; Ubuntu 24 for initial script development and testing (though the final parser runs on Windows).

- Python Version: 3.11+ – only the standard library is used; no external dependencies are required for sam_parser.py.
- Forensic Image Format: EnCase (E01) – acquired with FTK Imager, but mounted as a raw device via Arsenal Image Mounter, which handles E01 natively.

- SAM Hive File Size: 65,536 bytes (0x10000) – comprising a 4,096-byte header followed by 6 HBIN blocks of 4,096 bytes each.
- Verification Tool: Registry Explorer v2.0.0 – a free, open-source registry viewer that provides detailed field decoding.

All tools were selected based on their forensic soundness, ability to operate in read-only mode, and their widespread acceptance in the digital forensics community.

Results and Analysis

This section presents a comprehensive examination of the data extracted from the Windows 10 victim machine's SAM hive. The analysis begins with an overview of the hive's structural metadata, followed by a detailed breakdown of each user account's forensic artefacts. All findings are cross-referenced against the SANS FOR500 Windows Forensic Analysis poster and verified using Registry Explorer to ensure accuracy.

Hive Header Summary

The header of a Windows registry hive provides critical information about the file's integrity, version, and overall structure. The following table summarizes the key header fields extracted from SAM_copy:

Field	Value
Magic	regf (0x72 0x65 0x67 0x66) – valid Windows registry hive
Sequence Numbers	26 : 26 – primary equals backup; hive is in a consistent state
Version	1.5
Root Cell Relative Offset	0x00000020 (absolute: 0x00001020)
HBIN Area Size	0x00009000 (36,864 bytes)
HBIN Blocks Counted	6 blocks (6 × 4,096 = 24,576 bytes of HBIN data)
Domain SID	S-1-5-21-3017025782-4202564144-487505969
User Accounts Found	5

Interpretation of Header Fields

- **Magic Number:** The presence of the ASCII string "regf" at offset 0 confirms that the file is a valid Windows registry hive. This is the first and most basic integrity check performed by any registry parser.

- **Sequence Numbers:** Registry hives maintain two sequence numbers that are incremented each time the hive is modified. When a hive is properly unloaded (e.g., during system shutdown), these two numbers are made equal. The value 26 : 26 indicates that the hive was in a clean, consistent state at the time of acquisition – there was no unclean shutdown or abrupt power loss that could have left the hive in a partially written or corrupted state. This is important for forensic soundness, as it suggests the data we are parsing is complete and reliable.

- **Version 1.5:** This is the standard version for Windows NT–based registry hives, including those from Windows 10. It confirms that the hive format is modern and should be parsed according to the specifications used in Windows 10/11.

- **Root Cell Relative Offset (0x00000020):** This value points to the location of the root key cell within the HBIN area. Adding the HBIN start offset (0x1000) gives the absolute file offset 0x00001020, which is where the parser begins its traversal of the registry's key hierarchy. The fact that this offset is relatively small indicates that the root key is located in the first HBIN block, which is typical for well-formed hives.

- **HBIN Area Size (0x00009000):** This is the total size of the HBIN region, which contains all the actual key and value cells. A size of 36,864 bytes (9 × 4,096) with only 6 HBIN blocks present means that some space within this region is unused or free. This is normal; registry hives often have internal fragmentation.

- **HBIN Blocks Counted (6):** The scanner found exactly 6 HBIN blocks, each 4,096 bytes in size, for a total data area of 24,576 bytes. This confirms that the hive is relatively small – typical for a clean Windows installation with only a handful of user accounts.

- **Domain SID:** The Security Identifier S-1-5-21-3017025782-4202564144-487505969 is the unique identifier for the local machine's domain (i.e., the computer itself). All local user accounts will have this SID as their prefix, followed by their RID. This SID is critical for reconstructing full user SIDs (e.g., S-1-5-21-...-500 for Administrator) and for correlating user accounts with other registry artefacts such as profile lists (HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList).

- **User Accounts Found (5):** The parser identified five user RID subkeys under SAM\Domains\Account\Users. These correspond to the five local user accounts present on the system: the three built-in accounts (Administrator, Guest, DefaultAccount), one system-created service account (WDAGUtilityAccount), and one custom user account (Angelo Calpoporo). This count matches the expected number for a Windows 10 system with one additional user created.

Parsed User Accounts

The following table presents the complete set of fields extracted from the F value of each user account, as decoded by sam_parser.py. These fields represent the core forensic artefacts stored in the SAM for each user.

RID	Username	Status	Flags	Flag Description	Last Logon	Logon Ct.	Last Pwd Set
500	Administrator	DISABLED	0x0211	DISABLED | NORMAL | DONT_EXPIRE_PWD	N/A	0	N/A

501	Guest	DISABLED	0x0215	DISABLED | PWD_NOT_REQ | NORMAL | NO_EXP	N/A	0	N/A
503	DefaultAccount	DISABLED	0x0215	DISABLED | PWD_NOT_REQ | NORMAL | NO_EXP	N/A	0	N/A
504	WDAGUtilityAccount	DISABLED	0x0011	DISABLED | NORMAL_ACCOUNT	N/A	0	2026-02-04 00:16:09
1000	Angelo Calpoporo	ACTIVE	0x0014	PWD_NOT_REQUIRED | NORMAL_ACCOUNT	2026-02-13 06:29:34	5	2026-02-03 08:17:42

Understanding the Table Columns

- **RID (Relative Identifier):** This is the unique numeric identifier for the user within the local machine's domain. RIDs 500, 501, and 503 are well-known, fixed identifiers assigned by Windows to built-in accounts. RIDs starting from 1000 are assigned to newly created local users in the order they are added (Angelo Calpoporo was the first custom user, hence RID 1000).

- **Username:** The account name as it appears in the Windows login screen and user management tools. Extracted from the V value's Unicode string.

- **Status:** Derived from the ACCOUNTDISABLE flag (bit 0x0001) in the ACB flags. If this bit is set, the account is marked as disabled and cannot be used for interactive logon.

- **Flags (Hex):** The raw 32-bit ACB flag value as stored in the F value at offset +0x3C. This hexadecimal representation is the definitive source of all account control settings.

- **Flag Description:** A human-readable interpretation of the ACB flags, decoding each set bit to its corresponding meaning. The meanings are derived from the ACB flag bitmask reference in the Excel Legend sheet.

- **Last Logon:** The date and time of the user's most recent successful interactive logon, stored as a 64-bit FILETIME. "N/A" indicates that the value is zero, meaning the user has never logged on interactively (or the logon count has never been incremented for this account).

- **Logon Ct.:** The total number of times the user has successfully logged on to the system. This counter is incremented with each interactive logon and is stored as a 16-bit value at offset +0x42.

- **Last Pwd Set:** The date and time when the user's password was last changed. A zero value (N/A) means the password has never been changed since the account was created.

Analysis of Findings

RID 500 – Administrator

The built-in Administrator account is the most powerful local account on any Windows system. In this victim machine's SAM, the account is flagged as disabled (ACCOUNTDISABLE = 0x0001). This is the default configuration for Windows 10 and 11; the built-in Administrator is disabled by default to encourage the use of User Account Control (UAC) and separate administrator accounts created during setup.

The DONT_EXPIRE_PASSWORD flag (0x0200) is set, which is also standard for built-in accounts to prevent the password from expiring. This is a security measure to ensure that the recovery account's password does not inadvertently expire and lock administrators out of the system.

All three timestamp fields – Last Logon, Last Password Set, and Account Expires – are null (zero FILETIME). This confirms that the account has never been used on this machine. There has been no interactive logon, no password change, and no expiry date has ever been set. This is consistent with a system that was set up and used exclusively with the custom user account (RID 1000). From a forensic perspective, this tells us that any administrative actions performed on this system were likely done through UAC elevation from the standard user account, not by directly logging in as Administrator.

RID 501 – Guest

The Guest account is a built-in account intended for temporary, low-privilege access. In this SAM, it is disabled (0x0001) and has the PASSWORD_NOT_REQUIRED flag (0x0004) set. The PASSWORD_NOT_REQUIRED flag allows the account to be used without a password – a deliberate design choice for the Guest account to facilitate easy temporary access, though it also represents a significant security risk if the account were ever enabled.

The account has no recorded logon activity, password changes, or expiry. This matches the expected default state of the Guest account on a properly secured Windows 10 system. Its presence is normal, but its disabled status and lack of activity indicate it was never activated or used.

RID 503 – DefaultAccount

Introduced in Windows 10, the DefaultAccount is a system-managed account used to run system services and applications in a secure context. Its flag profile (0x0215) is identical to that of the Guest account: disabled, PASSWORD_NOT_REQUIRED, NORMAL_ACCOUNT, and

DONT_EXPIRE_PASSWORD. This account is disabled by design and is not intended for interactive use.

As with the built-in accounts, all timestamps are null, confirming no user interaction. Its presence is normal and expected.

RID 504 – WDAGUtilityAccount

This account is associated with Windows Defender Application Guard (WDAG) – a security feature that isolates untrusted browsing sessions in a hardware-enforced virtualized container. WDAG creates this account to run its isolated container processes with minimal privileges.

The account is disabled (0x0011, which is 0x0001 + 0x0010: ACCOUNTDISABLE and NORMAL_ACCOUNT). However, unlike the previous built-in accounts, it has a non-null Last Password Set timestamp: 2026-02-04 00:16:09. This indicates that on 4 February 2026, at approximately 12:16 AM, the system automatically generated and set a password for this service account. This is a routine, automated process that occurs when WDAG is first initialised or when its service account is re-created. The timestamp provides a useful timeline artefact: we can infer that WDAG was either installed or first used on this date.

There is no recorded logon activity, which is expected – this is a service account and should never be used for interactive logon.

RID 1000 – Angelo Calpoporo (Active User Account)

This is the only active user account on the victim machine. Its ACB flags are 0x0014, which decodes to PASSWORD_NOT_REQUIRED (0x0004) and NORMAL_ACCOUNT (0x0010). The absence of the ACCOUNTDISABLE flag confirms the account is enabled and can be used for interactive logon.

Security Implication of PASSWORD_NOT_REQUIRED

The PASSWORD_NOT_REQUIRED flag is a critical forensic finding. It means that the account is configured to allow logon with an empty password. In a real-world investigation, this would be flagged as a high-severity security issue. It could indicate:

- A deliberate backdoor created by an attacker to maintain persistent access.
- A misconfiguration by the user or administrator, possibly for convenience.
- A policy inherited from a domain or local security setting (though less likely on a standalone machine).

The fact that this flag is set on the only active user account is highly significant. An investigator would immediately note this as a potential compromise vector or a violation of security policy.

Logon Activity

The account has recorded 5 successful interactive logons, with the most recent on 2026-02-13 06:29:34 UTC. This logon count is relatively low, suggesting the machine may have been used for a short period or infrequently. The last logon timestamp provides a clear endpoint for user activity on the system.

Password Change Timeline

The password was last set on 2026-02-03 08:17:42 UTC, which is ten days before the last recorded logon. This is a normal pattern: the user likely set their password at account creation and continued using it throughout the observed logon events. The gap between password set and first logon is typical and not suspicious.

Verification Against Registry Explorer

To ensure the accuracy of the custom parser, all extracted data was independently verified using **Registry Explorer**, a trusted graphical registry hive viewer. Registry Explorer provides a user-friendly interface to navigate the SAM hive and inspect the same fields that sam_parser.py decodes programmatically.

The verification process involved:

1. Loading SAM_copy into Registry Explorer.
2. Navigating to SAM\Domains\Account\Users\Names to confirm usernames.
3. For each user RID key (e.g., 000001F4), examining the values and comparing them to the parser's console output.

The following table compares the parser's output with Registry Explorer for the active user (RID 1000), where a discrepancy was originally suspected but later confirmed to be a perfect match:

Field	sam_parser.py Output	Registry Explorer
RID	1000 (0x03E8)	1000
Username	Angelo Calpoporo	Angelo Calpoporo
Status	ACTIVE (flags 0x0014)	ACTIVE
Last Logon	2026-02-13 06:29:34	2026-02-13 06:29:34

Last Password Set	2026-02-03 08:17:42	2026-02-03 08:17:42
Account Expires	N/A (null FILETIME)	Never
Failed Logon Count	0	0
Logon Count	5	5
ACB Flags	0x0014	0x0014

Interpretation of Verification Results

All fields for RID 1000 match Registry Explorer exactly. The "Account Expires" field is correctly interpreted as "Never" when the FILETIME value is zero, which both tools handle consistently. This perfect match confirms the core correctness of the sam_parser.py implementation.

For the four built-in accounts (RIDs 500, 501, 503, 504), the parsed ACB flags and all timestamp fields also matched the Registry Explorer display. This cross-validation across all five accounts provides strong evidence that:

- The NK and VK cell parsing logic is accurate.
- The F value offset calculations (RID at +0x34, flags at +0x3C, counts at +0x40/0x42) are correct for this hive.
- The V value username extraction (using the pointer table at offset 0x0C + 0xCC) is correctly implemented.
- The filetime_to_str() conversion function correctly handles zero and sentinel values.

The verification step is a crucial part of forensic methodology: it demonstrates that the custom tool produces results consistent with an established, trusted tool, thereby validating its output for use in analysis.

Security Observations

The most significant forensic finding in this analysis is the PASSWORD_NOT_REQUIRED **flag (0x0004) set on the active user account (RID 1000, Angelo Calpoporo)**. This flag, when combined with NORMAL_ACCOUNT, means the account is configured to allow logon with a blank password.

Forensic Significance

In a live system, this configuration would represent a severe security vulnerability. An attacker who gains any foothold on the machine could potentially escalate privileges or maintain persistence by exploiting this weak authentication requirement. Alternatively, the flag could have been set intentionally by the user for convenience, indicating a lack of security awareness.

From a forensic investigation standpoint, this finding would trigger several lines of inquiry:

- Was the flag set by the user (e.g., via net user commands or Local Security Policy)?

- Could an attacker have modified the ACB flags to facilitate backdoor access?
- Are there other artefacts (e.g., Event Logs) that correlate with the timestamp of when the flag was set?
- Does this account have administrative privileges? (In this case, Angelo Calpoporo is a member of the Administrators group, making the empty-password vulnerability even more critical.)

Demonstration of Low-Level Forensic Value

This discovery was made possible only through direct binary parsing of the SAM hive. A high-level tool that simply lists users might not surface this detail, or might display it in a less prominent way. By examining the raw bytes at offset +0x3C of the F value and decoding each bit against the ACB flag reference, we obtained a precise, unambiguous view of the account's security configuration. This underscores the importance of low-level binary analysis in digital forensics: the most critical details are often hidden in plain sight within the byte structure of key artefacts.

Safety or Compliance Considerations (if applicable)

The following considerations governed the forensic handling of all materials throughout this project.

Consideration	Details
Read-Only Forensic Integrity	Arsenal Image Mounter was configured in read-only mode for all image mounting operations. No write operations were performed on the original disk image at any point. The SHA-256 hash of the image was verified before and after analysis to confirm no modification occurred.
Chain of Custody	The forensic disk image acquired in MP1/MP2 was stored on a dedicated project directory. Access was limited to group members. No copies were transferred to external storage without authorisation.
Handling of Sensitive Data	The SAM hive contains encrypted password hash blobs. While these blobs are RC4/AES-encrypted and cannot be used directly without the Syskey, they are still sensitive security artefacts. SAM_copy was stored only on group members' local project directories and was not uploaded to any shared cloud service.
No Password Cracking	In accordance with the project scope and academic integrity guidelines, no attempt was made to decrypt the NT or LM hash blobs, brute-force passwords, or pass-the-hash against any authentication service.
Academic Use Only	All forensic analysis was performed exclusively on a dedicated lab virtual machine created for this course. No analysis was performed on any production system, personal device, or system without explicit authorisation.
Tool Version Control	sam_parser.py is version-controlled with clear version numbers in the Change Control table. Only Python standard library modules were used, avoiding dependency on external packages that could introduce unvetted code.
Data Classification	This LAR is classified as Confidential – Academic Use Only. It contains hex-level representations of password hash blobs and should not be distributed outside the course evaluation context.

References

There are no sources in the current document.

The following references were consulted in the preparation of this report.

SANS Institute. (2024). SANS DFPS FOR500 v4.18: Windows Forensic Analysis Poster. SANS Digital Forensics & Incident Response. Retrieved from https://www.sans.org/posters/windows-forensic-analysis/

Carvey, H. (2012). Windows Registry Forensics: Advanced Digital Forensic Analysis of the Windows Registry (2nd ed.). Syngress / Elsevier.

Microsoft Corporation. (2022). [MS-SAMR]: Security Account Manager (SAM) Remote Protocol Specification (Client-to-Server). Microsoft Open Specifications. Retrieved from https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/

Libyal Project. (2023). libregf – Windows NT Registry File (REGF) Library. GitHub. Retrieved from https://github.com/libyal/libregf

Kornblum, J. (2002). Preservation of Fragile Digital Evidence by First Responders. Proceedings of the 2002 Digital Forensic Research Workshop (DFRWS). Retrieved from https://www.dfrws.org/sites/default/files/session-files/paper-preservation_of_fragile_digital_evidence_by_first_responders.pdf

Appendices

Appendix A – sam_parser.py Source Code

The following is the complete source code for sam_parser.py as submitted with Mini Project 3.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
r"""
SAM Hive Parser – Final Reliable Version with Field Mapping
NSSECU3 Mini Project 3
Author: Jose Angelo C. Calpoporo
Date: March 2026

Implements:
- Registry header parsing
- HBIN enumeration
- NK record parsing with sanity checks
- Subkey list parsing (lf, lh, li, ri)
- VK record parsing with strict validation
- Fallback scanning for VK records when value list is suspicious
- Recursive tree traversal from root
- Automatic extraction of user account data (F and V values)
- Authoritative username source: SAM\Domains\Account\Users\Names
- Hex dump output for manual Excel mapping
- CSV export of parsed user data (including hash candidates)
- **Field-level offset mapping** for Excel colour coding
"""

import sys
import struct
import csv
from datetime import datetime, timedelta

# ---------------------------------------------------------------------
# Constants
# ---------------------------------------------------------------------
REGF_MAGIC = b"regf"
HBIN_MAGIC = b"hbin"
NK_MAGIC = b"nk"
VK_MAGIC = b"vk"
LIST_MAGICS = {b'lf', b'lh', b'li', b'ri'}

FIRST_HBIN_ABS = 0x1000

# Registry value types (for display only)
REG_TYPES = {
    0: "REG_NONE", 1: "REG_SZ", 2: "REG_EXPAND_SZ", 3: "REG_BINARY",
    4: "REG_DWORD", 5: "REG_DWORD_BIG_ENDIAN", 6: "REG_LINK",
    7: "REG_MULTI_SZ", 8: "REG_RESOURCE_LIST", 9: "REG_FULL_RESOURCE_DESCRIPTOR",
    10: "REG_RESOURCE_REQUIREMENTS_LIST", 11: "REG_QWORD",
}
```

```
# Account flags (from Windows SDK)
UF_ACCOUNT_DISABLE = 0x00000002
UF_LOCKOUT = 0x00000010
UF_PASSWD_NOTREQD = 0x00000020
UF_PASSWD_CANT_CHANGE = 0x00000040
UF_NORMAL_ACCOUNT = 0x00000200
UF_DONT_EXPIRE_PASSWD = 0x00010000
UF_PASSWORD_EXPIRED = 0x00800000

# ---------------------------------------------------------------------
# Helper functions
# ---------------------------------------------------------------------
def u16(data, off): return struct.unpack_from("<H", data, off)[0]
def u32(data, off): return struct.unpack_from("<I", data, off)[0]
def i32(data, off): return struct.unpack_from("<i", data, off)[0]
def u64(data, off): return struct.unpack_from("<Q", data, off)[0]

def filetime_to_str(ft):
    if ft == 0 or ft == 0x7FFFFFFFFFFFFFFF:
        return "N/A"
    try:
        dt = datetime(1601, 1, 1) + timedelta(microseconds=ft / 10)
        return dt.strftime("%Y-%m-%d %H:%M:%S")
    except Exception:
        return str(ft)

def is_printable_ascii(bs):
    return bool(bs) and all(0x20 <= b <= 0x7E for b in bs)

def decode_name(raw):
    if not raw:
        return ""
    if is_printable_ascii(raw):
        return raw.decode("ascii", errors="ignore")
    try:
        s = raw.decode("utf-16le", errors="ignore").rstrip("\x00")
        if s and any(ch.isalnum() for ch in s):
            return s
    except Exception:
        pass
    return raw.decode("ascii", errors="ignore").rstrip("\x00")

def rel_to_abs(rel_off):
    if rel_off == 0xFFFFFFFF or rel_off == 0:
        return None
    return FIRST_HBIN_ABS + rel_off

def hex_dump(data, label="", start=0, length=None):
    """Print a classic hex dump with ASCII column for manual Excel mapping."""
    if length is None:
        length = len(data)
    print(f"\n{label} (offset 0x{start:04x}):")
    print("Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII")
    for i in range(0, length, 16):
        chunk = data[i:i+16]
        hex_part = ' '.join(f"{b:02x}" for b in chunk)
```

```
        ascii_part = ''.join(chr(b) if 32 <= b < 127 else '.' for b in chunk)
        print(f"{start+i:04x}    {hex_part:<47} {ascii_part}")

def entropyish(blob):
    """Simple heuristic: number of unique bytes."""
    return len(set(blob))

# ----------------------------------------------------------------------
# Automatic parsing functions
# ----------------------------------------------------------------------
def auto_find_utf16_string(data, target=None, min_len=3, max_len=64):
    """
    Find a UTF-16LE string in data.
    If target is provided, search for its exact UTF-16LE representation.
    Otherwise, return the first plausible string.
    Returns (start_offset, length_in_chars, string) or (None,0,None).
    """
    if target:
        # Try with null terminator
        target_bytes = target.encode("utf-16le") + b"\x00\x00"
        pos = data.find(target_bytes)
        if pos != -1:
            return pos, len(target), target
        # Try without null terminator
        target_bytes = target.encode("utf-16le")
        pos = data.find(target_bytes)
        if pos != -1:
            return pos, len(target), target
        return None, 0, None

    # Fallback: find first plausible string
    for i in range(0, len(data) - 2, 2):
        if data[i+1] == 0x00 and 0x20 <= data[i] <= 0x7E:
            j = i
            while j+1 < len(data) and data[j+1] == 0x00 and 0x20 <= data[j] <= 0x7E:
                j += 2
            length = (j - i) // 2
            if min_len <= length <= max_len:
                s = data[i:j+2].decode("utf-16le", errors="ignore").rstrip("\x00")
                return i, length, s
    return None, 0, None

def auto_parse_v(v_data, expected_username=None):
    """
    Extract username and hash blobs from V value data using the structured V header.

    The V value binary layout (all offsets relative to start of v_data which
    includes the 4-byte cell-size prefix):

        v_data[0:4]        – cell size (negative int32, skip it)
        v_data[4:]         – V payload (the actual SAM_USER_ACCOUNT_V struct)

    Within the V payload, each user-data field is described by a 12-byte entry
    (offset DWORD, length DWORD, type DWORD).  The table starts at V_payload+0x00
    and data strings are stored at  V_payload + 0xCC + field_offset.
```

```
    Entry positions in the header table (relative to V_payload start):
        +0x0C  username   (offset + length + type)
        +0x90  LM hash    (offset + length + type)
        +0x9C  NT hash    (offset + length + type)

    Returns dict: username, username_offset, lm_blob, nt_blob,
                  lm_offset, nt_offset  (all *absolute within v_data*).
    """
    # v_data includes the 4-byte cell header; V payload starts at offset 4
    V_HDR  = 4        # bytes to skip for cell header
    V_BASE = V_HDR + 0xCC   # base of the string/hash data area within v_data

    result = {
        "username": "",
        "username_offset": None,
        "lm_blob": b"",
        "nt_blob": b"",
        "lm_offset": None,
        "nt_offset": None,
    }

    # Minimum size check: need at least up to the NT-hash entry (+0x9C+12 = 0xA8)
    if len(v_data) < V_HDR + 0xA8:
        return None

    # ── Username (entry at V_payload+0x0C) ──────────────────────────────────
    usr_off = u32(v_data, V_HDR + 0x0C)   # relative offset inside data area
    usr_len = u32(v_data, V_HDR + 0x10)   # byte length of the UTF-16LE string

    if 0 < usr_len <= 512:
        ustart = V_BASE + usr_off
        if ustart + usr_len <= len(v_data):
            uname = v_data[ustart:ustart + usr_len].decode("utf-16le",
                                                           errors="ignore")
            result["username"] = uname
            result["username_offset"] = ustart

    if not result["username"]:
        return None   # no usable username → skip this record

    # ── LM hash blob (entry at V_payload+0x90) ──────────────────────────────
    lm_off = u32(v_data, V_HDR + 0x90)
    lm_len = u32(v_data, V_HDR + 0x94)
    if 0 < lm_len <= 64:
        lstart = V_BASE + lm_off
        if lstart + lm_len <= len(v_data):
            result["lm_blob"]   = v_data[lstart:lstart + lm_len]
            result["lm_offset"] = lstart

    # ── NT hash blob (entry at V_payload+0x9C) ──────────────────────────────
    nt_off = u32(v_data, V_HDR + 0x9C)
    nt_len = u32(v_data, V_HDR + 0xA0)
    if 0 < nt_len <= 64:
        nstart = V_BASE + nt_off
        if nstart + nt_len <= len(v_data):
```

```
        result["nt_blob"]   = v_data[nstart:nstart + nt_len]
        result["nt_offset"] = nstart

    return result

def auto_parse_f_improved(f_data, expected_rid):
    """
    Parse the F value using known structure of USER_ACCOUNT.
    Returns a dict with account flags, timestamps, etc., including offsets.
    """
    rid_bytes = struct.pack("<I", expected_rid)
    rid_off = None
    for off in range(len(f_data) - 3):
        if f_data[off:off+4] == rid_bytes:
            rid_off = off
            break
    if rid_off is None or rid_off < 0x30:
        return None

    struct_start = rid_off - 0x30  # approximate start of USER_ACCOUNT

    def get_dword(off):
        if struct_start + off + 4 <= len(f_data):
            return u32(f_data, struct_start + off)
        return 0
    def get_qword(off):
        if struct_start + off + 8 <= len(f_data):
            return u64(f_data, struct_start + off)
        return 0

    last_logon        = get_qword(0x08)   # payload+0x08  LastLogon FILETIME
    # payload+0x10 = LastLogoff (not displayed)
    last_pwd_set      = get_qword(0x18)   # payload+0x18  PasswordLastSet FILETIME
    account_expires   = get_qword(0x20)   # payload+0x20  AccountExpires FILETIME
    last_failed_logon = get_qword(0x28)   # payload+0x28  LastBadPasswordTime FILETIME
    # payload+0x30 = RID (anchor used above)
    # payload+0x34 = PrimaryGroupId  (NOT logon count!)
    # payload+0x38 = ACB flags
    # payload+0x3C = CountryCode (WORD) + CodePage (WORD)
    # payload+0x40 = BadPasswordCount (WORD)
    # payload+0x42 = LogonCount (WORD)
    rid               = get_dword(0x30)
    account_flags     = get_dword(0x38)   # ACB flags at +0x38
    failed_logon_count = struct.unpack_from("<H", f_data, struct_start + 0x40)[0]
    logon_count       = struct.unpack_from("<H", f_data, struct_start + 0x42)[0]

    # SAM ACB flag bit definitions (different from UF_ constants used in AD/LDAP)
    ACB_DISABLED   = 0x0001   # Account Disabled
    ACB_HOMEDIR    = 0x0002   # Home Directory Required
    ACB_PASSWD_NR  = 0x0004   # Password Not Required
    ACB_TEMP_DUP   = 0x0008   # Temp Duplicate Account
    ACB_NORMAL     = 0x0010   # Normal User Account
    ACB_MNS        = 0x0020   # MNS Logon Account
    ACB_DOMTRUST   = 0x0040   # Interdomain Trust Account
    ACB_WSTRUST    = 0x0080   # Workstation Trust Account
    ACB_SVRTRUST   = 0x0100   # Server Trust Account
```

```
    ACB_PWNOEXP    = 0x0200   # Password Does Not Expire
    ACB_AUTOLOCK   = 0x0400   # Account Auto Locked

    flags_list = []
    if account_flags & ACB_DISABLED:  flags_list.append("DISABLED")
    if account_flags & ACB_HOMEDIR:   flags_list.append("HOMEDIR_REQUIRED")
    if account_flags & ACB_PASSWD_NR: flags_list.append("PASSWORD_NOT_REQUIRED")
    if account_flags & ACB_TEMP_DUP:  flags_list.append("TEMP_DUPLICATE")
    if account_flags & ACB_NORMAL:    flags_list.append("NORMAL_ACCOUNT")
    if account_flags & ACB_MNS:       flags_list.append("MNS_LOGON")
    if account_flags & ACB_DOMTRUST:  flags_list.append("INTERDOMAIN_TRUST")
    if account_flags & ACB_WSTRUST:   flags_list.append("WORKSTATION_TRUST")
    if account_flags & ACB_SVRTRUST:  flags_list.append("SERVER_TRUST")
    if account_flags & ACB_PWNOEXP:   flags_list.append("PASSWORD_NEVER_EXPIRES")
    if account_flags & ACB_AUTOLOCK:  flags_list.append("AUTO_LOCKED")

    return {
        "rid": rid,
        "account_flags": account_flags,
        "flags_readable": ", ".join(flags_list) if flags_list else "None",
        "last_logon": filetime_to_str(last_logon),
        "last_pwd_set": filetime_to_str(last_pwd_set),
        "account_expires": filetime_to_str(account_expires),
        "last_failed_logon": filetime_to_str(last_failed_logon),
        "failed_logon_count": failed_logon_count,
        "logon_count": logon_count,
        "offsets": {
            "struct_start": struct_start,
            "rid":              rid_off,
            "flags":            struct_start + 0x38,
            "last_logon":       struct_start + 0x08,
            "last_pwd_set":     struct_start + 0x18,
            "account_expires":  struct_start + 0x20,
            "last_failed_logon": struct_start + 0x28,
            "failed_logon_count":struct_start + 0x40,
            "logon_count":      struct_start + 0x42,
        }
    }

# ----------------------------------------------------------------------
# Data structures
# ----------------------------------------------------------------------
class NKRecord:
    def __init__(self, abs_off, rel_off, parent_rel, name, flags,
                 subkey_count, subkey_list_rel, value_count, value_list_rel, timestamp):
        self.abs_off = abs_off
        self.rel_off = rel_off
        self.parent_rel = parent_rel
        self.name = name
        self.flags = flags
        self.subkey_count = subkey_count
        self.subkey_list_rel = subkey_list_rel
        self.value_count = value_count
        self.value_list_rel = value_list_rel
        self.timestamp = timestamp
```

```
class VKRecord:
    def __init__(self, abs_off, rel_off, name, value_type, data_len, data_rel, inline, data):
        self.abs_off = abs_off
        self.rel_off = rel_off
        self.name = name
        self.value_type = value_type
        self.data_len = data_len
        self.data_rel = data_rel
        self.inline = inline
        self.data = data

# ---------------------------------------------------------------------
# Main parser class
# ---------------------------------------------------------------------
class SAMParser:
    def __init__(self, path):
        self.path = path
        with open(path, "rb") as f:
            self.data = f.read()
        self.hbins = []
        self.nk_by_abs = {}
        self.root_abs = None

    # ---------------------------------------------------------------------
    # Header and HBIN enumeration
    # ---------------------------------------------------------------------
    def parse_header(self):
        if self.data[0:4] != REGF_MAGIC:
            raise ValueError("Not a valid registry hive (missing regf)")

        seq1 = u32(self.data, 0x04)
        seq2 = u32(self.data, 0x08)
        last_write_ft = u64(self.data, 0x0C)
        major = u32(self.data, 0x14)
        minor = u32(self.data, 0x18)
        root_cell_rel = u32(self.data, 0x24)
        hive_bins_size = u32(self.data, 0x28)

        self.root_abs = root_cell_rel + FIRST_HBIN_ABS

        print("[*] Registry hive header")
        print(f"    Magic         : {self.data[0:4].decode(errors='ignore')}")
        print(f"    Sequence      : {seq1}:{seq2}")
        print(f"    Last write    : {filetime_to_str(last_write_ft)}")
        print(f"    Version       : {major}.{minor}")
        print(f"    Root cell rel : 0x{root_cell_rel:08X}")
        print(f"    Root cell abs : 0x{self.root_abs:08X}")
        print(f"    HBIN area size: 0x{hive_bins_size:08X}")

    def parse_hbins(self):
        print("\n[*] Enumerating HBIN blocks")
        off = FIRST_HBIN_ABS
        while off + 0x20 <= len(self.data):
            if self.data[off:off + 4] != HBIN_MAGIC:
                break
            rel = u32(self.data, off + 0x04)
```

```
        size = u32(self.data, off + 0x08)
        if size == 0:
            break
        self.hbins.append((off, rel, size))
        off += size
    print(f"[*] Found {len(self.hbins)} HBIN block(s)")
    for i, (abs_off, rel, size) in enumerate(self.hbins):
        print(f"    HBIN[{i}] abs=0x{abs_off:08X} rel=0x{rel:08X} size=0x{size:08X}")

# ----------------------------------------------------------------
# NK record parsing
# ----------------------------------------------------------------
def try_parse_nk(self, cell_abs):
    try:
        cell_size = i32(self.data, cell_abs)
        if cell_size >= 0:
            return None
        if self.data[cell_abs + 4:cell_abs + 6] != NK_MAGIC:
            return None

        flags = u16(self.data, cell_abs + 0x06)
        timestamp = u64(self.data, cell_abs + 0x08)
        parent_rel = u32(self.data, cell_abs + 0x14)
        subkey_count = u32(self.data, cell_abs + 0x18)
        subkey_list_rel = u32(self.data, cell_abs + 0x20)
        # +0x24 = volatile subkey list (always 0xFFFFFFFF if none) — NOT value_count
        # +0x28 = actual value count,  +0x2C = value list relative offset
        value_count = u32(self.data, cell_abs + 0x28)
        value_list_rel = u32(self.data, cell_abs + 0x2C)

        name_len = u16(self.data, cell_abs + 0x4C)
        name_start = cell_abs + 0x50
        name_end = name_start + name_len
        if name_end > len(self.data):
            return None
        raw_name = self.data[name_start:name_end]
        name = decode_name(raw_name).strip()

        rel_off = cell_abs - FIRST_HBIN_ABS
        return NKRecord(
            abs_off=cell_abs,
            rel_off=rel_off,
            parent_rel=parent_rel,
            name=name,
            flags=flags,
            subkey_count=subkey_count,
            subkey_list_rel=subkey_list_rel,
            value_count=value_count,
            value_list_rel=value_list_rel,
            timestamp=timestamp,
        )
    except Exception:
        return None

def scan_nks(self):
    print("\n[*] Scanning HBIN cells for NK records")
```

```
    for hbin_abs, _, hbin_size in self.hbins:
        cell = hbin_abs + 0x20
        hbin_end = hbin_abs + hbin_size
        while cell + 4 <= hbin_end and cell + 4 <= len(self.data):
            try:
                raw_size = i32(self.data, cell)
            except Exception:
                break
            if raw_size == 0:
                break
            step = abs(raw_size)
            if step < 4:
                break
            nk = self.try_parse_nk(cell)
            if nk:
                self.nk_by_abs[cell] = nk
            cell += step
    print(f"[*] Total NK records found: {len(self.nk_by_abs)}")

# ---------------------------------------------------------------------
# Subkey list parsing
# ---------------------------------------------------------------------
def parse_subkey_list(self, offset_rel, count):
    offset_abs = rel_to_abs(offset_rel)
    if offset_abs is None or offset_abs + 4 > len(self.data):
        return []
    magic = self.data[offset_abs + 4:offset_abs + 6]

    if magic not in LIST_MAGICS:
        offsets = []
        for i in range(count):
            off = u32(self.data, offset_abs + 8 + i * 4)
            if off == 0:
                break
            nk_abs = rel_to_abs(off)
            if nk_abs:
                offsets.append(nk_abs)
        return offsets

    if magic == b'ri':
        num_lists = u32(self.data, offset_abs + 8)
        offsets = []
        for i in range(num_lists):
            list_rel = u32(self.data, offset_abs + 12 + i * 4)
            offsets.extend(self.parse_subkey_list(list_rel, 0))
        return offsets

    entries = []
    for i in range(count):
        entry_off = offset_abs + 8 + i * 8
        if entry_off + 4 > len(self.data):
            break
        nk_rel = u32(self.data, entry_off)
        if nk_rel == 0:
            break
        nk_abs = rel_to_abs(nk_rel)
```

```
        if nk_abs:
            entries.append(nk_abs)
    return entries

# ----------------------------------------------------------------
# Tree traversal
# ----------------------------------------------------------------
def get_key_path(self, nk):
    parts = []
    cur = nk
    seen = set()
    while cur is not None:
        if cur.rel_off in seen:
            break
        seen.add(cur.rel_off)
        if cur.parent_rel == 0 or cur.parent_rel == 0xFFFFFFFF:
            if cur.abs_off == self.root_abs:
                parts.append("ROOT")
            else:
                parts.append(f"[UNKNOWN_ROOT_{cur.rel_off:08X}]")
            break
        else:
            parts.append(cur.name)
            parent_abs = rel_to_abs(cur.parent_rel)
            cur = self.nk_by_abs.get(parent_abs, None)
            if cur is None:
                break
    return "\\".join(reversed(parts))

def find_key_by_path(self, path):
    parts = path.split('\\')
    if parts[0] != "ROOT":
        return None
    current_nk = self.nk_by_abs.get(self.root_abs)
    if not current_nk:
        return None
    for part in parts[1:]:
        if current_nk.subkey_count == 0:
            return None
        child_offsets = self.parse_subkey_list(current_nk.subkey_list_rel, current_nk.subkey_count)
        found = False
        for child_abs in child_offsets:
            child = self.nk_by_abs.get(child_abs)
            if child and child.name == part:
                current_nk = child
                found = True
                break
        if not found:
            return None
    return current_nk

# ----------------------------------------------------------------
# VK record parsing with strict validation
# ----------------------------------------------------------------
def parse_vk(self, vk_abs):
    try:
```

```
        cell_size = i32(self.data, vk_abs)
        if cell_size >= 0:
            return None
        if self.data[vk_abs + 4:vk_abs + 6] != VK_MAGIC:
            return None
        name_len = u16(self.data, vk_abs + 0x06)
        if name_len > 512:
            return None
        data_len_raw = u32(self.data, vk_abs + 0x08)
        data_rel = u32(self.data, vk_abs + 0x0C)
        value_type = u32(self.data, vk_abs + 0x10)
        flags = u16(self.data, vk_abs + 0x14)

        name_start = vk_abs + 0x18
        name_end = name_start + name_len
        if name_end > len(self.data):
            return None
        raw_name = self.data[name_start:name_end]

        if flags & 0x0001:
            name = raw_name.decode("ascii", errors="ignore").rstrip("\x00")
        else:
            name = decode_name(raw_name)

        inline = bool(data_len_raw & 0x80000000)
        data_len = data_len_raw & 0x7FFFFFFF

        data = b""
        if inline:
            data = struct.pack("<I", data_rel)[:data_len]
        else:
            data_abs = rel_to_abs(data_rel)
            if data_abs is not None and data_len > 0 and data_abs + data_len <= len(self.data):
                data = self.data[data_abs:data_abs + data_len]
            else:
                data = b""

        rel_off = vk_abs - FIRST_HBIN_ABS
        return VKRecord(
            abs_off=vk_abs,
            rel_off=rel_off,
            name=name,
            value_type=value_type,
            data_len=data_len,
            data_rel=data_rel,
            inline=inline,
            data=data,
        )
    except Exception:
        return None

def scan_vk_near_nk(self, nk, max_bytes=0x200):
    """Scan for VK records starting just after the NK cell."""
    vks = []
    cell_size = abs(i32(self.data, nk.abs_off))
    start = nk.abs_off + cell_size
```

```
        end = min(start + max_bytes, len(self.data))
        pos = start
        while pos + 8 < end:
            cell_sz = i32(self.data, pos)
            if cell_sz == 0:
                break
            if cell_sz < 0 and self.data[pos+4:pos+6] == VK_MAGIC:
                vk = self.parse_vk(pos)
                if vk:
                    vks.append(vk)
            pos += abs(cell_sz)
        return vks

    def get_vk_list(self, nk):
        vks = []
        # With corrected NK offsets, value_count is now accurate.
        # Keep the guard for genuinely corrupt hives (threshold raised to 128).
        if nk.value_count > 128 or nk.value_count == 4294967295:
            print(f"    [!] Suspicious value_count={nk.value_count}, scanning nearby...")
            return self.scan_vk_near_nk(nk)

        if nk.value_count == 0 or nk.value_list_rel in (0, 0xFFFFFFFF):
            return vks

        list_abs = rel_to_abs(nk.value_list_rel)
        if list_abs is None:
            return vks

        for i in range(nk.value_count):
            # +4 to skip the 4-byte cell-size header that precedes the pointer array
            off_pos = list_abs + 4 + i * 4
            if off_pos + 4 > len(self.data):
                break
            vk_rel = u32(self.data, off_pos)
            if vk_rel == 0xFFFFFFFF:
                continue
            vk_abs = rel_to_abs(vk_rel)
            if vk_abs is None:
                continue
            vk = self.parse_vk(vk_abs)
            if vk:
                vks.append(vk)
        return vks

    # ---------------------------------------------------------------------
    # Map RID to username from Names subkey (authoritative)
    # ---------------------------------------------------------------------
    def map_rid_to_username(self):
        r"""Return dict {rid: username} from SAM\Domains\Account\Users\Names."""
        mapping = {}
        names_nk = self.find_key_by_path("ROOT\\SAM\\Domains\\Account\\Users\\Names")
        if not names_nk:
            return mapping
        name_offsets = self.parse_subkey_list(names_nk.subkey_list_rel, names_nk.subkey_count)
        for off in name_offsets:
            name_nk = self.nk_by_abs.get(off)
```

```
        if not name_nk:
            continue
        # The default value contains the RID (binary DWORD)
        vks = self.get_vk_list(name_nk)
        for vk in vks:
            # In SAM Names subkeys the default value's TYPE field holds the RID
            # (data_len is 0 / inline flag set; actual type encodes the RID integer)
            if vk.name == "":
                rid = vk.value_type   # overloaded: type == RID
                if rid > 0:
                    mapping[rid] = name_nk.name
                break
    return mapping

# ----------------------------------------------------------------------
# New methods for field mapping
# ----------------------------------------------------------------------
def _get_bytes_hex(self, abs_off, length):
    """Return a hex string of `length` bytes starting at `abs_off`."""
    if abs_off + length > len(self.data):
        return "(out of range)"
    return ' '.join(f"{b:02x}" for b in self.data[abs_off:abs_off+length])

def _print_field_mapping(self, rid, username, nk_abs, f_abs, f_info, v_abs, v_info):
    """Print a nicely formatted table of absolute offsets for key fields."""
    print("\n" + "="*60)
    print(f"FIELD MAPPING FOR RID {rid} ({username})")
    print("="*60)
    print(f"{'Field':<25} {'Abs Offset':<12} {'Bytes (hex)':<50}")
    print("-"*90)

    # NK record (key)
    print(f"{'NK record start':<25} {f'0x{nk_abs:04X}':<12} {self._get_bytes_hex(nk_abs, 4)} ...")
    # NK timestamp (at +0x08)
    ts_abs = nk_abs + 0x08
    print(f"{'  NK timestamp':<25} {f'0x{ts_abs:04X}':<12} {self._get_bytes_hex(ts_abs, 8)}")

    # F value data
    print(f"{'F value start':<25} {f'0x{f_abs:04X}':<12} {self._get_bytes_hex(f_abs, 4)} ...")
    # Cell size (first 4 bytes of F)
    print(f"{'  Cell size':<25} {f'0x{f_abs:04X}':<12} {self._get_bytes_hex(f_abs, 4)}")
    if f_info:
        off = f_info['offsets']
        # RID
        rid_abs = f_abs + off['rid']
        print(f"{'  RID':<25} {f'0x{rid_abs:04X}':<12} {self._get_bytes_hex(rid_abs, 4)}")
        # Flags
        flags_abs = f_abs + off['flags']
        print(f"{'  Flags':<25} {f'0x{flags_abs:04X}':<12} {self._get_bytes_hex(flags_abs, 4)}")
        # Timestamps
        last_logon_abs = f_abs + off['last_logon']
        print(f"{'  Last logon':<25} {f'0x{last_logon_abs:04X}':<12} {self._get_bytes_hex(last_logon_abs,
8)}")
        last_pwd_abs = f_abs + off['last_pwd_set']
        print(f"{'  Last pwd set':<25} {f'0x{last_pwd_abs:04X}':<12} {self._get_bytes_hex(last_pwd_abs,
8)}")
```

```
            expires_abs = f_abs + off['account_expires']
            print(f"{'  Account expires':<25} {f'0x{expires_abs:04X}':<12} {self._get_bytes_hex(expires_abs,
8)}")
            last_fail_abs = f_abs + off['last_failed_logon']
            print(f"{'  Last failed logon':<25} {f'0x{last_fail_abs:04X}':<12} {self._get_bytes_hex(last_fail_abs,
8)}")
            fail_count_abs = f_abs + off['failed_logon_count']
            print(f"{'  Failed logon count':<25} {f'0x{fail_count_abs:04X}':<12}
{self._get_bytes_hex(fail_count_abs, 4)}")
            logon_count_abs = f_abs + off['logon_count']
            print(f"{'  Logon count':<25} {f'0x{logon_count_abs:04X}':<12}
{self._get_bytes_hex(logon_count_abs, 4)}")

        # V value data
        print(f"{'V value start':<25} {f'0x{v_abs:04X}':<12} {self._get_bytes_hex(v_abs, 4)} ...")
        print(f"{'  Cell size':<25} {f'0x{v_abs:04X}':<12} {self._get_bytes_hex(v_abs, 4)}")
        if v_info and v_info['username_offset'] is not None:
            user_abs = v_abs + v_info['username_offset']
            # Show up to 16 bytes of username
            user_len = min(16, len(v_info['username'])*2)
            print(f"{'  Username':<25} {f'0x{user_abs:04X}':<12} {self._get_bytes_hex(user_abs, user_len)} ...")
        if v_info and v_info['lm_offset'] is not None:
            lm_abs = v_abs + v_info['lm_offset']
            print(f"{'  LM hash (candidate)':<25} {f'0x{lm_abs:04X}':<12} {self._get_bytes_hex(lm_abs, 16)}")
        if v_info and v_info['nt_offset'] is not None:
            nt_abs = v_abs + v_info['nt_offset']
            print(f"{'  NT hash (candidate)':<25} {f'0x{nt_abs:04X}':<12} {self._get_bytes_hex(nt_abs, 16)}")
        print("="*60 + "\n")

    # ----------------------------------------------------------------
    # User extraction (main) – modified to call mapping
    # ----------------------------------------------------------------
    def extract_users(self):
        print("\n[*] Extracting user account data from SAM\\Domains\\Account\\Users")
        users_nk = self.find_key_by_path("ROOT\\SAM\\Domains\\Account\\Users")
        if not users_nk:
            print("[-] Users key not found.")
            return

        print(f"[+] Found Users key at rel=0x{users_nk.rel_off:08X}")
        child_offsets = self.parse_subkey_list(users_nk.subkey_list_rel, users_nk.subkey_count)
        print(f"[+] Found {len(child_offsets)} subkeys under Users")

        # Build RID->username map from Names subkey (authoritative)
        rid_to_name = self.map_rid_to_username()
        if rid_to_name:
            print("[+] Authoritative username mapping from Names subkey:")
            for rid, name in sorted(rid_to_name.items()):
                print(f"    RID {rid} -> {name}")

        # Collect results for CSV export
        results = []

        for child_abs in child_offsets:
            nk = self.nk_by_abs.get(child_abs)
            if not nk or len(nk.name) != 8:
```

```
        continue
    try:
        rid = int(nk.name, 16)
    except ValueError:
        continue

    path = self.get_key_path(nk)
    vks = self.get_vk_list(nk)

    print(f"\n[+] RID key: {nk.name} (RID {rid})")
    print(f"    Path      : {path}")
    print(f"    Timestamp : {filetime_to_str(nk.timestamp)}")
    print(f"    Values    : {len(vks)}")

    if not vks:
        print("    [!] No VK records found for this RID key")
        continue

    f_vk = v_vk = None
    for vk in vks:
        type_name = REG_TYPES.get(vk.value_type, f"UNKNOWN({vk.value_type})")
        print(f"      VK name={vk.name!r} type={type_name} len={vk.data_len} inline={vk.inline}")
        if vk.name == "F":
            f_vk = vk
        elif vk.name == "V":
            v_vk = vk

    # Prepare user record (username from Names)
    username = rid_to_name.get(rid, "")
    user_rec = {
        "rid": rid,
        "username": username,          # authoritative
        "account_flags": "",
        "last_logon": "",
        "last_pwd_set": "",
        "account_expires": "",
        "last_failed_logon": "",
        "failed_logon_count": "",
        "logon_count": "",
        "lm_blob": "",
        "nt_blob": "",
        "v_username": "",              # from V (for verification)
        "v_match": False
    }

    f_abs_start = None
    v_abs_start = None
    f_info = None
    v_info = None

    # ----- F value -----
    if f_vk:
        # Absolute offset of F data
        f_abs_start = f_vk.data_rel + FIRST_HBIN_ABS
        hex_dump(f_vk.data, f"F value for {nk.name} (RID {rid})", f_abs_start)
        f_info = auto_parse_f_improved(f_vk.data, rid)
```

```
            if f_info:
                self._print_f_info(f_info)
                user_rec.update({
                    "account_flags": f_info['flags_readable'],
                    "last_logon": f_info['last_logon'],
                    "last_pwd_set": f_info['last_pwd_set'],
                    "account_expires": f_info['account_expires'],
                    "last_failed_logon": f_info['last_failed_logon'],
                    "failed_logon_count": f_info['failed_logon_count'],
                    "logon_count": f_info['logon_count'],
                })
            else:
                print("    Could not parse F value automatically. Use the hex dump above for manual
mapping.")

        # ----- V value -----
        if v_vk:
            v_abs_start = v_vk.data_rel + FIRST_HBIN_ABS
            hex_dump(v_vk.data, f"V value for {nk.name} (RID {rid})", v_abs_start)
            v_info = auto_parse_v(v_vk.data, expected_username=username if username else None)
            if v_info and v_info['username']:
                print(f"        [Auto] Found username at offset 0x{v_info['username_offset']:X}")
                self._print_v_info(v_info)
                user_rec.update({
                    "v_username": v_info['username'],
                    "v_match": (v_info['username'] == username),
                    "lm_blob": v_info.get('lm_blob', b'').hex(),
                    "nt_blob": v_info.get('nt_blob', b'').hex(),
                })
                if username and v_info['username'] != username:
                    print(f"        [!] Username mismatch: V says '{v_info['username']}', Names says
'{username}'")
            else:
                print("        Could not parse V value automatically. Use the hex dump above for manual
mapping.")

        # ----- Print field mapping for Excel -----
        if f_abs_start is not None and v_abs_start is not None:
            self._print_field_mapping(rid, username, nk.abs_off, f_abs_start, f_info, v_abs_start, v_info)

        results.append(user_rec)

    # Write CSV
    if results:
        csv_path = "sam_users.csv"
        with open(csv_path, "w", newline="", encoding="utf-8") as f:
            fieldnames = ["rid", "username", "account_flags", "last_logon", "last_pwd_set",
                          "account_expires", "last_failed_logon", "failed_logon_count",
                          "logon_count", "lm_blob", "nt_blob", "v_username", "v_match"]
            w = csv.DictWriter(f, fieldnames=fieldnames)
            w.writeheader()
            w.writerows(results)
        print(f"\n[+] CSV summary saved to {csv_path}")

def _print_f_info(self, f_info):
    print(f"        RID in F           : {f_info['rid']}")
```

```
        print(f"        Account flags      : 0x{f_info['account_flags']:08X} ({f_info['flags_readable']})")
        print(f"        Last logon         : {f_info['last_logon']}")
        print(f"        Last password set  : {f_info['last_pwd_set']}")
        print(f"        Account expires    : {f_info['account_expires']}")
        print(f"        Last failed logon  : {f_info['last_failed_logon']}")
        print(f"        Failed logon count : {f_info['failed_logon_count']}")
        print(f"        Logon count        : {f_info['logon_count']}")

    def _print_v_info(self, v_info):
        print(f"        Username  : {v_info['username']}")
        lm = v_info.get('lm_blob', b'')
        nt = v_info.get('nt_blob', b'')
        if lm:
            print(f"        LM blob   : {lm.hex()} (raw 16-byte candidate; may be encrypted)")
        if nt:
            print(f"        NT blob   : {nt.hex()} (raw 16-byte candidate; may be encrypted)")

    # ----------------------------------------------------------------
    # Search for interesting keys (optional)
    # ----------------------------------------------------------------
    def search_interesting(self):
        print("\n[*] Searching for interesting names directly")
        exact_targets = {"SAM", "Domains", "Account", "Users", "Names",
                         "Administrator", "Guest", "DefaultAccount"}
        found_exact = []
        found_rids = []
        found_probable = []

        for nk in self.nk_by_abs.values():
            if nk.name in exact_targets:
                found_exact.append(nk)
            if len(nk.name) == 8:
                try:
                    rid = int(nk.name, 16)
                    found_rids.append((nk, rid))
                except ValueError:
                    pass
            if nk.name and any(c.isalpha() for c in nk.name):
                if nk.name not in {"ROOT", "RXACT"} and len(nk.name) < 64:
                    found_probable.append(nk)

        if found_exact:
            print("\n[+] Exact target names found")
            for nk in found_exact:
                print(f"    name={nk.name} rel=0x{nk.rel_off:08X} parent=0x{nk.parent_rel:08X}
subkeys={nk.subkey_count} values={nk.value_count}")
        if found_rids:
            print("\n[+] RID-style keys found")
            for nk, rid in found_rids:
                print(f"    name={nk.name} RID={rid} rel=0x{nk.rel_off:08X} parent=0x{nk.parent_rel:08X}
values={nk.value_count}")
        if found_probable:
            print("\n[+] Likely user/group names")
            seen = set()
            for nk in found_probable:
```

```
            if nk.name not in seen:
                seen.add(nk.name)
                print(f"    {nk.name}")

    # ----------------------------------------------------------------
    # Main entry point
    # ----------------------------------------------------------------
    def run(self):
        self.parse_header()
        self.parse_hbins()
        self.scan_nks()
        self.search_interesting()
        self.extract_users()


# ----------------------------------------------------------------------
# Main
# ----------------------------------------------------------------------
def main():
    if len(sys.argv) != 2:
        print("Usage: py sam_parser.py <SAM_hive_file>")
        sys.exit(1)

    parser = SAMParser(sys.argv[1])
    parser.run()


if __name__ == "__main__":
    main()
```

Appendix B – SAM HIVE MAP

B.1 Overview of all users

The Overview — All Users sheet provides a high-level summary of the Windows SAM hive structure extracted from the victim machine. This sheet consolidates metadata obtained from the regf header and the HBIN allocation blocks to verify that the registry hive is valid and structurally consistent before deeper parsing is performed.

Key metadata fields displayed in this sheet include the registry magic signature (regf), version number, sequence numbers, root cell offset, and HBIN allocation size. These values confirm that the file is a legitimate Windows registry hive and that the hive structure is intact. For example, the HBIN area size and block count indicate how many allocation blocks exist in the hive and help verify that the parser enumerated all cells correctly.

This sheet also summarizes the user accounts discovered during parsing, including their Relative Identifiers (RIDs) and associated usernames. By presenting these results in a consolidated format, the

overview sheet provides investigators with a quick verification that the parser successfully located the expected user entries within the SAM hive.

Overall, this sheet serves as the initial validation layer for the forensic workflow, ensuring that the hive structure is correct before interpreting individual account records.

SAM HIVE FORENSIC ANALYSIS — MP3 | Windows Registry User Account Mapping

Parsed from: SAM_copy | Machine: S-1-5-21-3017025782-4202564144-487505969 | Parsed: 2026-02-13

HIVE HEADER INFORMATION	
Magic	regf
Sequence	26:26 (primary = backup, consistent)
Version	1.5
Root Cell Rel	0x00000020
Root Cell Abs	0x00001020
HBIN Area Size	0x00009000 (36,864 bytes)
HBIN Blocks	6 blocks found
NK Records	80 total NK records scanned
Domain SID	S-1-5-21-3017025782-4202564144-487505969

ALL USER ACCOUNTS — PARSED FIELDS

RID	Username	RID (hex)	Status	Account Flags	Last Logon	Last Pwd Set	Account Expires	Last Failed	Fail Count	Logon Count	LM Hash (enc.)	NT Hash (enc.)	F Offset	V Offset
500	Administrator	0x000001F4	DISABLED	DISABLED | NORMAL | PWD_NEVER_EXPIRES	N/A	N/A	Never	N/A	0	0	encrypted	4596080c…5f3c	0x4080	0x4B38
501	Guest	0x000001F5	DISABLED	DISABLED | PWD_NOT_REQ | NORMAL | PWD_NEVER_EXPIRES	N/A	N/A	Never	N/A	0	0	encrypted	e76cf1a8…6e9c	0x4488	0x52D0
503	DefaultAccount	0x000001F7	DISABLED	DISABLED | PWD_NOT_REQ | NORMAL | PWD_NEVER_EXPIRES	N/A	N/A	Never	N/A	0	0	encrypted	b55f32e2…3c1d	0x4840	0x5020
504	WDAGUtilityAccount	0x000001F8	DISABLED	DISABLED | NORMAL	N/A	2026-02-04 00:16:09	Never	N/A	0	0	encrypted	3a130f7c…e79b	0x3E60	0x84B8
1000	Angelo Calpoporo	0x000003E8	ACTIVE	PWD_NOT_REQ | NORMAL	2026-02-13 06:29:34	2026-02-03 08:17:42	Never	N/A	0	5	encrypted	29edf989…15fc	0x7078	0x79E0

B.2 Legend and Color Guide

The Legend & Color Guide sheet explains the colour-coding scheme used throughout the Excel byte maps. Each colour corresponds to a specific field type within the registry hive structure, enabling investigators to quickly identify the semantic meaning of each byte in the hex dump.

For example, magic numbers such as regf, hbin, nk, and vk are highlighted to indicate structural boundaries within the registry hive. Fields that represent timestamps stored as FILETIME values are colored differently from account identifiers or flags, allowing the reader to visually distinguish between metadata categories.

Additional colours identify important forensic artifacts such as:

- RID values, which uniquely identify each user account
- Account control flags (ACB flags) indicating account status and security settings
- Username strings stored in UTF-16LE
- Encrypted NT and LM hash blobs
- Logon counters and authentication metadata

By using this colour mapping system, the workbook transforms a raw hexadecimal dump into a visually interpretable forensic artifact, allowing investigators to trace how each parsed value corresponds to the underlying registry bytes.

SAM HIVE BYTE MAPPING — LEGEND & COLOR GUIDE

Windows Security Account Manager (SAM) Registry Hive — Field Color Coding Reference

Field / Category	Description	Example Value
Magic / Signature	Registry hive magic bytes (regf / hbin)	72 65 67 66
Cell Size	Size of the registry cell (negated for active)	A8 FF FF FF
NK Timestamp	Last write time of the NK record (FILETIME)	99 BE 9D 97 6B 95 DC 01
RID (Relative Identifier)	Windows user/group RID stored little-endian	F4 01 00 00 → RID 500
Account Flags	Bitmask: DISABLED, NORMAL, PWD_NEVER_EXPIRES ...	11 02 00 00
Last Logon Time	FILETIME of most recent interactive logon	C4 29 47 1E B2 9C DC 01
Last Password Set	FILETIME when password was last changed	9B 25 59 91 E5 94 DC 01
Account Expires	FILETIME; 7FFFFFFFFFFFFFFF = never expires	FF FF FF FF FF FF FF 7F
Last Failed Logon	FILETIME of most recent failed logon attempt	00 00 00 00 00 00 00 00
Failed Logon Count	Number of consecutive failed logon attempts	00 00
Logon Count	Total successful interactive logon count	05 00
Username (UTF-16LE)	Account name encoded in UTF-16 LE in V value	41 00 6E 00 67 00 ...
LM Hash (encrypted)	LAN Manager hash blob — RC4/AES encrypted in SAM	01 02 00 00 07 00 00 00 ...
NT Hash (encrypted)	NTLM hash blob — RC4/AES encrypted in SAM	02 00 02 00 00 00 00 00 ...
Reserved / Unknown	Padding or reserved bytes with no parsed meaning	00 00 00 00
Structural Padding	Alignment padding bytes in V value header	00 00 00 00

ACCOUNT FLAG BITMASK REFERENCE (F Value, offset +0x3C)

Flag Name	Hex Bitmask	Meaning
ACCOUNTDISABLE	0x0001	Account is disabled
HOMEDIR_REQUIRED	0x0002	Home directory required
PASSWORD_NOT_REQUIRED	0x0004	No password required to log in
TEMP_DUPLICATE_ACCOUNT	0x0008	Temporary duplicate account
NORMAL_ACCOUNT	0x0010	Standard user account type
MNS_LOGON_ACCOUNT	0x0020	MNS logon account
INTERDOMAIN_TRUST	0x0040	Interdomain trust account
WORKSTATION_TRUST	0x0080	Workstation trust account
SERVER_TRUST	0x0100	Server trust account
DONT_EXPIRE_PASSWORD	0x0200	Password never expires
ACCOUNT_AUTO_LOCKED	0x0400	Account is auto-locked
ENCRYPTED_TEXT_PWD	0x0800	Encrypted text password allowed
SMARTCARD_REQUIRED	0x1000	Smart card required for logon
TRUSTED_FOR_DELEGATION	0x2000	Trusted for Kerberos delegation
NOT_DELEGATED	0x4000	Account is sensitive and cannot be delegated
USE_DES_KEY_ONLY	0x8000	Use only DES encryption keys

B.3 F and V Value Hex Dump and Field Map (Administrator, RID 500)

The RID 500 sheet documents the byte-level structure of the built-in Administrator account within the SAM hive. RID 500 is a well-known Windows identifier assigned to the default administrative account on local systems.

The sheet contains the hex dump of the F and V registry values associated with this account. The F value stores account metadata such as timestamps, account flags, logon counts, and failure counters. These

fields reveal important authentication and account management information, including when the account was last used and whether any login failures occurred.

The V value contains user-specific data, including the username and encrypted password hash blobs. Although the encrypted hashes are not decrypted in this project, their presence confirms that the SAM hive contains authentication material used by Windows.

SAM HIVE BYTE MAP — RID 500 (Administrator)
Windows SAM Registry Hive | F Value & V Value Parsed Field Mapping

ACCOUNT SUMMARY

Username	Administrator
RID (decimal)	500
RID (hex)	0x000001F4
Account Flags	0x00000211 — DISABLED | NORMAL_ACCOUNT | PASSWORD_NEVER_EXPIRES
Last Logon	N/A (never logged on)
Last Password Set	N/A
Account Expires	Never (0x7FFFFFFFFFFFFFFF)
Last Failed Logon	N/A
Failed Logon Count	0
Logon Count	0

F VALUE (abs offset 0x4080) — Account Metadata

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
4080	A8	FF	FF	FF	04	00	01	00	00	00	00	00	00	00	00	00	.....
4090	00	00	00	00	03	00	00	00	00	00	00	00	00	00	00	00	.....
40A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.....
40B0	00	00	00	00	FF	FF	FF	FF	FF	FF	7F	00	00	00	00	00	.....
40C0	00	00	00	00	F4	01	00	00	01	02	00	00	11	02	00	00	.....
					00	00	00	00	01	00	00	00	00	00	00	00	.....

F VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
cell_sz	0x4080	+0x00	4	A8 FF FF FF	—	cell_sz
Last Logon	0x4088	+0x08	8	00 00 00 00 00 00 00 00	—	logon
Last Pwd Set	0x4098	+0x18	8	00 00 00 00 00 00 00 00	—	pwd_set
Expires	0x40A0	+0x20	8	00 00 00 00 FF FF FF FF	0x7FFFFFFFFFFFFFFF → Never Expires	expires
Last Failed	0x40A8	+0x28	8	FF FF FF 7F 00 00 00 00	—	fail_logon
RID	0x40B4	+0x34	4	F4 01 00 00	500 (0x01F4) — Built-in Administrator	rid
Flags	0x40BC	+0x3C	4	11 02 00 00	0x0211 → DISABLED(0x0001) + NORMAL(0x0010) + PWD_NEVER_EXPIRE(0x0200)	flags
Fail Count	0x40C0	+0x40	4	00 00 00 00	—	fail_cnt
Logon Count	0x40C4	+0x44	4	00 00 00 00	—	logon_cnt

V VALUE (abs offset 0x4CFC) — Username & Password Hashes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
4CFC	41	00	64	00	6D	00	69	00	6E	00	69	00	73	00	74	00	A.d.m.i.n.i.s.t.
4D0C	72	00	61	00	74	00	6F	00	72	00	01	02	00	00	07	00	r.a.t.o.r.....
4D1C	00	00	02	00	02	00	00	00	00	00	45	96	08	0C	3D	30	.....E...=0
4D2C	E6	05	25	37	FC	18	6D	8E	5F	3C	02	00	02	00	00	00	..%7..m_.<.....
4D3C	00	00	1D	E7	52	38	3B	C8	E9	99	50	73	8C	8D	80	56	....R8;...Ps...V
4D4C	7E	53															~S

V VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
cell_sz	0x4CFC	+0x00	4	41 00 64 00	—	cell_sz
Username	0x4D0C	+0x10	26	72 00 61 00 74 00 6F 00 72 00 01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00	Administrator [UTF-16LE: 41 00 64 00 … 72 00]	username
LM Blob Hdr	0x4D26	+0x2A	8	45 96 08 0C 3D 30 E6 05	—	lm_hash
gap	0x4D2E	+0x32	8	25 37 FC 18 6D 8E 5F 3C	—	reserved
NT Hash	0x4D36	+0x3A	16	02 00 02 00 00 00 00 00 1D E7 52 38 3B C8 E9 99	4596080c3d30e6052537fc186d8e5f3c (RC4/AES encrypted in SAM)	nt_hash

B.4 F and V Value Hex Dump and Field Map (Guest, RID 501)

The RID 501 sheet represents the Windows Guest account, which is a built-in account typically disabled by default in modern Windows systems. This account is designed to allow temporary or limited access to the system without requiring a permanent user profile.

The Excel byte map illustrates the internal structure of the Guest account's F and V values. Through colour-coded mapping, the sheet highlights fields such as the RID, logon counters, account flags, and timestamps. In many systems, the Guest account will show minimal activity or unused timestamp fields, which may appear as zeroed FILETIME values.

From a forensic perspective, examining the Guest account is important because attackers sometimes enable or abuse this account to gain access while avoiding detection. By analyzing the metadata associated with RID 501, investigators can determine whether the account has been used, disabled, or modified during the system's operational history.

SAM HIVE BYTE MAP — RID 501 (Guest)
Windows SAM Registry Hive | F Value & V Value Parsed Field Mapping

ACCOUNT SUMMARY	
Username	Guest
RID (decimal)	501
RID (hex)	0x000001F5
Account Flags	0x00000215 — DISABLED | PASSWORD_NOT_REQUIRED | NORMAL_ACCOUNT | PASSWORD_NEVER_EXPIRES
Last Logon	N/A (never logged on)
Last Password Set	N/A
Account Expires	Never (0x7FFFFFFFFFFFFFFF)
Last Failed Logon	N/A
Failed Logon Count	0
Logon Count	0

F VALUE (abs offset 0x4488) — Account Metadata

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
4488	A8	FF	FF	FF	03	00	01	00	00	00	00	00	00	00	00	00	.....
4498	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.....
44A8	00	00	00	00	FF	FF	FF	FF	FF	FF	7F	00	00	00	00	00	.....
44B8	00	00	00	00	F5	01	00	00	01	02	00	00	15	02	00	00	.....
44C8	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.....

F VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
cell_sz	0x4488	+0x00	4	A8 FF FF FF	—	cell_sz
Last Logon	0x4490	+0x08	8	00 00 00 00 00 00 00 00	—	logon
Last Pwd Set	0x44A0	+0x18	8	00 00 00 00 00 00 00 00	—	pwd_set
Expires	0x44A8	+0x20	8	00 00 00 00 FF FF FF FF	0x7FFFFFFFFFFFFFFF → Never Expires	expires
Last Failed	0x44B0	+0x28	8	FF FF FF 7F 00 00 00 00	—	fail_logon
RID	0x44BC	+0x34	4	F5 01 00 00	501 (0x01F5) — Built-in Guest	rid
Flags	0x44C4	+0x3C	4	15 02 00 00	0x0215 → DISABLED(0x0001) + PWD_NOT_REQUIRED(0x0004) + NORMAL(0x0010) + PWD_NEVER_EXPIRE(0x0200)	flags
Fail Count	0x44C8	+0x40	4	00 00 00 00	—	fail_cnt
Logon Count	0x44CC	+0x44	4	00 00 00 00	—	logon_cnt

V VALUE (abs offset 0x5488) — Username & Password Hashes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
5488	47	00	75	00	65	00	73	00	74	00	00	00	01	02	00	00	G.u.e.s.t.....
5498	07	00	00	00	02	00	02	00	00	00	00	00	E7	6C	F1	A8	.....l..
54A8	44	83	09	69	4F	F4	09	50	FE	43	6E	9C	02	00	02	00	D..iO..P.Cn.....
54B8	00	00	00	00	FC	1F	8E	CA	18	4F	32	49	7C	27	6E	AA	.....O2I|'n.
54C8	7C	40	E5	E7													|@..

V VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
Username	0x5488	+0x00	10	47 00 75 00 65 00 73 00 74 00	Guest [UTF-16LE: 47 00 75 00 65 00 73 00 74 00]	username
LM Blob Hdr	0x5494	+0x0C	8	01 02 00 00 07 00 00 00	—	lm_hash
gap	0x549C	+0x14	8	02 00 02 00 00 00 00 00	—	reserved
NT Hash	0x54A4	+0x1C	16	E7 6C F1 A8 44 83 09 69 4F F4 09 50 FE 43 6E 9C	e76cf1a8448309694ff40950fe436e9c (RC4/AES encrypted in SAM)	nt_hash

B.5 F and V Value Hex Dump and Field Map (DefaultAccount, RID 503)

The RID 503 sheet documents the DefaultAccount, a special Windows system account introduced in modern versions of Windows. This account is used internally by the operating system for application management and system services.

Unlike standard user accounts, the DefaultAccount is typically hidden from normal login interfaces and is not intended for direct user interaction. Its presence in the SAM hive, however, provides valuable insight into the system's configuration and internal account management.

The byte map for RID 503 illustrates how Windows stores metadata for system-level accounts. Fields such as the RID, account flags, and timestamps are decoded directly from the registry bytes. The absence of significant logon activity or authentication events often confirms that the account is functioning purely as an internal system component rather than an interactive user account.

SAM HIVE BYTE MAP — RID 503 (DefaultAccount)

Windows SAM Registry Hive | F Value & V Value Parsed Field Mapping

ACCOUNT SUMMARY

Username	DefaultAccount
RID (decimal)	503
RID (hex)	0x000001F7
Account Flags	0x00000215 — DISABLED | PASSWORD_NOT_REQUIRED | NO PASSWORD_NEVER_EXPIRES
Last Logon	N/A
Last Password Set	N/A
Account Expires	Never (0x7FFFFFFFFFFFFFFF)
Last Failed Logon	N/A
Failed Logon Count	0
Logon Count	0

F VALUE (abs offset 0x4840) — Account Metadata

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
4840	A8	FF	FF	FF	03	00	01	00	00	00	00	00	00	00	00	00	.....
4850	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.....
4860	00	00	00	00	FF	FF	FF	FF	FF	FF	7F	00	00	00	00	00	.....
4870	00	00	00	00	F7	01	00	00	01	02	00	00	15	02	00	00	.....
4880	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.....

F VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
cell_sz	0x4840	+0x00	4	A8 FF FF FF	—	cell_sz
Last Logon	0x4848	+0x08	8	00 00 00 00 00 00 00 00	—	logon
Last Pwd Set	0x4858	+0x18	8	00 00 00 00 00 00 00 00	—	pwd_set
Expires	0x4860	+0x20	8	00 00 00 00 FF FF FF FF	0x7FFFFFFFFFFFFFFF → Never Expires	expires
Last Failed	0x4868	+0x28	8	FF FF FF 7F 00 00 00 00	—	fail_logon
RID	0x4874	+0x34	4	F7 01 00 00	503 (0x01F7) — DefaultAccount (system managed)	rid
Flags	0x487C	+0x3C	4	15 02 00 00	0x0215 → DISABLED(0x0001) + PWD_NOT_REQUIRED(0x0004) + NORMAL(0x0010) + PWD_NEVER_EXPIRE(0x0200)	flags
Fail Count	0x4880	+0x40	4	00 00 00 00	—	fail_cnt
Logon Count	0x4884	+0x44	4	00 00 00 00	—	logon_cnt

V VALUE (abs offset 0x51FC) — Username & Password Hashes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
51FC	44	00	65	00	66	00	61	00	75	00	6C	00	74	00	41	00	D.e.f.a.u.l.t.A.
520C	63	00	63	00	6F	00	75	00	6E	00	74	00	01	02	00	00	c.c.o.u.n.t.....
521C	07	00	00	00	02	00	02	00	00	00	00	00	B5	5F	32	E2	2
522C	E0	06	2E	F5	91	BE	AD	CA	BF	53	3C	1D	02	00	02	00	.....S<.....
523C	00	00	00	00	BD	B8	99	BA	1E	FC	5C	AC	5F	A6	FF	92	....._...
524C	56	01	D8	23													V..#

V VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
Username	0x51FC	+0x00	28	44 00 65 00 66 00 61 00 75 00 6C 00 74 00 41 00 63 00 63 00 6F 00 75 00 6E 00 74 00	DefaultAccount [UTF-16LE: 44 00 65 00 ... 74 00]	username
LM Blob Hdr	0x5218	+0x1C	8	01 02 00 00 07 00 00 00	—	lm_hash
gap	0x5220	+0x24	8	02 00 02 00 00 00 00 00	—	reserved
NT Hash	0x5228	+0x2C	16	B5 5F 32 E2 E0 06 2E F5 91 BE AD CA BF 53 3C 1D	b55f32e2e0062ef591beadcabf533c1d (RC4/AES encrypted in SAM)	nt_hash

B.6 F and V Value Hex Dump and Field Map (WDAGUtility, RID 504)

The RID 504 sheet corresponds to the WDAGUtilityAccount, which is associated with Windows Defender Application Guard (WDAG). This account is used by Windows to isolate untrusted applications within a virtualized environment for security purposes.

The byte-level mapping in this sheet shows how the SAM hive records metadata for this specialized account. As with other accounts, the F value contains timestamps, logon counters, and account flags, while the V value contains the account name and associated authentication data structures.

From a forensic standpoint, this account is particularly relevant when analyzing systems that utilize virtualization-based security features. Its presence indicates that the system may have been configured to run isolated browsing sessions or application containers, which can affect how user activity and malware behavior appear in forensic artifacts.

SAM HIVE BYTE MAP — RID 504 (WDAGUtilityAccount)
Windows SAM Registry Hive | F Value & V Value Parsed Field Mapping

ACCOUNT SUMMARY

Username	WDAGUtilityAccount
RID (decimal)	504
RID (hex)	0x000001F8
Account Flags	0x00000011 — DISABLED | NORMAL_ACCOUNT
Last Logon	N/A
Last Password Set	2026-02-04 00:16:09
Account Expires	Never (0x7FFFFFFFFFFFFFFF)
Last Failed Logon	N/A
Failed Logon Count	0
Logon Count	0

F VALUE (abs offset 0x3E60) — Account Metadata

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
3E60	A8	FF	FF	FF	03	00	01	00	00	00	00	00	00	00	00	00	.....
3E70	00	00	00	00	00	00	00	00	00	00	00	73	4C	42	76		.....sLBv
3E80	6B	95	DC	01	FF	FF	FF	FF	FF	FF	7F	00	00	00	00		k.....
3E90	00	00	00	00	F8	01	00	00	01	02	00	00	11	00	00		.....
3EA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		.....

F VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
cell_sz	0x3E60	+0x00	4	A8 FF FF FF	—	cell_sz
Last Logon	0x3E68	+0x08	8	00 00 00 00 00 00 00 00	—	logon
Last Pwd Set	0x3E78	+0x18	8	00 00 00 00 73 4C 42 76	2026-02-04 00:16:09 [FILETIME: 73 4C 42 76 6B 95 DC 01]	pwd_set
Expires	0x3E80	+0x20	8	6B 95 DC 01 FF FF FF FF	0x7FFFFFFFFFFFFFFF → Never Expires	expires
Last Failed	0x3E88	+0x28	8	FF FF FF 7F 00 00 00 00	—	fail_logon
RID	0x3E94	+0x34	4	F8 01 00 00	504 (0x01F8) — WDAGUtilityAccount	rid
Flags	0x3E9C	+0x3C	4	11 00 00 00	0x0011 → DISABLED(0x0001) + NORMAL_ACCOUNT(0x0010)	flags
Fail Count	0x3EA0	+0x40	4	00 00 00 00	—	fail_cnt
Logon Count	0x3EA4	+0x44	4	00 00 00 00	—	logon_cnt

V VALUE (abs offset 0x8694) — Username & Password Hashes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
8694	57	00	44	00	41	00	47	00	55	00	74	00	69	00	6C	00	W.D.A.G.U.t.i.l.
86A4	69	00	74	00	79	00	41	00	63	00	63	00	6F	00	75	00	i.t.y.A.c.c.o.u.
86B4	6E	00	74	00	01	02	00	00	07	00	00	00	02	00	02	00	n.t.....
86C4	00	00	00	00	3A	13	0F	7C	BD	71	A7	DA	90	15	E3	79	.....:.|.q.....y
86D4	F3	56	E7	9B	02	00	02	00	10	00	00	00	29	FE	BC	A3	.V.....)...
86E4	43	A4	4B	C3	CE	54	77	40	05	C1	CF	8B					C.K..Tw@....

V VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
Username	0x8694	+0x00	36	57 00 44 00 41 00 47 00 55 00 74 00 69 00 6C 00 69 00 74 00 79 00 41 00 63 00 63 00 6F 00 75 00 6E 00 74 00	WDAGUtilityAccount [UTF-16LE: 57 00 44 00 41 00 … 74 00]	username
LM Blob Hdr	0x86B8	+0x24	8	01 02 00 00 07 00 00 00	—	lm_hash
gap	0x86C0	+0x2C	8	02 00 02 00 00 00 00 00	—	reserved
NT Hash	0x86C8	+0x34	16	3A 13 0F 7C BD 71 A7 DA 90 15 E3 79 F3 56 E7 9B	3a130f7cbd71a7da9015e379f356e79b (RC4/AES encrypted in SAM)	nt_hash

B.7 F and V Value Hex Dump and Field Map (Angelo Calpoporo, RID 1000)

The RID 1000 sheet represents the primary user account created on the victim machine. Unlike the built-in system accounts, RID values starting at 1000 typically correspond to normal user accounts created during system setup or later user activity.

This sheet provides the most relevant forensic information because it reflects the actions of the actual system user. The byte mapping highlights critical fields extracted from the SAM hive, including:

- Last logon timestamp
- Last password change time
- Account flags
- Logon count and failed logon count
- Username stored in UTF-16LE

These values provide insights into how the account was used on the system. For example, the logon counters reveal how many times the account authenticated successfully, while timestamp fields indicate when the account last accessed the system.

SAM HIVE BYTE MAP — RID 1000 (Angelo Calpoporo)
Windows SAM Registry Hive | F Value & V Value Parsed Field Mapping

ACCOUNT SUMMARY

Username	Angelo Calpoporo
RID (decimal)	1000
RID (hex)	0x000003E8
Account Flags	0x00000014 — PASSWORD_NOT_REQUIRED | NORMAL_ACCOUNT
Last Logon	2026-02-13 06:29:34 [FILETIME: C4 29 47 1E B2 9C DC 01]
Last Password Set	2026-02-03 08:17:42 [FILETIME: 9B 25 59 91 E5 94 DC 01]
Account Expires	Never (0x0000000000000000)
Last Failed Logon	N/A
Failed Logon Count	0
Logon Count	5

F VALUE (abs offset 0x7078) — Account Metadata

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
7078	A8	FF	FF	FF	03	00	01	00	00	00	00	00	C4	29	47	1E	.....)G.
7088	B2	9C	DC	01	00	00	00	00	00	00	00	9B	25	59	91		.....%Y.
7098	E5	94	DC	01	00	00	00	00	00	00	00	00	00	00	00		.....
70A8	00	00	00	00	E8	03	00	00	01	02	00	00	14	00	00	00	.....
70B8	01	00	E4	04	00	00	05	00	01	00	00	00	00	00	00	00	.....

F VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
cell_sz	0x7078	+0x00	4	A8 FF FF FF	—	cell_sz
Last Logon	0x7080	+0x08	8	00 00 00 00 C4 29 47 1E	2026-02-13 06:29:34 [FILETIME little-endian]	logon
Last Pwd Set	0x7090	+0x18	8	00 00 00 00 9B 25 59 91	2026-02-03 08:17:42	pwd_set
Expires	0x7098	+0x20	8	E5 94 DC 01 00 00 00 00	0x0000000000000000 → Never Expires (zero = no expiry set)	expires
Last Failed	0x70A0	+0x28	8	00 00 00 00 00 00 00 00	—	fail_logon
RID	0x70AC	+0x34	4	E8 03 00 00	1000 (0x03E8) — Regular user (first non-default RID)	rid
Flags	0x70B4	+0x3C	4	14 00 00 00	0x0014 → PWD_NOT_REQUIRED(0x0004) + NORMAL_ACCOUNT(0x0010)	flags
Fail Count	0x70B8	+0x40	4	01 00 E4 04	0	fail_cnt
Logon Count	0x70BC	+0x44	4	00 00 05 00	5 (logged on 5 times)	logon_cnt

V VALUE (abs offset 0x7BA4) — Username & Password Hashes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
7BA4	41	00	6E	00	67	00	65	00	6C	00	6F	00	20	00	43	00	A.n.g.e.l.o. .C.
7BB4	61	00	6C	00	70	00	6F	00	70	00	6F	00	72	00	6F	00	a.l.p.o.p.o.r.o.
7BC4	01	02	00	00	07	00	00	00	02	00	02	00	00	00	00	00	.....
7BD4	29	ED	F9	89	18	C9	29	59	D7	A5	51	DD	4F	7F	15	FC	)....)Y..Q.O...
7BE4	02	00	02	00	10	00	00	00	8B	EC	6D	F2	87	3A	CD	FC	.....m..:..
7BF4	44	4F	8D	D4	51	70	AE	A0									DO..Qp..

V VALUE — FIELD DECODE

Field	Abs Offset	Rel Offset	Length (B)	Raw Bytes (hex)	Parsed Value	Color Code
Username	0x7BA4	+0x00	32	41 00 6E 00 67 00 65 00 6C 00 6F 00 20 00 43 00 61 00 6C 00 70 00 6F 00 70 00 6F 00 72 00 6F 00	Angelo Calpoporo [UTF-16LE: 41 00 6E 00 67 00 65 00 6C 00 6F 00 20 00 ...]	username
LM Blob Hdr	0x7BC4	+0x20	8	01 02 00 00 07 00 00 00	—	lm_hash
gap	0x7BCC	+0x28	8	02 00 02 00 00 00 00 00	—	reserved
NT Hash	0x7BD4	+0x30	16	29 ED F9 89 18 C9 29 59 D7 A5 51 DD 4F 7F 15 FC	29edf98918c92959d7a551dd4f7f15fc (RC4/AES encrypted in SAM)	nt_hash

Appendix C – Output

C.1 Registry Hive Header Summary

This group shows the high-level metadata of the registry hive. It verifies that the file being parsed is a valid Windows registry hive by checking the magic signature (regf), sequence numbers, root cell offsets, and total HBIN allocation area. These fields are important because they confirm the structural integrity of the SAM hive before any account-level parsing is performed.

```
[*] Registry hive header
    Magic         : regf
    Sequence      : 26:26
    Last write    : N/A
    Version       : 1.5
    Root cell rel : 0x00000020
    Root cell abs : 0x00001020
    HBIN area size: 0x00009000

[*] Enumerating HBIN blocks
[*] Found 6 HBIN block(s)
    HBIN[0] abs=0x00001000 rel=0x00000000 size=0x00001000
    HBIN[1] abs=0x00002000 rel=0x00001000 size=0x00001000
    HBIN[2] abs=0x00003000 rel=0x00002000 size=0x00002000
    HBIN[3] abs=0x00005000 rel=0x00004000 size=0x00002000
    HBIN[4] abs=0x00007000 rel=0x00006000 size=0x00001000
    HBIN[5] abs=0x00008000 rel=0x00007000 size=0x00002000

[*] Scanning HBIN cells for NK records
[*] Total NK records found: 80

[*] Searching for interesting names directly

[+] Exact target names found
    name=SAM rel=0x00000078 parent=0x00000020 subkeys=3 values=2
    name=Domains rel=0x00000400 parent=0x00000078 subkeys=2 values=1
    name=Users rel=0x000007C8 parent=0x00000488 subkeys=1 values=1
    name=Names rel=0x00000848 parent=0x000007C8 subkeys=0 values=1
    name=Names rel=0x00000958 parent=0x000008D0 subkeys=0 values=1
    name=Names rel=0x00000A60 parent=0x000009D8 subkeys=19 values=1
    name=Users rel=0x00000DC8 parent=0x00000A60 subkeys=0 values=1
    name=DefaultAccount rel=0x00001A40 parent=0x00002A08 subkeys=0 values=1
    name=Account rel=0x00002608 parent=0x00000400 subkeys=3 values=2
    name=Users rel=0x00002990 parent=0x00002608 subkeys=6 values=1
    name=Names rel=0x00002A08 parent=0x00002990 subkeys=5 values=1
    name=Names rel=0x00002B18 parent=0x00002A90 subkeys=1 values=1
    name=Names rel=0x00002C20 parent=0x00002B98 subkeys=0 values=1
    name=Administrator rel=0x00002F68 parent=0x00002A08 subkeys=0 values=1
    name=Guest rel=0x00003360 parent=0x00002A08 subkeys=0 values=1
```

The regf magic confirms that the file is a registry hive. The matching sequence numbers (26:26) indicate that the primary and backup sequence values are consistent, suggesting the hive is stable and was not mid-update when acquired. The root cell offset identifies where traversal of the registry structure begins. The HBIN area size indicates the total space used by hive bins, which hold the registry cells to be parsed.

C.2 HBIN Block Enumeration

This group lists all HBIN blocks found in the registry hive. HBIN blocks are the allocation containers that store registry cells such as keys, values, and subkey lists.

```
[*] Enumerating HBIN blocks
[*] Found 6 HBIN block(s)
    HBIN[0] abs=0x00001000 rel=0x00000000 size=0x00001000
    HBIN[1] abs=0x00002000 rel=0x00001000 size=0x00001000
    HBIN[2] abs=0x00003000 rel=0x00002000 size=0x00002000
    HBIN[3] abs=0x00005000 rel=0x00004000 size=0x00002000
    HBIN[4] abs=0x00007000 rel=0x00006000 size=0x00001000
    HBIN[5] abs=0x00008000 rel=0x00007000 size=0x00002000
```

HBIN blocks are essential because all NK, VK, and data cells are stored inside them. Enumerating these blocks allows the parser to identify where registry data structures are located in the file. The output confirms that the hive contains six HBIN blocks, which matches the expected layout for this SAM hive.

C.3 NK Record Scan Summary

This group shows the total number of NK (Named Key) records discovered during scanning. NK cells represent registry keys and are fundamental to navigating the hierarchical registry structure.

```
[*] Scanning HBIN cells for NK records
[*] Total NK records found: 80
```

The parser found 80 NK records in the hive. This indicates that the internal registry structure was successfully traversed and that the hive contains a sufficient number of registry key nodes to support normal Windows account and security structures.

C.4 Direct Search for Important Registry Names

This group shows the results of directly searching for registry key names relevant to SAM parsing, such as `SAM`, `Domains`, `Account`, `Users`, and `Names`.

```
[*] Searching for interesting names directly

[+] Exact target names found
    name=SAM rel=0x00000078 parent=0x00000020 subkeys=3 values=2
    name=Domains rel=0x00000400 parent=0x00000078 subkeys=2 values=1
    name=Users rel=0x000007C8 parent=0x00000488 subkeys=1 values=1
    name=Names rel=0x00000848 parent=0x000007C8 subkeys=0 values=1
    name=Names rel=0x00000958 parent=0x000008D0 subkeys=0 values=1
    name=Names rel=0x00000A60 parent=0x000009D8 subkeys=19 values=1
    name=Users rel=0x00000DC8 parent=0x00000A60 subkeys=0 values=1
    name=DefaultAccount rel=0x00001A40 parent=0x00002A08 subkeys=0 values=1
    name=Account rel=0x00002608 parent=0x00000400 subkeys=3 values=2
    name=Users rel=0x00002990 parent=0x00002608 subkeys=6 values=1
    name=Names rel=0x00002A08 parent=0x00002990 subkeys=5 values=1
    name=Names rel=0x00002B18 parent=0x00002A90 subkeys=1 values=1
    name=Names rel=0x00002C20 parent=0x00002B98 subkeys=0 values=1
    name=Administrator rel=0x00002F68 parent=0x00002A08 subkeys=0 values=1
    name=Guest rel=0x00003360 parent=0x00002A08 subkeys=0 values=1
```

This confirms that the parser successfully located the expected path within the SAM hive, especially `SAM\Domains\Account\Users` and `SAM\Domains\Account\Users\Names`. These keys are critical because the `Users` key stores RID-based account records, while the `Names` key provides the authoritative mapping between usernames and RIDs.

C.5 RID-Style Key Discovery

This group lists all subkeys whose names match RID-style hexadecimal identifiers. These keys represent user accounts, groups, aliases, and built-in security principals stored within the SAM and Builtin branches.

```
[+] RID-style keys found
    name=000001F4 RID=500 rel=0x00000BF8 parent=0x00000B70 values=1
    name=00000220 RID=544 rel=0x00000D28 parent=0x000009D8 values=1
    name=00000221 RID=545 rel=0x00001050 parent=0x000009D8 values=1
    name=000001F5 RID=501 rel=0x000012E8 parent=0x00000B70 values=1
    name=00000222 RID=546 rel=0x000013F0 parent=0x000009D8 values=1
    name=000001F7 RID=503 rel=0x000016E8 parent=0x00000B70 values=1
    name=00000245 RID=581 rel=0x000017F0 parent=0x000009D8 values=1
    name=00000247 RID=583 rel=0x00001AE0 parent=0x000009D8 values=1
    name=00000227 RID=551 rel=0x00001D80 parent=0x000009D8 values=1
    name=00000228 RID=552 rel=0x00001EA0 parent=0x000009D8 values=1
    name=0000022F RID=559 rel=0x00001F80 parent=0x000009D8 values=1
    name=00000223 RID=547 rel=0x000023A0 parent=0x000009D8 values=1
    name=00000242 RID=578 rel=0x00002880 parent=0x000009D8 values=1
    name=00000243 RID=579 rel=0x000028F8 parent=0x000009D8 values=1
    name=00000201 RID=513 rel=0x00002DB0 parent=0x00002A90 values=1
    name=000001F4 RID=500 rel=0x00002FF0 parent=0x00002990 values=3
    name=000001F5 RID=501 rel=0x000033E8 parent=0x00002990 values=3
    name=0000022B RID=555 rel=0x00003500 parent=0x000009D8 values=1
    name=0000022C RID=556 rel=0x000035C8 parent=0x000009D8 values=1
    name=000001F7 RID=503 rel=0x000037C8 parent=0x00002990 values=4
    name=0000022E RID=558 rel=0x00003A80 parent=0x000009D8 values=1
    name=00000232 RID=562 rel=0x00003EE8 parent=0x000009D8 values=1
    name=00000238 RID=568 rel=0x00003FA0 parent=0x000009D8 values=1
    name=00000239 RID=569 rel=0x00004E28 parent=0x000009D8 values=1
    name=00000011 RID=17 rel=0x00005130 parent=0x000050D8 values=1
    name=0000023D RID=573 rel=0x00005268 parent=0x000009D8 values=1
    name=00000244 RID=580 rel=0x00005470 parent=0x000009D8 values=1
    name=00000004 RID=4 rel=0x00005A38 parent=0x000050D8 values=1
    name=0000000B RID=11 rel=0x00005A90 parent=0x000050D8 values=1
    name=000001F8 RID=504 rel=0x00005CB0 parent=0x00002990 values=5
    name=000003E8 RID=1000 rel=0x00006020 parent=0x00002990 values=4
    name=000003E8 RID=1000 rel=0x000060D0 parent=0x00000B70 values=1
```

Not every RID-style key corresponds to a normal user account. Some represent built-in groups or aliases. The important user-account RIDs in this case are 500, 501, 503, 504, and 1000, which are later parsed from `SAM\Domains\Account\Users`. This grouping is useful because it shows that the parser did not rely only on one path but also detected RID-formatted keys during broader scanning.

C.6 Likely User and Group Names

This group lists account names and group names identified during scanning of the hive.

```
[+] Likely user/group names
    SAM
    Domains
    Builtin
    Users
    Names
    Groups
    Aliases
    Members
    S-1-5-21-3017025782-4202564144-487505969
    000001F4
    Administrators
    Guests
    Backup Operators
    Replicator
    Remote Desktop Users
    Power Users
    000001F5
    000001F7
    Device Owners
    DefaultAccount
    Event Log Readers
    None
    0000022F
    Account
    Hyper-V Administrators
    Administrator
    Network Configuration Operators
    Guest
    0000022B
    Performance Monitor Users
    0000022C
    Remote Management Users
    Performance Log Users
    0000022E
    Distributed COM Users
    IIS_IUSRS
    LastSkuUpgrade
    Cryptographic Operators
    Access Control Assistance Operators
    S-1-5
    System Managed Accounts Group
    0000023D
    WDAGUtilityAccount
    0000000B
    Angelo Calpoporo
    000001F8
    000003E8
```

This output shows that the parser successfully recovered both built-in principals and actual account names. This strengthens confidence that the hive contains interpretable account metadata and that the parser can identify both usernames and security-related group entries.

C.7 Extraction of User Account Records from `SAM\Domains\Account\Users`

This group shows the actual user-account extraction step. It confirms that the `Users` key was found and that the authoritative username-to-RID mapping from the `Names` subkey was recovered.

```
[*] Extracting user account data from SAM\Domains\Account\Users
[+] Found Users key at rel=0x00002990
[+] Found 6 subkeys under Users
[+] Authoritative username mapping from Names subkey:
    RID 500 -> Administrator
    RID 501 -> Guest
    RID 503 -> DefaultAccount
    RID 504 -> WDAGUtilityAccount
    RID 1000 -> Angelo Calpoporo
```

This is the most important account-discovery step because it uses the proper `Names` subkey inside the SAM account database. The mapping confirms the five user accounts analyzed in the project. It also distinguishes actual user account entries from unrelated group or alias entries seen during broader scanning.

C.8 Parsed User Account – RID 500 (Administrator)

RID 500 is the default built-in Administrator account in Windows. The F value shows that the account is **disabled**, marked as a **normal account**, and its password is configured to **never expire**. The logon-related timestamps are empty, which indicates that no recorded login activity was stored for this account in the parsed SAM hive. The logon count and failed logon count are both zero.

```
[+] RID key: 000001F4 (RID 500)
    Path      : ROOT\SAM\Domains\Account\Users\000001F4
    Timestamp : 2026-02-04 00:17:05
    Values    : 3
      VK name='F' type=REG_BINARY len=80 inline=False
      VK name='V' type=REG_BINARY len=688 inline=False
      VK name='SupplementalCredentials' type=REG_BINARY len=64 inline=False

F value for 000001F4 (RID 500) (offset 0x4080):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
4080    a8 ff ff ff 03 00 01 00 00 00 00 00 00 00 00 00 ................
4090    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
40a0    00 00 00 00 ff ff ff ff ff ff ff ff 7f 00 00 00 ................
40b0    00 00 00 00 f4 01 00 00 01 02 00 00 11 02 00 00 ................
40c0    00 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00 ................
        RID in F            : 500
        Account flags       : 0x00000211 (DISABLED, NORMAL_ACCOUNT, PASSWORD_NEVER_EXPIRES)
        Last logon          : N/A
        Last password set   : N/A
        Account expires     : N/A
        Last failed logon   : N/A
        Failed logon count  : 0
        Logon count         : 0
```

```
V value for 000001F4 (RID 500) (offset 0x4b38):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
4b38    48 fd ff ff 00 00 00 00 f4 00 00 00 03 00 01 00 H...............
4b48    f4 00 00 00 1a 00 00 00 00 00 00 00 10 01 00 00 ................
4b58    00 00 00 00 00 00 00 00 10 01 00 00 6c 00 00 00 ............l...
4b68    00 00 00 00 7c 01 00 00 00 00 00 00 00 00 00 00 ....|...........
4b78    7c 01 00 00 00 00 00 00 00 00 00 00 7c 01 00 00 |...........|...
4b88    00 00 00 00 00 00 00 00 7c 01 00 00 00 00 00 00 ........|.......
4b98    00 00 00 00 7c 01 00 00 00 00 00 00 00 00 00 00 ....|...........
4ba8    7c 01 00 00 00 00 00 00 00 00 00 00 7c 01 00 00 |...........|...
4bb8    00 00 00 00 00 00 00 00 7c 01 00 00 00 00 00 00 ........|.......
4bc8    00 00 00 00 7c 01 00 00 08 00 00 00 01 00 00 00 ....|...........
4bd8    84 01 00 00 18 00 00 00 00 00 00 00 9c 01 00 00 ................
4be8    18 00 00 00 00 00 00 00 b4 01 00 00 18 00 00 00 ................
4bf8    00 00 00 00 cc 01 00 00 18 00 00 00 00 00 00 00 ................
4c08    01 00 14 80 d4 00 00 00 e4 00 00 00 14 00 00 00 ................
4c18    44 00 00 00 02 00 30 00 02 00 00 00 02 c0 14 00 D.....0.........
4c28    44 00 05 01 01 01 00 00 00 00 00 01 00 00 00 00 D...............
4c38    02 c0 14 00 ff ff 1f 00 01 01 00 00 00 00 00 05 ................
4c48    07 00 00 00 02 00 90 00 04 00 00 00 00 00 14 00 ................
4c58    5b 03 02 00 01 01 00 00 00 00 00 01 00 00 00 00 [...............
4c68    00 00 18 00 ff 07 0f 00 01 02 00 00 00 00 00 05 ................
4c78    20 00 00 00 20 02 00 00 00 00 38 00 1b 03 02 00  ... .....8.....
4c88    01 0a 00 00 00 00 00 0f 03 00 00 00 00 04 00 00 ................
4c98    de a2 28 67 21 3e d2 af 19 ad 5d 79 b0 c1 07 29 ..(g!>....]y...)
4ca8    27 56 fc 20 d8 ad 66 f6 10 f2 68 fa df 2a f8 0f 'V. ..f...h..*..
4cb8    00 00 24 00 44 00 02 00 01 05 00 00 00 00 00 05 ..$.D...........
4cc8    15 00 00 00 f6 28 d4 b3 30 0a 7e fa 31 c0 0e 1d .....(..0.~.1...
4cd8    f4 01 00 00 01 02 00 00 00 00 00 05 20 00 00 00 ............ ...
4ce8    20 02 00 00 01 02 00 00 00 00 00 05 20 00 00 00  ........... ...
4cf8    20 02 00 00 41 00 64 00 6d 00 69 00 6e 00 69 00  ...A.d.m.i.n.i.
4d08    73 00 74 00 72 00 61 00 74 00 6f 00 72 00 1c 09 s.t.r.a.t.o.r...
4d18    42 00 75 00 69 00 6c 00 74 00 2d 00 69 00 6e 00 B.u.i.l.t.-.i.n.
4d28    20 00 61 00 63 00 63 00 6f 00 75 00 6e 00 74 00  .a.c.c.o.u.n.t.
4d38    20 00 66 00 6f 00 72 00 20 00 61 00 64 00 6d 00  .f.o.r. .a.d.m.
4d48    69 00 6e 00 69 00 73 00 74 00 65 00 72 00 69 00 i.n.i.s.t.e.r.i.
4d58    6e 00 67 00 20 00 74 00 68 00 65 00 20 00 63 00 n.g. .t.h.e. .c.
4d68    6f 00 6d 00 70 00 75 00 74 00 65 00 72 00 2f 00 o.m.p.u.t.e.r./.
4d78    64 00 6f 00 6d 00 61 00 69 00 6e 00 01 02 00 00 d.o.m.a.i.n.....
4d88    07 00 00 00 02 00 02 00 00 00 00 00 45 96 08 0c ............E...
4d98    3d 30 e6 05 25 37 fc 18 6d 8e 5f 3c 02 00 02 00 =0..%7..m._<....
4da8    00 00 00 00 1d e7 52 38 3b c8 e9 99 50 73 8c 8d ......R8;...Ps..
4db8    80 56 7e 53 02 00 02 00 00 00 00 00 8e 34 9a 92 .V~S.........4..
4dc8    e4 f6 39 65 ba d5 7b 8c 9f bf 40 46 02 00 02 00 ..9e..{...@F....
4dd8    00 00 00 00 33 74 66 13 7f 12 e2 32 e1 24 79 d1 ....3tf....2.$y.
        [Auto] Found username at offset 0x1C4
        Username   : Administrator
        LM blob    : 0102000007000000 (raw 16-byte candidate; may be encrypted)
        NT blob    : 020002000000000004596080c3d30e6052537fc186d8e5f3c (raw 16-byte candidate; may be encrypted)
```

```
============================================================
FIELD MAPPING FOR RID 500 (Administrator)
============================================================
Field                      Abs Offset   Bytes (hex)
-----------------------------------------------------------------------------------------
NK record start            0x3FF0       a8 ff ff ff ...
  NK timestamp             0x3FF8       99 be 9d 97 6b 95 dc 01
F value start              0x4080       a8 ff ff ff ...
  Cell size                0x4080       a8 ff ff ff
  RID                      0x40B4       f4 01 00 00
  Flags                    0x40BC       11 02 00 00
  Last logon               0x408C       00 00 00 00 00 00 00 00
  Last pwd set             0x409C       00 00 00 00 00 00 00 00
  Account expires          0x40A4       ff ff ff ff ff ff ff 7f
  Last failed logon        0x40AC       00 00 00 00 00 00 00 00
  Failed logon count       0x40C4       00 00 00 00
  Logon count              0x40C6       00 00 01 00
V value start              0x4B38       48 fd ff ff ...
  Cell size                0x4B38       48 fd ff ff
  Username                 0x4CFC       41 00 64 00 6d 00 69 00 6e 00 69 00 73 00 74 00 ...
  LM hash (candidate)      0x4D84       01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00
  NT hash (candidate)      0x4D8C       02 00 02 00 00 00 00 00 45 96 08 0c 3d 30 e6 05
============================================================
```

The built-in Administrator account exists by default in Windows systems. In this case, its metadata suggests it was not actively used. The zero logon count and null timestamps support the interpretation that the account remained inactive. The presence of encrypted candidate LM and NT hash blobs confirms that authentication material is stored for the account even though the plaintext password is not recoverable from the SAM alone.

C.9 Parsed User Account – RID 501 (Guest)

RID 501 corresponds to the built-in Guest account. Its flags show that it is **disabled**, **password not required**, **normal account**, and **password never expires**. Similar to the Administrator account, its timestamps are empty and both failed and successful logon counts are zero.

```
[+] RID key: 000001F5 (RID 501)
    Path      : ROOT\SAM\Domains\Account\Users\000001F5
    Timestamp : 2026-02-04 00:17:05
    Values    : 3
      VK name='F' type=REG_BINARY len=80 inline=False
      VK name='V' type=REG_BINARY len=664 inline=False
      VK name='SupplementalCredentials' type=REG_BINARY len=64 inline=False

F value for 000001F5 (RID 501) (offset 0x4488):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
4488    a8 ff ff ff 03 00 01 00 00 00 00 00 00 00 00 00 ................
4498    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
44a8    00 00 00 00 ff ff ff ff ff ff ff ff 7f 00 00 00 ................
44b8    00 00 00 00 f5 01 00 00 01 02 00 00 15 02 00 00 ................
44c8    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
        RID in F           : 501
        Account flags      : 0x00000215 (DISABLED, PASSWORD_NOT_REQUIRED, NORMAL_ACCOUNT, PASSWORD_NEVER_EXPIRES)
        Last logon         : N/A
        Last password set  : N/A
        Account expires    : N/A
        Last failed logon  : N/A
        Failed logon count : 0
        Logon count        : 0
```

```
V value for 000001F5 (RID 501) (offset 0x52d0):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
52d0    60 fd ff ff 00 00 00 00 e8 00 00 00 03 00 01 00 `...............
52e0    e8 00 00 00 0a 00 00 00 00 00 00 00 f4 00 00 00 ................
52f0    00 00 00 00 00 00 00 00 f4 00 00 00 70 00 00 00 ............p...
5300    00 00 00 00 64 01 00 00 00 00 00 00 00 00 00 00 ....d...........
5310    64 01 00 00 00 00 00 00 00 00 00 00 64 01 00 00 d...........d...
5320    00 00 00 00 00 00 00 00 64 01 00 00 00 00 00 00 ........d.......
5330    00 00 00 00 64 01 00 00 00 00 00 00 00 00 00 00 ....d...........
5340    64 01 00 00 00 00 00 00 00 00 00 00 64 01 00 00 d...........d...
5350    00 00 00 00 00 00 00 00 64 01 00 00 00 00 00 00 ........d.......
5360    00 00 00 00 64 01 00 00 08 00 00 00 01 00 00 00 ....d...........
5370    6c 01 00 00 18 00 00 00 00 00 00 00 84 01 00 00 l...............
5380    18 00 00 00 00 00 00 00 9c 01 00 00 18 00 00 00 ................
5390    00 00 00 00 b4 01 00 00 18 00 00 00 00 00 00 00 ................
53a0    01 00 14 80 c8 00 00 00 d8 00 00 00 14 00 00 00 ................
53b0    44 00 00 00 02 00 30 00 02 00 00 00 02 c0 14 00 D.....0.........
53c0    44 00 05 01 01 01 00 00 00 00 00 01 00 00 00 00 D...............
53d0    02 c0 14 00 ff ff 1f 00 01 01 00 00 00 00 00 05 ................
53e0    07 00 00 00 02 00 84 00 04 00 00 00 00 00 14 00 ................
53f0    1b 03 02 00 01 01 00 00 00 00 00 01 00 00 00 00 ................
5400    00 00 18 00 ff 07 0f 00 01 02 00 00 00 00 00 05 ................
5410    20 00 00 00 20 02 00 00 00 00 18 00 ff 07 0f 00  ... ...........
5420    01 02 00 00 00 00 00 05 20 00 00 00 24 02 00 00 ........ ...$...
5430    00 00 38 00 1b 03 02 00 01 0a 00 00 00 00 00 0f ..8.............
5440    03 00 00 00 00 04 00 00 de a2 28 67 21 3e d2 af ..........(g!>..
5450    19 ad 5d 79 b0 c1 07 29 27 56 fc 20 d8 ad 66 f6 ..]y...)'V. ..f.
5460    10 f2 68 fa df 2a f8 0f 01 02 00 00 00 00 00 05 ..h..*..........
5470    20 00 00 00 20 02 00 00 01 02 00 00 00 00 00 05  ... ...........
5480    20 00 00 00 20 02 00 00 47 00 75 00 65 00 73 00  ... ...G.u.e.s.
5490    74 00 00 00 42 00 75 00 69 00 6c 00 74 00 2d 00 t...B.u.i.l.t.-.
54a0    69 00 6e 00 20 00 61 00 63 00 63 00 6f 00 75 00 i.n. .a.c.c.o.u.
54b0    6e 00 74 00 20 00 66 00 6f 00 72 00 20 00 67 00 n.t. .f.o.r. .g.
54c0    75 00 65 00 73 00 74 00 20 00 61 00 63 00 63 00 u.e.s.t. .a.c.c.
54d0    65 00 73 00 73 00 20 00 74 00 6f 00 20 00 74 00 e.s.s. .t.o. .t.
54e0    68 00 65 00 20 00 63 00 6f 00 6d 00 70 00 75 00 h.e. .c.o.m.p.u.
54f0    74 00 65 00 72 00 2f 00 64 00 6f 00 6d 00 61 00 t.e.r./.d.o.m.a.
5500    69 00 6e 00 01 02 00 00 07 00 00 00 02 00 02 00 i.n.............
5510    00 00 00 00 e7 6c f1 a8 44 83 09 69 4f f4 09 50 .....l..D..iO..P
5520    fe 43 6e 9c 02 00 02 00 00 00 00 00 fc 1f 8e ca .Cn.............
5530    18 4f 32 49 7c 27 6e aa 7c 40 e5 e7 02 00 02 00 .O2I|'n.|@......
5540    00 00 00 00 9a b4 60 e3 d7 f8 4c 72 a6 bd 3e 5a ......`...Lr..>Z
5550    04 6e 45 43 02 00 02 00 00 00 00 00 b0 62 44 41 .nEC.........bDA
5560    26 bc 3f 02 4c 29 c5 ed                          &.?.L)..
        [Auto] Found username at offset 0x1B8
        Username  : Guest
        LM blob   : 0102000007000000 (raw 16-byte candidate; may be encrypted)
        NT blob   : 02000200000000000e76cf1a8448309694ff40950fe436e9c (raw 16-byte candidate; may be encrypted)
```

```
============================================================
FIELD MAPPING FOR RID 501 (Guest)
============================================================
Field                    Abs Offset   Bytes (hex)
-----------------------------------------------------------------------------------------
NK record start          0x43E8       a8 ff ff ff ...
  NK timestamp           0x43F0       99 be 9d 97 6b 95 dc 01
F value start            0x4488       a8 ff ff ff ...
  Cell size              0x4488       a8 ff ff ff
  RID                    0x44BC       f5 01 00 00
  Flags                  0x44C4       15 02 00 00
  Last logon             0x4494       00 00 00 00 00 00 00 00
  Last pwd set           0x44A4       00 00 00 00 00 00 00 00
  Account expires        0x44AC       ff ff ff ff ff ff ff 7f
  Last failed logon      0x44B4       00 00 00 00 00 00 00 00
  Failed logon count     0x44CC       00 00 00 00
  Logon count            0x44CE       00 00 00 00
V value start            0x52D0       60 fd ff ff ...
  Cell size              0x52D0       60 fd ff ff
  Username               0x5488       47 00 75 00 65 00 73 00 74 00 ...
  LM hash (candidate)    0x5504       01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00
  NT hash (candidate)    0x550C       02 00 02 00 00 00 00 00 e7 6c f1 a8 44 83 09 69
============================================================
```

The Guest account is normally disabled in modern Windows systems. The metadata in the F value confirms that status here. The lack of successful or failed login activity suggests that the account was not used. From a forensic perspective, this is important because a Guest account with unexpected activity may indicate misuse or unauthorized access.

C.10 Parsed User Account – RID 503 (DefaultAccount)

RID 503 corresponds to `DefaultAccount`, a special internal Windows-managed account. Its flags indicate that it is **disabled**, **password not required**, **normal account**, and **password never expires**. All major activity counters and timestamps are empty.

```
[+] RID key: 000001F7 (RID 503)
    Path      : ROOT\SAM\Domains\Account\Users\000001F7
    Timestamp : 2026-02-04 00:17:05
    Values    : 4
      VK name='F' type=REG_BINARY len=80 inline=False
      VK name='V' type=REG_BINARY len=680 inline=False
      VK name='UserDontShowInLogonUI' type=REG_BINARY len=4 inline=True
      VK name='SupplementalCredentials' type=REG_BINARY len=64 inline=False

F value for 000001F7 (RID 503) (offset 0x4840):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
4840    a8 ff ff ff 03 00 01 00 00 00 00 00 00 00 00 00 ................
4850    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
4860    00 00 00 00 ff ff ff ff ff ff ff ff 7f 00 00 00 ................
4870    00 00 00 00 f7 01 00 00 01 02 00 00 15 02 00 00 ................
4880    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
        RID in F           : 503
        Account flags      : 0x00000215 (DISABLED, PASSWORD_NOT_REQUIRED, NORMAL_ACCOUNT, PASSWORD_NEVER_EXPIRES)
        Last logon         : N/A
        Last password set  : N/A
        Account expires    : N/A
        Last failed logon  : N/A
        Failed logon count : 0
        Logon count        : 0
```

```
V value for 000001F7 (RID 503) (offset 0x5020):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
5020    50 fd ff ff 00 00 00 00 0c 01 00 00 03 00 01 00 P...............
5030    0c 01 00 00 1c 00 00 00 00 00 00 00 28 01 00 00 ............(...
5040    00 00 00 00 00 00 00 00 28 01 00 00 4a 00 00 00 ........(...J...
5050    00 00 00 00 74 01 00 00 00 00 00 00 00 00 00 00 ....t...........
5060    74 01 00 00 00 00 00 00 00 00 00 00 74 01 00 00 t...........t...
5070    00 00 00 00 00 00 00 00 74 01 00 00 00 00 00 00 ........t.......
5080    00 00 00 00 74 01 00 00 00 00 00 00 00 00 00 00 ....t...........
5090    74 01 00 00 00 00 00 00 00 00 00 00 74 01 00 00 t...........t...
50a0    00 00 00 00 00 00 00 00 74 01 00 00 00 00 00 00 ........t.......
50b0    00 00 00 00 74 01 00 00 08 00 00 00 01 00 00 00 ....t...........
50c0    7c 01 00 00 18 00 00 00 00 00 00 00 94 01 00 00 |...............
50d0    18 00 00 00 00 00 00 00 ac 01 00 00 18 00 00 00 ................
50e0    00 00 00 00 c4 01 00 00 18 00 00 00 00 00 00 00 ................
50f0    01 00 14 80 ec 00 00 00 fc 00 00 00 14 00 00 00 ................
5100    44 00 00 00 02 00 30 00 02 00 00 00 02 c0 14 00 D.....0.........
5110    44 00 05 01 01 01 00 00 00 00 00 01 00 00 00 00 D...............
5120    02 c0 14 00 ff ff 1f 00 01 01 00 00 00 00 00 05 ................
5130    07 00 00 00 02 00 a8 00 05 00 00 00 00 00 14 00 ................
5140    5b 03 02 00 01 01 00 00 00 00 00 01 00 00 00 00 [...............
5150    00 00 18 00 ff 07 0f 00 01 02 00 00 00 00 00 05 ................
5160    20 00 00 00 20 02 00 00 00 00 18 00 ff 07 0f 00  ... ...........
5170    01 02 00 00 00 00 00 05 20 00 00 00 24 02 00 00 ........ ...$...
5180    00 00 38 00 1b 03 02 00 01 0a 00 00 00 00 00 0f ..8.............
5190    03 00 00 00 00 04 00 00 de a2 28 67 21 3e d2 af ..........(g!>..
51a0    19 ad 5d 79 b0 c1 07 29 27 56 fc 20 d8 ad 66 f6 ..]y...)'V. ..f.
51b0    10 f2 68 fa df 2a f8 0f 00 00 24 00 44 00 02 00 ..h..*....$.D...
51c0    01 05 00 00 00 00 00 05 15 00 00 00 f6 28 d4 b3 ............(..
51d0    30 0a 7e fa 31 c0 0e 1d f7 01 00 00 01 02 00 00 0.~.1...........
51e0    00 00 00 05 20 00 00 00 20 02 00 00 01 02 00 00 .... ... .......
51f0    00 00 00 05 20 00 00 00 20 02 00 00 44 00 65 00 .... ... ...D.e.
5200    66 00 61 00 75 00 6c 00 74 00 41 00 63 00 63 00 f.a.u.l.t.A.c.c.
5210    6f 00 75 00 6e 00 74 00 41 00 20 00 75 00 73 00 o.u.n.t.A. .u.s.
5220    65 00 72 00 20 00 61 00 63 00 63 00 6f 00 75 00 e.r. .a.c.c.o.u.
5230    6e 00 74 00 20 00 6d 00 61 00 6e 00 61 00 67 00 n.t. .m.a.n.a.g.
5240    65 00 64 00 20 00 62 00 79 00 20 00 74 00 68 00 e.d. .b.y. .t.h.
5250    65 00 20 00 73 00 79 00 73 00 74 00 65 00 6d 00 e. .s.y.s.t.e.m.
5260    2e 00 69 00 01 02 00 00 07 00 00 00 02 00 02 00 ..i.............
5270    00 00 00 00 b5 5f 32 e2 e0 06 2e f5 91 be ad ca ....._2.........
5280    bf 53 3c 1d 02 00 02 00 00 00 00 00 bd b8 99 ba .S<.............
5290    1e fc 5c ac 5f a6 ff 92 56 01 d8 23 02 00 02 00 ..\._...V..#....
52a0    00 00 00 00 53 2c c3 92 30 3e d1 51 71 51 34 d9 ....S,..0>.QqQ4.
52b0    8e 8a 9f f7 02 00 02 00 00 00 00 00 e5 ab a3 6b ...............k
52c0    3e 84 3a c4 e6 f9 b8 1d                          >.:.....
        [Auto] Found username at offset 0x1DC
        Username  : DefaultAccount
        LM blob   : 0102000007000000 (raw 16-byte candidate; may be encrypted)
        NT blob   : 0200020000000000b55f32e2e0062ef591beadcabf533c1d (raw 16-byte candidate; may be encrypted)
```

```
============================================================
FIELD MAPPING FOR RID 503 (DefaultAccount)
============================================================
Field                     Abs Offset   Bytes (hex)
-----------------------------------------------------------------------------------------
NK record start           0x47C8       a8 ff ff ff ...
  NK timestamp            0x47D0       99 be 9d 97 6b 95 dc 01
F value start             0x4840       a8 ff ff ff ...
  Cell size               0x4840       a8 ff ff ff
  RID                     0x4874       f7 01 00 00
  Flags                   0x487C       15 02 00 00
  Last logon              0x484C       00 00 00 00 00 00 00 00
  Last pwd set            0x485C       00 00 00 00 00 00 00 00
  Account expires         0x4864       ff ff ff ff ff ff ff 7f
  Last failed logon       0x486C       00 00 00 00 00 00 00 00
  Failed logon count      0x4884       00 00 00 00
  Logon count             0x4886       00 00 00 00
V value start             0x5020       50 fd ff ff ...
  Cell size               0x5020       50 fd ff ff
  Username                0x51FC       44 00 65 00 66 00 61 00 75 00 6c 00 74 00 41 00 ...
  LM hash (candidate)     0x5264       01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00
  NT hash (candidate)     0x526C       02 00 02 00 00 00 00 00 b5 5f 32 e2 e0 06 2e f5
============================================================
```

The DefaultAccount is typically used internally by Windows and is not intended for normal interactive use. The absence of logon activity is therefore expected. Its presence in the SAM hive is normal and serves as evidence of standard system account provisioning rather than suspicious user activity.

C.11 Parsed User Account – RID 504 (WDAGUtilityAccount)

RID 504 corresponds to `WDAGUtilityAccount`. Its flags indicate **disabled** and **normal account** status. Unlike the previous system accounts, this account has a valid **Last Password Set** timestamp of `2026-02-04 00:16:09`, although the last logon remains empty and the logon count is zero.

```
[+] RID key: 000001F8 (RID 504)
    Path      : ROOT\SAM\Domains\Account\Users\000001F8
    Timestamp : 2026-02-04 00:17:05
    Values    : 5
      VK name='F' type=REG_BINARY len=80 inline=False
      VK name='V' type=REG_BINARY len=836 inline=False
      VK name='ForcePasswordReset' type=REG_BINARY len=4 inline=True
      VK name='SupplementalCredentials' type=REG_BINARY len=1168 inline=False
      VK name='UserDontShowInLogonUI' type=REG_BINARY len=4 inline=True

F value for 000001F8 (RID 504) (offset 0x3e60):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
3e60    a8 ff ff ff 03 00 01 00 00 00 00 00 00 00 00 00 ................
3e70    00 00 00 00 00 00 00 00 00 00 00 00 73 4c 42 76 ............sLBv
3e80    6b 95 dc 01 ff ff ff ff ff ff ff 7f 00 00 00 00 k...............
3e90    00 00 00 00 f8 01 00 00 01 02 00 00 11 00 00 00 ................
3ea0    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
        RID in F           : 504
        Account flags      : 0x00000011 (DISABLED, NORMAL_ACCOUNT)
        Last logon         : N/A
        Last password set  : 2026-02-04 00:16:09
        Account expires    : N/A
        Last failed logon  : N/A
        Failed logon count : 0
        Logon count        : 0
```

```
V value for 000001F8 (RID 504) (offset 0x84b8):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
84b8    b8 fc ff ff 00 00 00 00 0c 01 00 00 03 00 01 00 ................
84c8    0c 01 00 00 24 00 00 00 00 00 00 00 30 01 00 00 ....$.......0...
84d8    00 00 00 00 00 00 00 00 30 01 00 00 be 00 00 00 ........0.......
84e8    00 00 00 00 f0 01 00 00 00 00 00 00 00 00 00 00 ................
84f8    f0 01 00 00 00 00 00 00 00 00 00 00 f0 01 00 00 ................
8508    00 00 00 00 00 00 00 00 f0 01 00 00 00 00 00 00 ................
8518    00 00 00 00 f0 01 00 00 00 00 00 00 00 00 00 00 ................
8528    f0 01 00 00 00 00 00 00 00 00 00 00 f0 01 00 00 ................
8538    00 00 00 00 00 00 00 00 f0 01 00 00 00 00 00 00 ................
8548    00 00 00 00 f0 01 00 00 08 00 00 00 01 00 00 00 ................
8558    f8 01 00 00 18 00 00 00 00 00 00 00 10 02 00 00 ................
8568    38 00 00 00 00 00 00 00 48 02 00 00 18 00 00 00 8.......H.......
8578    00 00 00 00 60 02 00 00 18 00 00 00 00 00 00 00 ....`...........
8588    01 00 14 80 ec 00 00 00 fc 00 00 00 14 00 00 00 ................
8598    44 00 00 00 02 00 30 00 02 00 00 00 02 c0 14 00 D.....0.........
85a8    44 00 05 01 01 01 00 00 00 00 00 01 00 00 00 00 D...............
85b8    02 c0 14 00 ff ff 1f 00 01 01 00 00 00 00 00 05 ................
85c8    07 00 00 00 02 00 a8 00 05 00 00 00 00 00 14 00 ................
85d8    5b 03 02 00 01 01 00 00 00 00 00 01 00 00 00 00 [...............
85e8    00 00 18 00 ff 07 0f 00 01 02 00 00 00 00 00 05 ................
85f8    20 00 00 00 20 02 00 00 00 00 18 00 ff 07 0f 00  ... ...........
8608    01 02 00 00 00 00 00 05 20 00 00 00 24 02 00 00 ........ ...$...
8618    00 00 38 00 1b 03 02 00 01 0a 00 00 00 00 00 0f ..8.............
8628    03 00 00 00 00 04 00 00 de a2 28 67 21 3e d2 af ..........(g!>..
8638    19 ad 5d 79 b0 c1 07 29 27 56 fc 20 d8 ad 66 f6 ..]y...)'V. ..f.
8648    10 f2 68 fa df 2a f8 0f 00 00 24 00 44 00 02 00 ..h..*....$.D...
8658    01 05 00 00 00 00 00 00 05 15 00 00 00 f6 28 d4 b3 .............(..
8668    30 0a 7e fa 31 c0 0e 1d f8 01 00 00 01 02 00 00 0.~.1...........
8678    00 00 00 05 20 00 00 00 00 20 02 00 00 01 02 00 00 .... ... .......
8688    00 00 00 05 20 00 00 00 20 02 00 00 57 00 44 00 .... ... ...W.D.
8698    41 00 47 00 55 00 74 00 69 00 6c 00 69 00 74 00 A.G.U.t.i.l.i.t.
86a8    79 00 41 00 63 00 63 00 6f 00 75 00 6e 00 74 00 y.A.c.c.o.u.n.t.
86b8    41 00 20 00 75 00 73 00 65 00 72 00 20 00 61 00 A. .u.s.e.r. .a.
86c8    63 00 63 00 6f 00 75 00 6e 00 74 00 20 00 6d 00 c.c.o.u.n.t. .m.
86d8    61 00 6e 00 61 00 67 00 65 00 64 00 20 00 61 00 a.n.a.g.e.d. .a.
86e8    6e 00 64 00 20 00 75 00 73 00 65 00 64 00 20 00 n.d. .u.s.e.d. .
86f8    62 00 79 00 20 00 74 00 68 00 65 00 20 00 73 00 b.y. .t.h.e. .s.
8708    79 00 73 00 74 00 65 00 6d 00 20 00 66 00 6f 00 y.s.t.e.m. .f.o.
8718    72 00 20 00 57 00 69 00 6e 00 64 00 6f 00 77 00 r. .W.i.n.d.o.w.
8728    73 00 20 00 44 00 65 00 66 00 65 00 6e 00 64 00 s. .D.e.f.e.n.d.
8738    65 00 72 00 20 00 41 00 70 00 70 00 6c 00 69 00 e.r. .A.p.p.l.i.
8748    63 00 61 00 74 00 69 00 6f 00 6e 00 20 00 47 00 c.a.t.i.o.n. .G.
8758    75 00 61 00 72 00 64 00 20 00 73 00 63 00 65 00 u.a.r.d. .s.c.e.
8768    6e 00 61 00 72 00 69 00 6f 00 73 00 2e 00 00 00 n.a.r.i.o.s.....
8778    01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00 ................
8788    3a 13 0f 7c bd 71 a7 da 90 15 e3 79 f3 56 e7 9b :..|.q.....y.V..
8798    02 00 02 00 10 00 00 00 29 fe bc a3 43 a4 4b c3 ........)...C.K.
87a8    ce 54 77 40 05 c1 cf 8b 6b b8 d6 25 5e a5 1c b5 .Tw@....k..%^...
87b8    84 45 3c b5 4d 4a f9 b5 97 25 8d 5c 9f 59 86 5f .E<.MJ...%.\.Y._
87c8    84 e7 d3 99 b4 f1 cc 6c 02 00 02 00 00 00 00 00 .......l........
87d8    d7 86 20 6b 1e 41 40 ee 4c b2 d4 c9 b5 14 03 37 .. k.A@.L......7
87e8    02 00 02 00 00 00 00 00 22 32 8a e0 35 cf 06 65 ........"2..5..e
87f8    da 79 3c f0                                     .y<.
```

```
87d8    d7 86 20 6b 1e 41 40 ee 4c b2 d4 c9 b5 14 03 37 .. k.A@.L......7
87e8    02 00 02 00 00 00 00 00 22 32 8a e0 35 cf 06 65 ........"2..5..e
87f8    da 79 3c f0                                     .y<.
        [Auto] Found username at offset 0x1DC
        Username   : WDAGUtilityAccount
        LM blob    : 0102000007000000 (raw 16-byte candidate; may be encrypted)
        NT blob    : 0200020000000003a130f7cbd71a7da9015e379f356e79b (raw 16-byte candidate; may be encrypted)

============================================================
FIELD MAPPING FOR RID 504 (WDAGUtilityAccount)
============================================================
Field                    Abs Offset   Bytes (hex)
-----------------------------------------------------------------------------------------
NK record start          0x6CB0       a8 ff ff ff ...
  NK timestamp           0x6CB8       99 be 9d 97 6b 95 dc 01
F value start            0x3E60       a8 ff ff ff ...
  Cell size              0x3E60       a8 ff ff ff
  RID                    0x3E94       f8 01 00 00
  Flags                  0x3E9C       11 00 00 00
  Last logon             0x3E6C       00 00 00 00 00 00 00 00
  Last pwd set           0x3E7C       73 4c 42 76 6b 95 dc 01
  Account expires        0x3E84       ff ff ff ff ff ff ff 7f
  Last failed logon      0x3E8C       00 00 00 00 00 00 00 00
  Failed logon count     0x3EA4       00 00 00 00
  Logon count            0x3EA6       00 00 00 00
V value start            0x84B8       b8 fc ff ff ...
  Cell size              0x84B8       b8 fc ff ff
  Username               0x8694       57 00 44 00 41 00 47 00 55 00 74 00 69 00 6c 00 ...
  LM hash (candidate)    0x8778       01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00
  NT hash (candidate)    0x8780       02 00 02 00 00 00 00 00 3a 13 0f 7c bd 71 a7 da
============================================================
```

The presence of a password-set timestamp but no actual logon activity suggests that the account was provisioned or updated by the operating system, not interactively used by a human user. This is consistent with the role of WDAGUtilityAccount as a Windows Defender Application Guard support account. This distinction is useful in forensic analysis because it helps separate automated system-managed activity from genuine user activity.

C.12 Parsed User Account – RID 1000 (Angelo Calpoporo)

RID 1000 corresponds to the primary active user account, Angelo Calpoporo. Its F value indicates:

- **Last logon:** 2026-02-13 06:29:34
- **Last password set:** 2026-02-03 08:17:42
- **Account flags:** 0x00000014 meaning **PASSWORD_NOT_REQUIRED** and **NORMAL_ACCOUNT**
- **Failed logon count:** 0
- **Logon count:** 5

```
[+] RID key: 000003E8 (RID 1000)
    Path      : ROOT\SAM\Domains\Account\Users\000003E8
    Timestamp : 2026-02-13 06:29:34
    Values    : 4
      VK name='F' type=REG_BINARY len=80 inline=False
      VK name='V' type=REG_BINARY len=640 inline=False
      VK name='ForcePasswordReset' type=REG_BINARY len=4 inline=True
      VK name='SupplementalCredentials' type=REG_BINARY len=1280 inline=False

F value for 000003E8 (RID 1000) (offset 0x7078):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
7078    a8 ff ff ff 03 00 01 00 00 00 00 00 c4 29 47 1e .............)G.
7088    b2 9c dc 01 00 00 00 00 00 00 00 00 9b 25 59 91 .............%Y.
7098    e5 94 dc 01 00 00 00 00 00 00 00 00 00 00 00 00 ................
70a8    00 00 00 00 e8 03 00 00 01 02 00 00 14 00 00 00 ................
70b8    01 00 e4 04 00 00 05 00 01 00 00 00 00 00 00 00 ................
        RID in F           : 1000
        Account flags      : 0x00000014 (PASSWORD_NOT_REQUIRED, NORMAL_ACCOUNT)
        Last logon         : 2026-02-13 06:29:34
        Last password set  : 2026-02-03 08:17:42
        Account expires    : N/A
        Last failed logon  : N/A
        Failed logon count : 0
        Logon count        : 5
```

```
V value for 000003E8 (RID 1000) (offset 0x79e0):
Offset  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  ASCII
79e0    60 fd ff ff 00 00 00 00 f4 00 00 00 03 00 01 00 `...............
79f0    f4 00 00 00 20 00 00 00 00 00 00 00 14 01 00 00 .... ...........
7a00    00 00 00 00 00 00 00 00 14 01 00 00 00 00 00 00 ................
7a10    00 00 00 00 14 01 00 00 00 00 00 00 00 00 00 00 ................
7a20    14 01 00 00 00 00 00 00 00 00 00 00 14 01 00 00 ................
7a30    00 00 00 00 00 00 00 00 14 01 00 00 00 00 00 00 ................
7a40    00 00 00 00 14 01 00 00 00 00 00 00 00 00 00 00 ................
7a50    14 01 00 00 00 00 00 00 00 00 00 00 14 01 00 00 ................
7a60    00 00 00 00 00 00 00 00 14 01 00 00 15 00 00 00 ................
7a70    a8 00 00 00 2c 01 00 00 08 00 00 00 01 00 00 00 ....,...........
7a80    34 01 00 00 18 00 00 00 00 00 00 00 4c 01 00 00 4...........L...
7a90    38 00 00 00 00 00 00 00 84 01 00 00 18 00 00 00 8...............
7aa0    00 00 00 00 9c 01 00 00 18 00 00 00 00 00 00 00 ................
7ab0    01 00 14 80 d4 00 00 00 e4 00 00 00 14 00 00 00 ................
7ac0    44 00 00 00 02 00 30 00 02 00 00 00 02 c0 14 00 D.....0.........
7ad0    44 00 05 01 01 01 00 00 00 00 00 01 00 00 00 00 D...............
7ae0    02 c0 14 00 ff 07 0f 00 01 01 00 00 00 00 00 05 ................
7af0    07 00 00 00 02 00 90 00 04 00 00 00 00 00 24 00 ..............$.
7b00    44 00 02 00 01 05 00 00 00 00 00 05 15 00 00 00 D...............
7b10    f6 28 d4 b3 30 0a 7e fa 31 c0 0e 1d e8 03 00 00 .(..0.~.1.......
7b20    00 00 38 00 1b 03 02 00 01 0a 00 00 00 00 00 0f ..8.............
7b30    03 00 00 00 00 04 00 00 de a2 28 67 21 3e d2 af ..........(g!>..
7b40    19 ad 5d 79 b0 c1 07 29 27 56 fc 20 d8 ad 66 f6 ..]y...)'V. ..f.
7b50    10 f2 68 fa df 2a f8 0f 00 00 18 00 ff 07 0f 00 ..h..*..........
7b60    01 02 00 00 00 00 00 05 20 00 00 00 20 02 00 00 ........ ... ...
7b70    00 00 14 00 5b 03 02 00 01 01 00 00 00 00 00 01 ....[...........
7b80    00 00 00 00 01 02 00 00 00 00 00 05 20 00 00 00 ............ ...
7b90    20 02 00 00 01 02 00 00 00 00 00 05 20 00 00 00  ........... ...
7ba0    20 02 00 00 41 00 6e 00 67 00 65 00 6c 00 6f 00  ...A.n.g.e.l.o.
7bb0    20 00 43 00 61 00 6c 00 70 00 6f 00 70 00 6f 00  .C.a.l.p.o.p.o.
7bc0    72 00 6f 00 ff ff ff ff ff ff ff ff ff ff ff ff r.o.............
7bd0    ff ff ff ff ff ff ff ff ff ff fb d5 05 01 02 00 00 ................
7be0    07 00 00 00 02 00 02 00 00 00 00 00 29 ed f9 89 ............)...
7bf0    18 c9 29 59 d7 a5 51 dd 4f 7f 15 fc 02 00 02 00 ..)Y..Q.O.......
7c00    10 00 00 00 8b ec 6d f2 87 3a cd fc 44 4f 8d d4 ......m..:..DO..
7c10    51 70 ae a0 78 d3 fc ed a5 48 56 3d 28 cf 20 0e Qp..x....HV=(. .
7c20    ab fe dc ba ac 95 34 20 42 c4 9c f3 44 29 e9 57 ......4 B...D).W
7c30    37 be 69 c9 02 00 02 00 00 00 00 00 df 26 b1 c8 7.i.........&..
7c40    a8 39 64 8d 11 a8 6f 0a f9 13 5d d0 02 00 02 00 .9d...o...]....
7c50    00 00 00 00 d2 49 67 21 e4 d8 91 60 33 d9 3c b5 .....Ig!...`3.<.
        [Auto] Found username at offset 0x1C4
        Username  : Angelo Calpoporo
        LM blob   : 0102000007000000 (raw 16-byte candidate; may be encrypted)
        NT blob   : 020002000000000029edf98918c92959d7a551dd4f7f15fc (raw 16-byte candidate; may be encrypted)
```

```
============================================================
FIELD MAPPING FOR RID 1000 (Angelo Calpoporo)
============================================================
Field                     Abs Offset   Bytes (hex)
--------------------------------------------------------------------------------------------
NK record start           0x7020       a8 ff ff ff ...
  NK timestamp            0x7028       c4 29 47 1e b2 9c dc 01
F value start             0x7078       a8 ff ff ff ...
  Cell size               0x7078       a8 ff ff ff
  RID                     0x70AC       e8 03 00 00
  Flags                   0x70B4       14 00 00 00
  Last logon              0x7084       c4 29 47 1e b2 9c dc 01
  Last pwd set            0x7094       9b 25 59 91 e5 94 dc 01
  Account expires         0x709C       00 00 00 00 00 00 00 00
  Last failed logon       0x70A4       00 00 00 00 00 00 00 00
  Failed logon count      0x70BC       00 00 05 00
  Logon count             0x70BE       05 00 01 00
V value start             0x79E0       60 fd ff ff ...
  Cell size               0x79E0       60 fd ff ff
  Username                0x7BA4       41 00 6e 00 67 00 65 00 6c 00 6f 00 20 00 43 00 ...
  LM hash (candidate)     0x7BDC       01 02 00 00 07 00 00 00 02 00 02 00 00 00 00 00
  NT hash (candidate)     0x7BE4       02 00 02 00 00 00 00 00 29 ed f9 89 18 c9 29 59
============================================================
```

This is the only account in the parsed output showing clear evidence of active use. The populated last logon timestamp and nonzero logon count indicate that this was the actual user account used on the machine. The absence of failed logons suggests no unsuccessful local login attempts were recorded in the parsed F value. This account therefore represents the primary source of user-authentication activity in the SAM hive.